



深度学习平台与应用

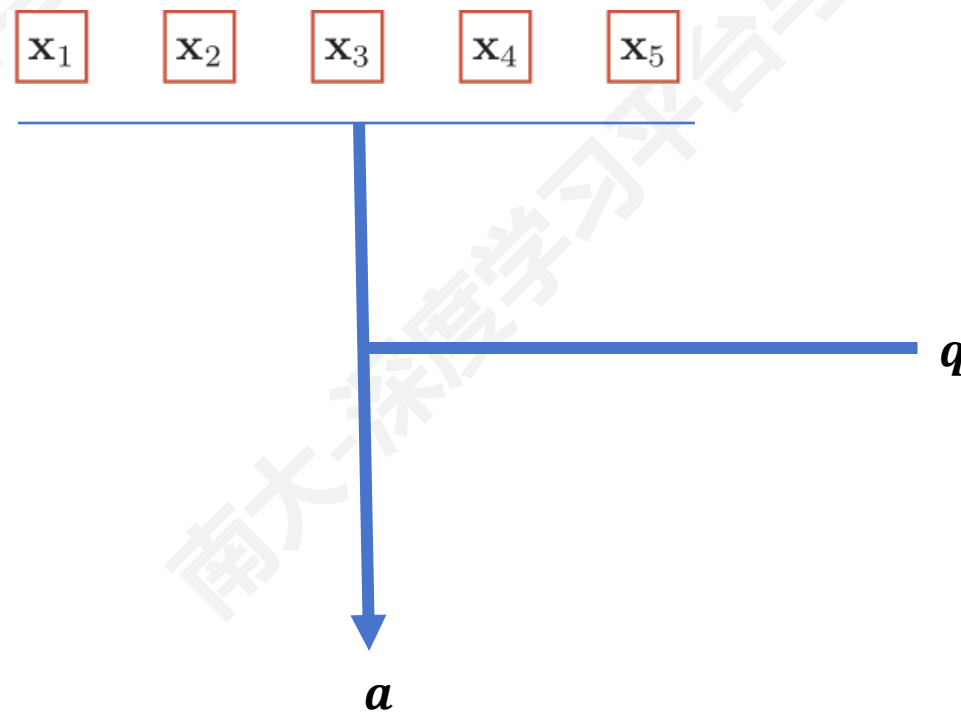
第九讲：注意力机制与 Transformer

大 纲

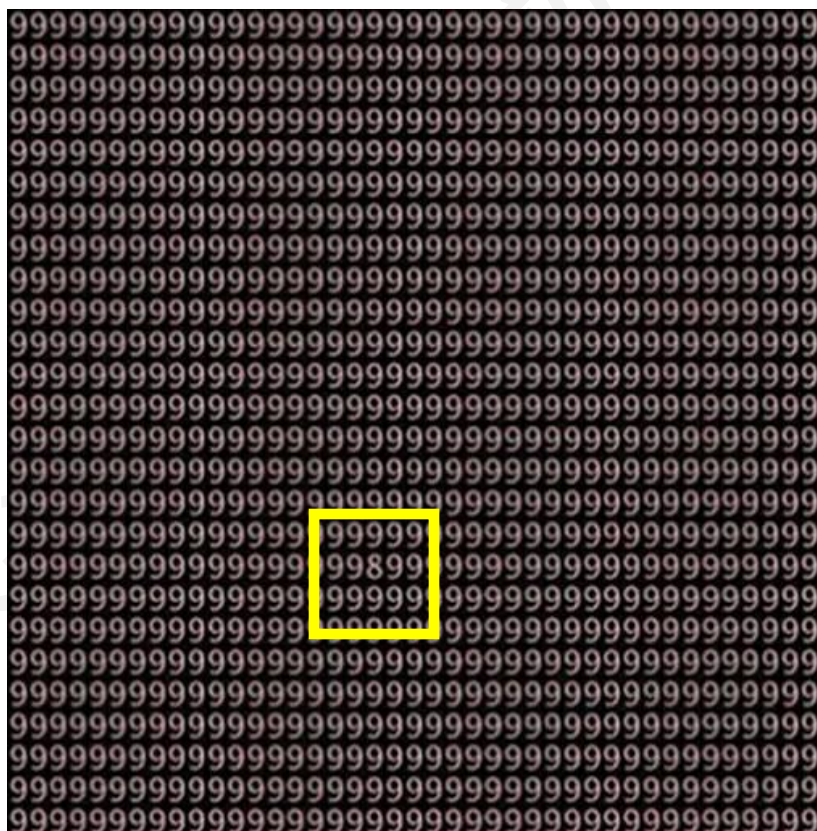
- **注意力机制**
- **Transformer**

南大-深度学习平台与应用

■ 在 8 秒内找到 “8”



■ 在 8 秒内找到 “8”



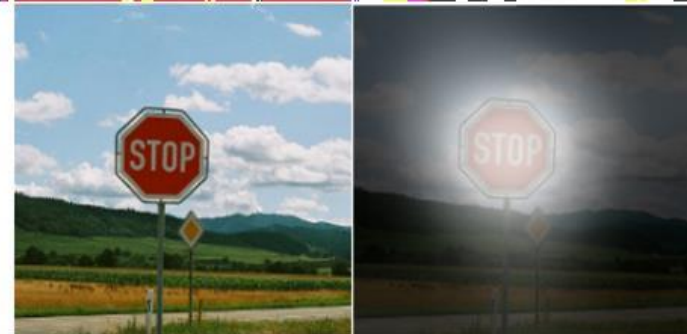
■ 视觉中的注意力机制



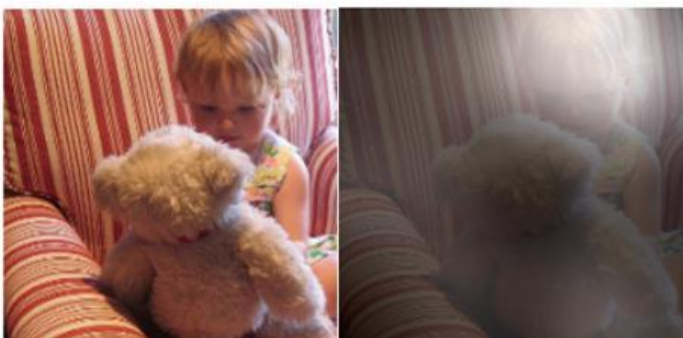
A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.

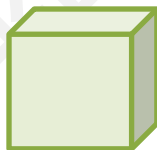
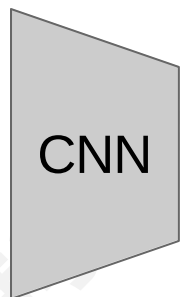


A giraffe standing in a forest with trees in the background.

■ CNN 中的注意力机制

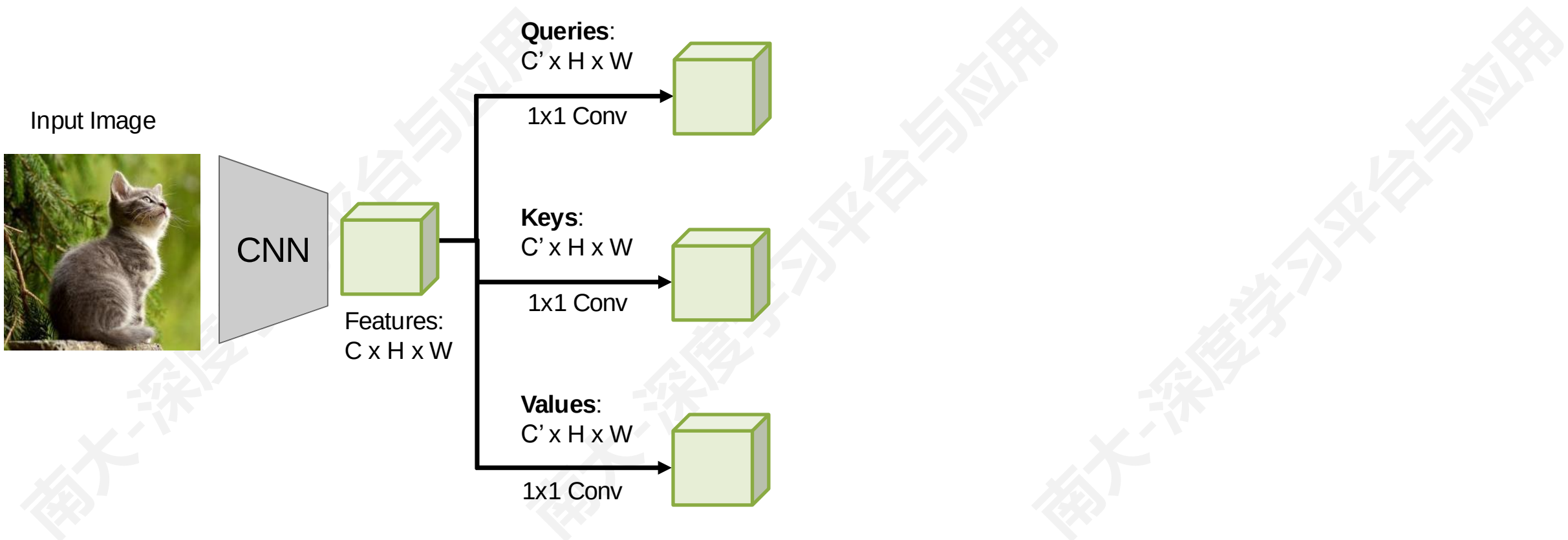
{ self
cross

Input Image

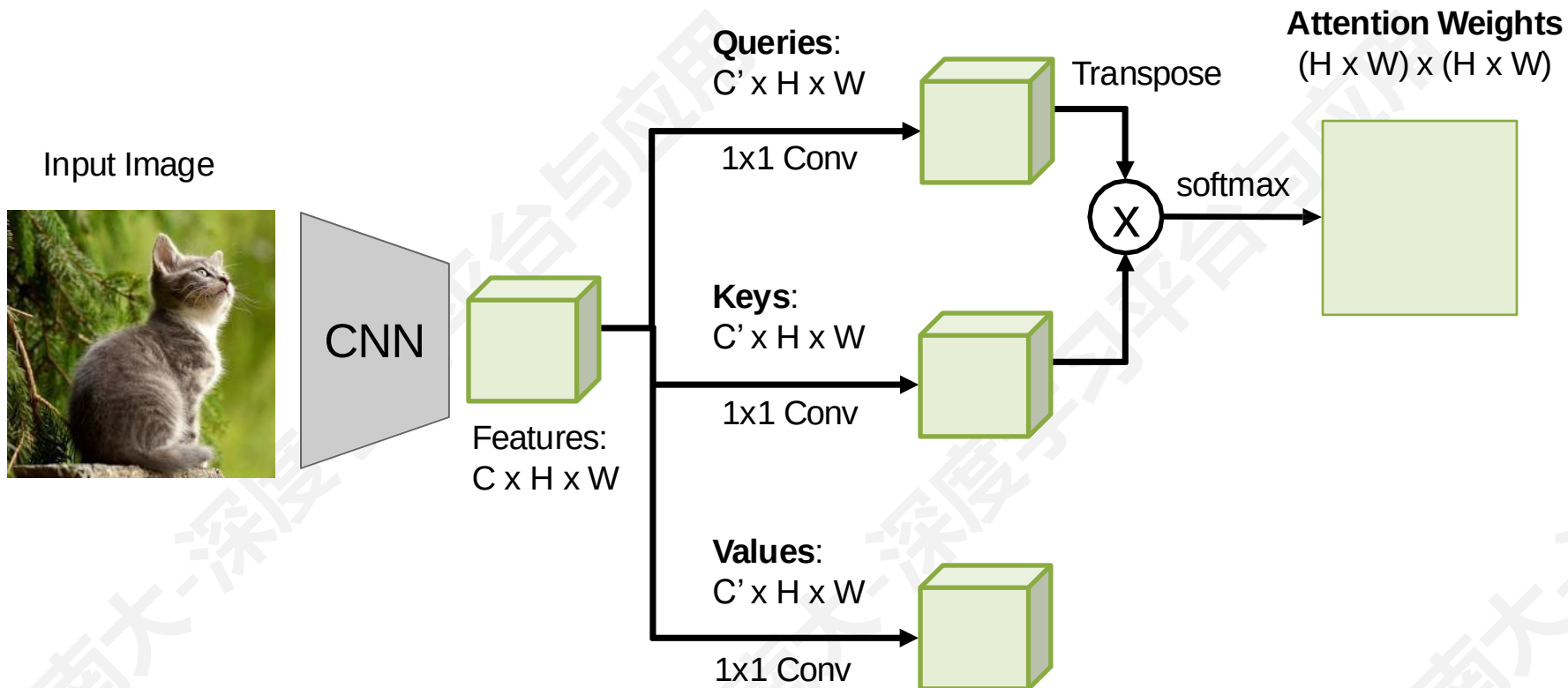


Features:
 $C \times H \times W$

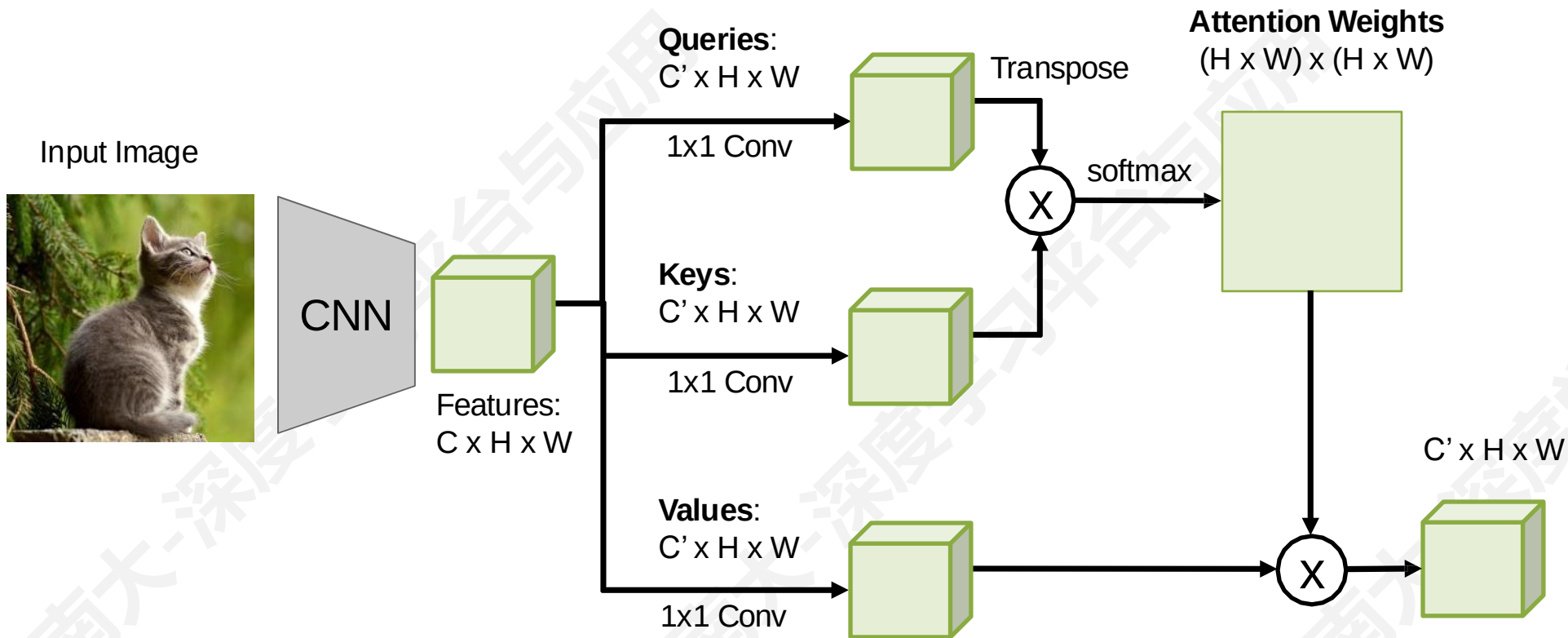
注意力机制



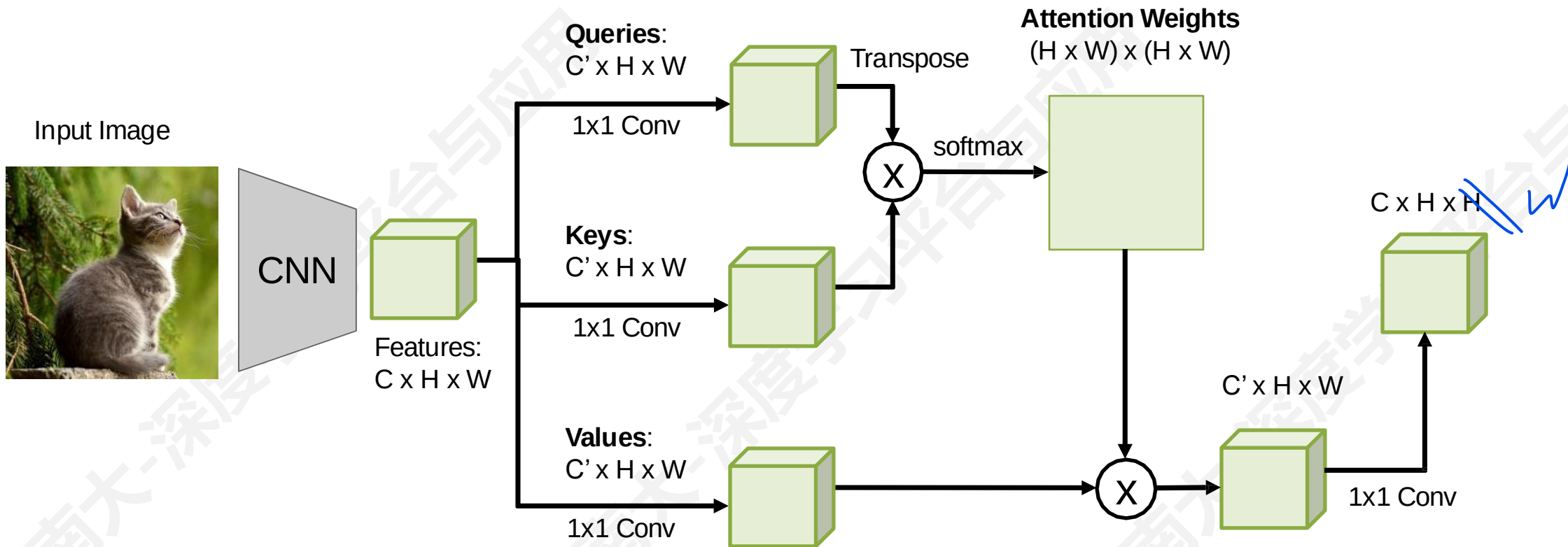
注意力机制



注意力机制



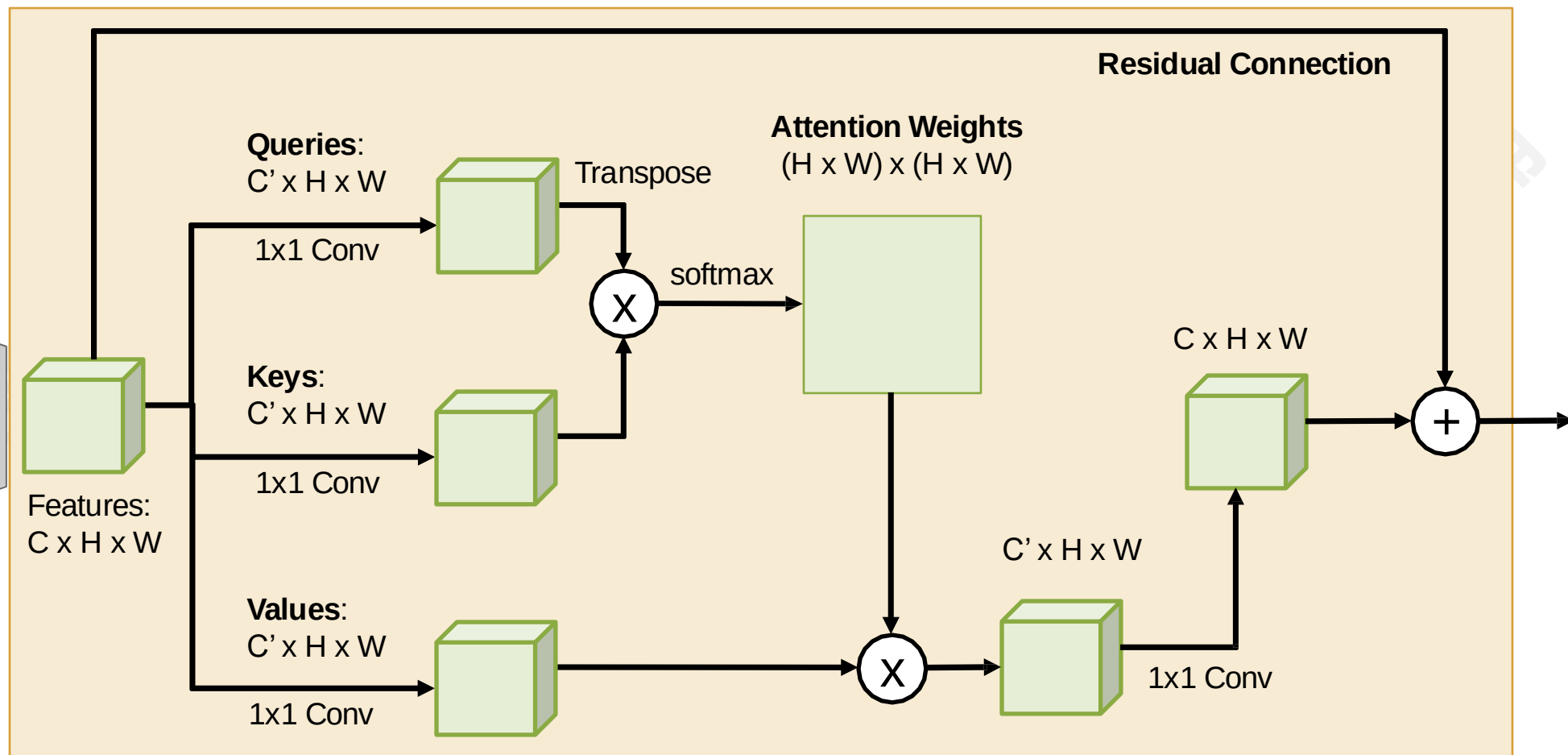
注意力机制



注意力机制



CNN



Self-Attention Module

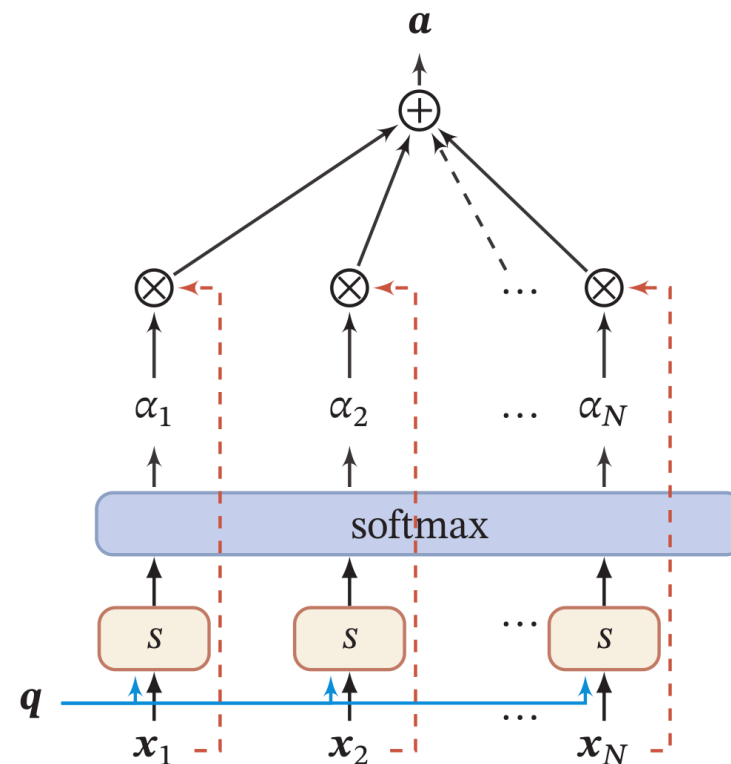
■ 注意力机制可以分两步

■ 1. 计算注意力分布 α

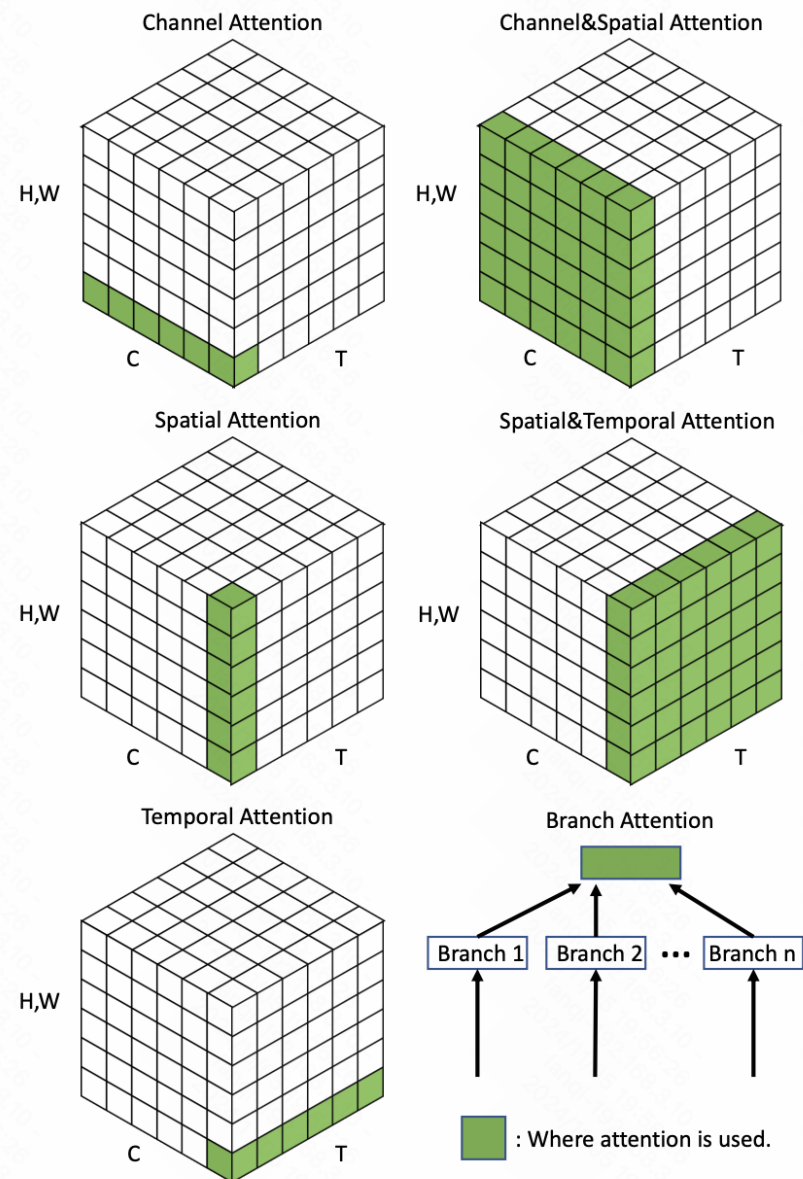
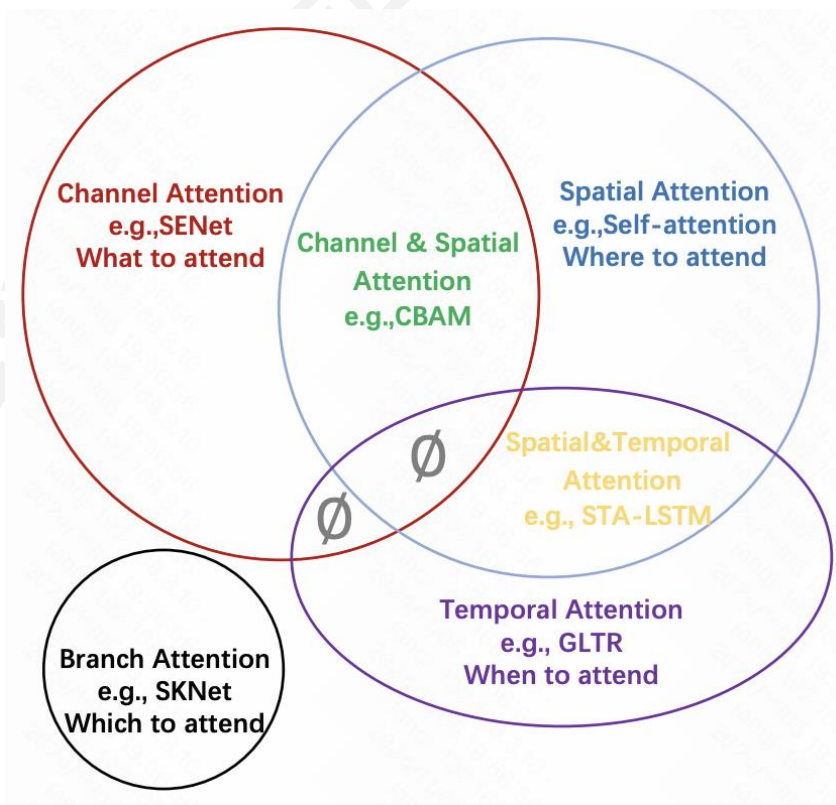
$$\begin{aligned}\alpha_n &= p(z = n|X, q) \\ &= \text{softmax}(s(\mathbf{x}_n, q)) \\ &= \frac{\exp(s(\mathbf{x}_n, q))}{\sum_{j=1}^N \exp(s(\mathbf{x}_j, q))}\end{aligned}$$

■ 2. 根据 α 计算输入信息的加权平均

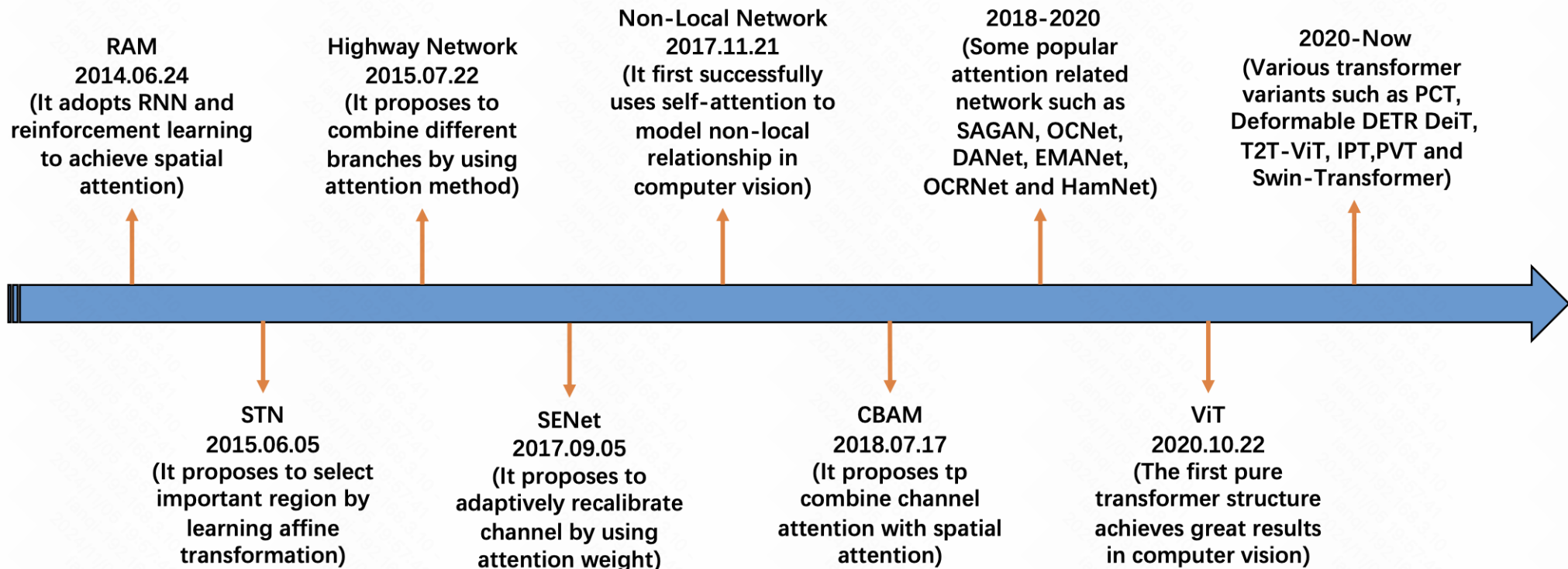
$$\begin{aligned}\text{att}(X, q) &= \sum_{n=1}^N \alpha_n \mathbf{x}_n, \\ &= \mathbb{E}_{z \sim p(z|X, q)}[\mathbf{x}_z]\end{aligned}$$



■ 注意力类型



■ 注意力发展历史

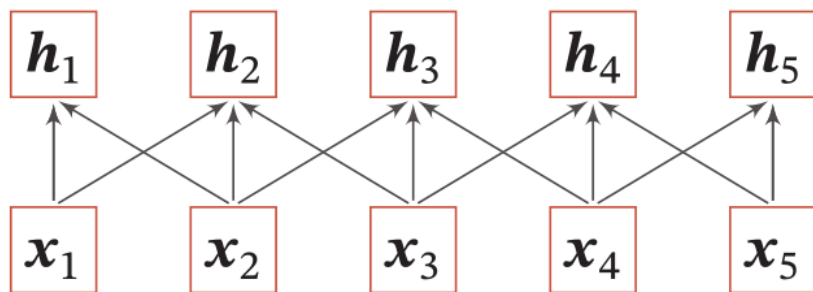


■ 注意力类型文献总结

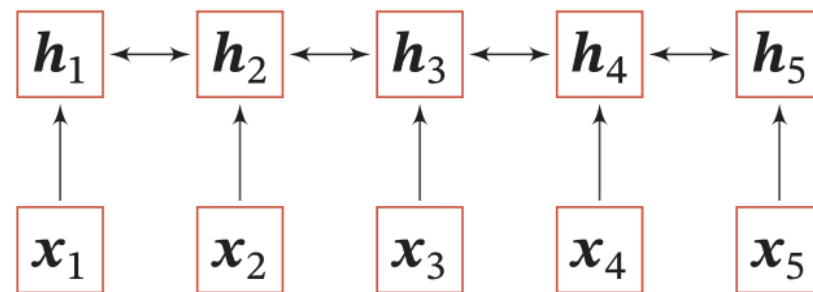
Brief summary of attention categories and key related works.

Attention category	Description	Related work
Channel attention	Generate attention mask across the channel domain and use it to select important channels.	[5], [37], [53], [54], [55], [56], [57], [58] [25], [59], [60]
Spatial attention	Generate attention mask across spatial domains and use it to select important spatial regions (e.g. [15], [61]) or predict the most relevant spatial position directly (e.g. [7], [31]).	[8], [9], [15], [21], [31], [32], [34], [35], [22], [26], [62], [63], [64], [65], [66], [67], [41], [68], [69], [70], [71], [72], [73], [74], [8], [34], [42], [43], [75], [76], [77], [78], [27], [44], [45], [46], [79], [80], [81], [82], [61], [83], [84], [85], [86], [87], [88], [89], [47], [90], [91], [92], [93], [94], [95], [96], [97], [98], [99], [100], [101], [102], [103], [104], [20], [105], [106], [107], [108], [109]
Temporal attention	Generate attention mask in time and use it to select key frames.	[110], [111], [112]
Branch attention	Generate attention mask across the different branches and use it to select important branches.	[113], [114], [115], [116]
Channel & spatial attention	Predict channel and spatial attention masks separately (e.g. [6], [117]) or generate a joint 3-D channel, height, width attention mask directly (e.g. [118], [119]) and use it to select important features.	[6], [50], [117], [119], [120], [121], [122], [10], [101], [118], [123], [124], [125], [126], [13], [14], [127], [128], [129]
Spatial & temporal attention	Compute temporal and spatial attention masks separately (e.g. [16], [130]), or produce a joint spatiotemporal attention mask (e.g. [131]), to focus on informative regions.	[130], [132], [133], [134], [135], [136], [137], [138], [139]

- 处理变长的向量序列
- CNN 和 RNN 只建模了输入信息的局部依赖关系

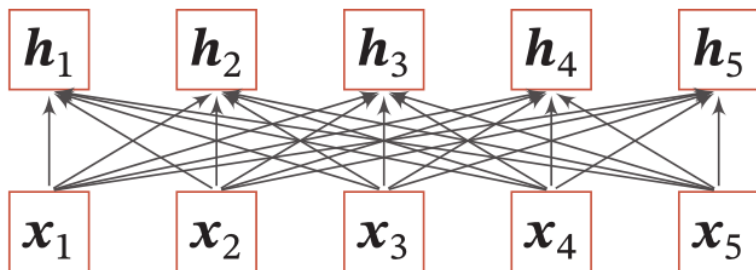


(a) 卷积网络

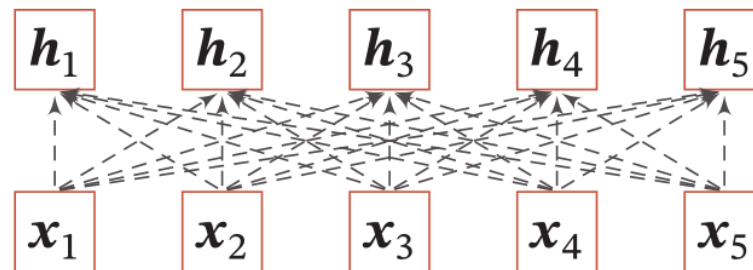


(b) 双向循环网络

- 处理变长的向量序列
- 全连接层？ 可以处理全局信息， 但无法处理变长问题
- 自注意力模型： 连接权重 α_{ij} 由注意力机制动态生成

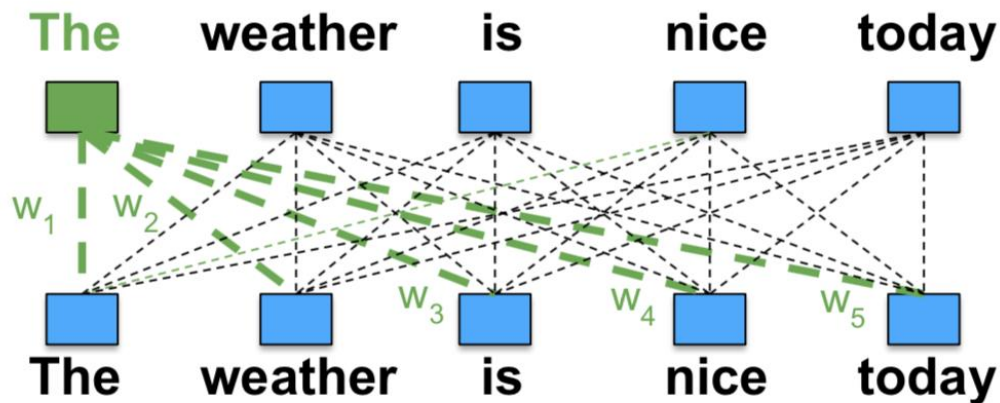


(a) 全连接模型



(b) 自注意力模型

■ 自注意力示例



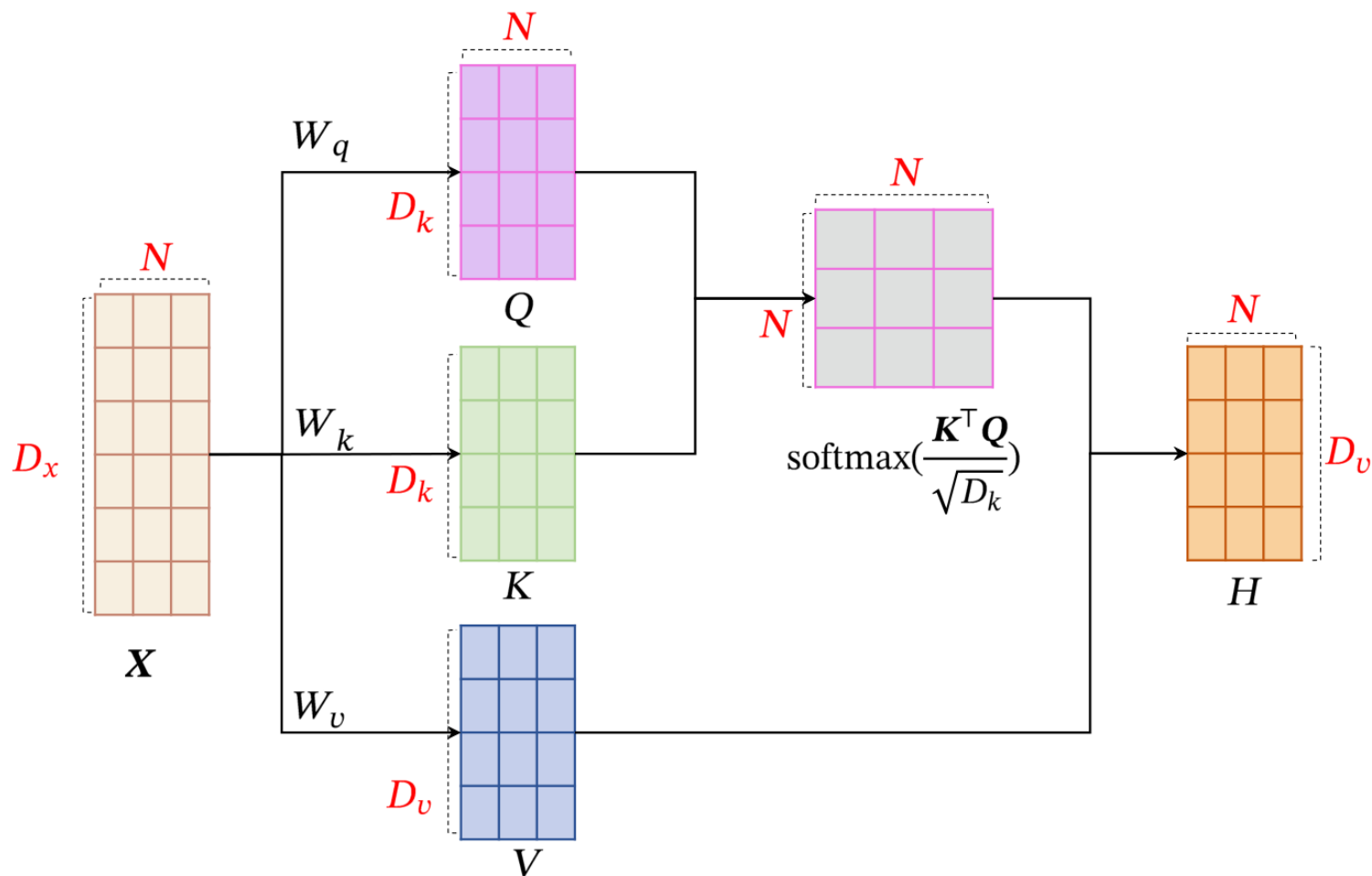
$$w_1, w_2, w_3, w_4, w_5 = \text{softmax} \left(\begin{bmatrix} 0.6 & 0.2 & 0.8 \end{bmatrix} \times \begin{bmatrix} 0.6 & 0.2 & 0.9 & 0.4 & 0.4 \\ 0.2 & 0.3 & 0.1 & 0.1 & 0.1 \\ 0.8 & 0.1 & 0.8 & 0.4 & 0.6 \end{bmatrix} \right)$$

The The weather is nice today

$$\begin{bmatrix} 1.8 \\ 2.3 \\ 0.4 \end{bmatrix} = w_1 \times \begin{bmatrix} 0.6 \\ 0.2 \\ 0.8 \end{bmatrix} + w_2 \times \begin{bmatrix} 0.2 \\ 0.3 \\ 0.1 \end{bmatrix} + w_3 \times \begin{bmatrix} 0.9 \\ 0.1 \\ 0.8 \end{bmatrix} + w_4 \times \begin{bmatrix} 0.4 \\ 0.1 \\ 0.4 \end{bmatrix} + w_5 \times \begin{bmatrix} 0.4 \\ 0.1 \\ 0.6 \end{bmatrix}$$

The weather is nice today

■ 自注意力的 Q-K-V



$$Q = W_q X \in \mathbb{R}^{D_k \times N},$$

$$K = W_k X \in \mathbb{R}^{D_k \times N},$$

$$V = W_v X \in \mathbb{R}^{D_v \times N},$$

$$h_n = \text{att}((K, V), q_n)$$

$$H = V \text{softmax}(\frac{K^T Q}{\sqrt{D_k}}),$$

■ 数据库：Q-K-V

```
{ "order_100": { "items": "a1", "delivery_date": "a2", ... },  
  "order_101": { "items": "b1", "delivery_date": "b2", ... },  
  "order_102": { "items": "c1", "delivery_date": "c2", ... },  
  "order_103": { "items": "d1", "delivery_date": "d2", ... },  
  "order_104": { "items": "e1", "delivery_date": "e2", ... },  
  "order_105": { "items": "f1", "delivery_date": "f2", ... },  
  "order_106": { "items": "g1", "delivery_date": "g2", ... },  
  "order_107": { "items": "h1", "delivery_date": "h2", ... },  
  "order_108": { "items": "i1", "delivery_date": "i2", ... },  
  "order_109": { "items": "j1", "delivery_date": "j2", ... },  
  "order_110": { "items": "k1", "delivery_date": "k2", ... }
```

■ 数据库：Query

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

```
{ "order_100": { "items": "a1", "delivery_date": "a2", ... },  
  { "order_101": { "items": "b1", "delivery_date": "b2", ... },  
  { "order_102": { "items": "c1", "delivery_date": "c2", ... },  
  { "order_103": { "items": "d1", "delivery_date": "d2", ... },  
  { "order_104": { "items": "e1", "delivery_date": "e2", ... },  
  { "order_105": { "items": "f1", "delivery_date": "f2", ... },  
  { "order_106": { "items": "g1", "delivery_date": "g2", ... },  
  { "order_107": { "items": "h1", "delivery_date": "h2", ... },  
  { "order_108": { "items": "i1", "delivery_date": "i2", ... },  
  { "order_109": { "items": "j1", "delivery_date": "j2", ... },  
  { "order_110": { "items": "k1", "delivery_date": "k2", ... }
```

■ 数据库: Key

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

```
{ "order_100": { "items": "a1", "delivery_date": "a2", ... },  
  { "order_101": { "items": "b1", "delivery_date": "b2", ... },  
  { "order_102": { "items": "c1", "delivery_date": "c2", ... },  
  { "order_103": { "items": "d1", "delivery_date": "d2", ... },  
  { "order_104": { "items": "e1", "delivery_date": "e2", ... },  
  { "order_105": { "items": "f1", "delivery_date": "f2", ... },  
  { "order_106": { "items": "g1", "delivery_date": "g2", ... },  
  { "order_107": { "items": "h1", "delivery_date": "h2", ... },  
  { "order_108": { "items": "i1", "delivery_date": "i2", ... },  
  { "order_109": { "items": "j1", "delivery_date": "j2", ... },  
  { "order_110": { "items": "k1", "delivery_date": "k2", ... }
```

■ 数据库: Value

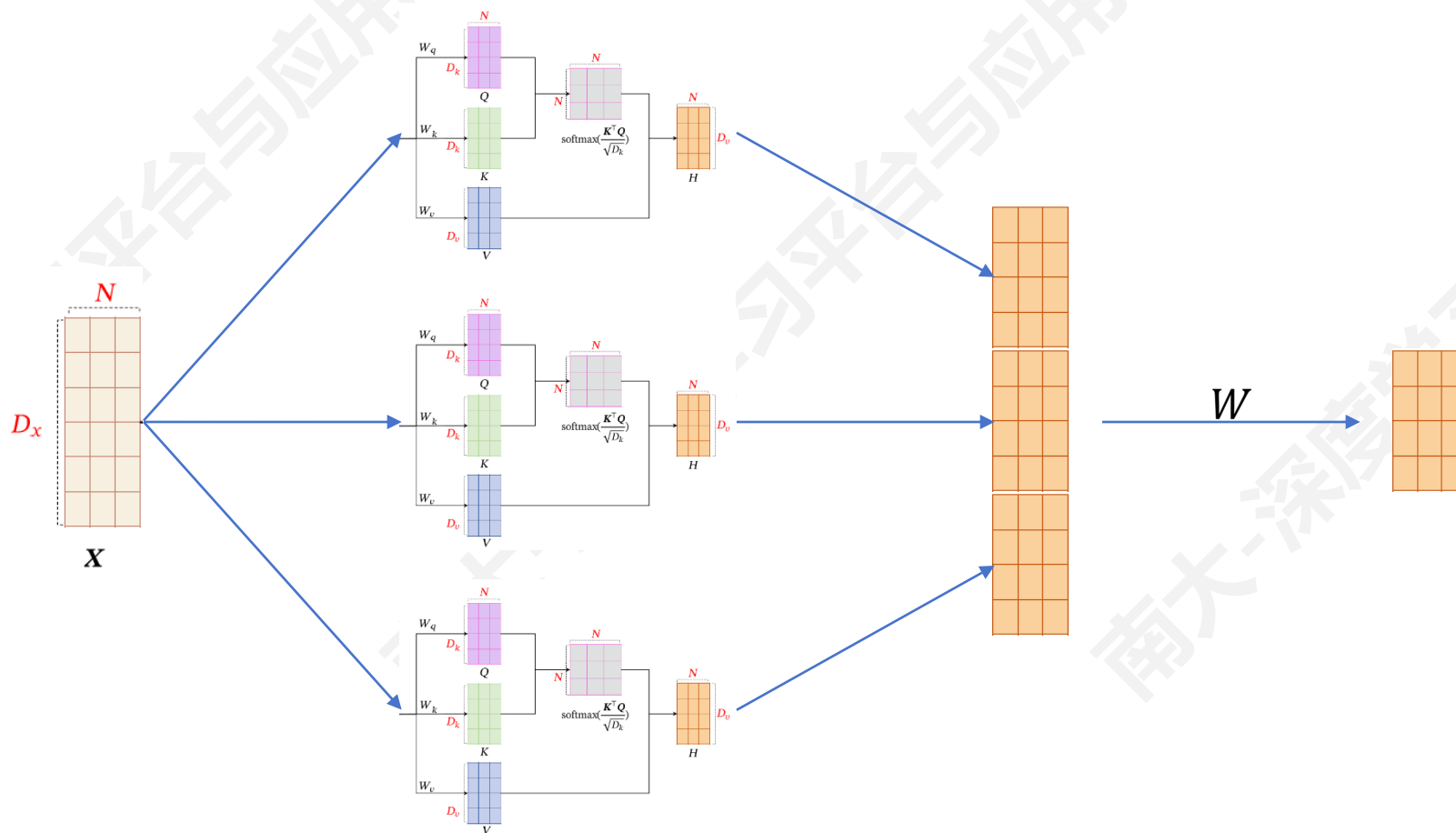
{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

```
{ "order_100": { "items": "a1", "delivery_date": "a2", ... },  
  { "order_101": { "items": "b1", "delivery_date": "b2", ... },  
  { "order_102": { "items": "c1", "delivery_date": "c2", ... },  
  { "order_103": { "items": "d1", "delivery_date": "d2", ... },  
  { "order_104": { "items": "e1", "delivery_date": "e2", ... },  
  { "order_105": { "items": "f1", "delivery_date": "f2", ... },  
  { "order_106": { "items": "g1", "delivery_date": "g2", ... },  
  { "order_107": { "items": "h1", "delivery_date": "h2", ... },  
  { "order_108": { "items": "i1", "delivery_date": "i2", ... },  
  { "order_109": { "items": "j1", "delivery_date": "j2", ... },  
  { "order_110": { "items": "k1", "delivery_date": "k2", ... }
```

■ 多头注意力 (Multi-head Attention)



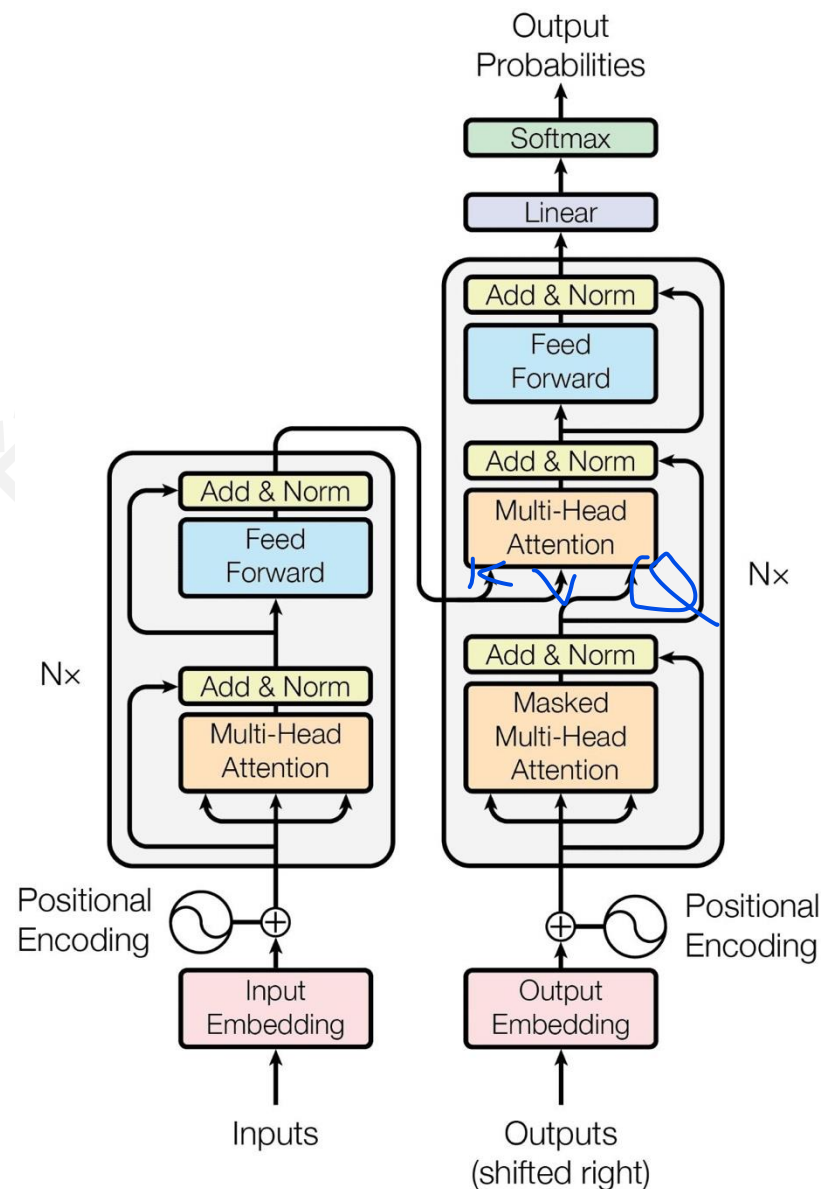
大 纲

- 注意力机制
- Transformer

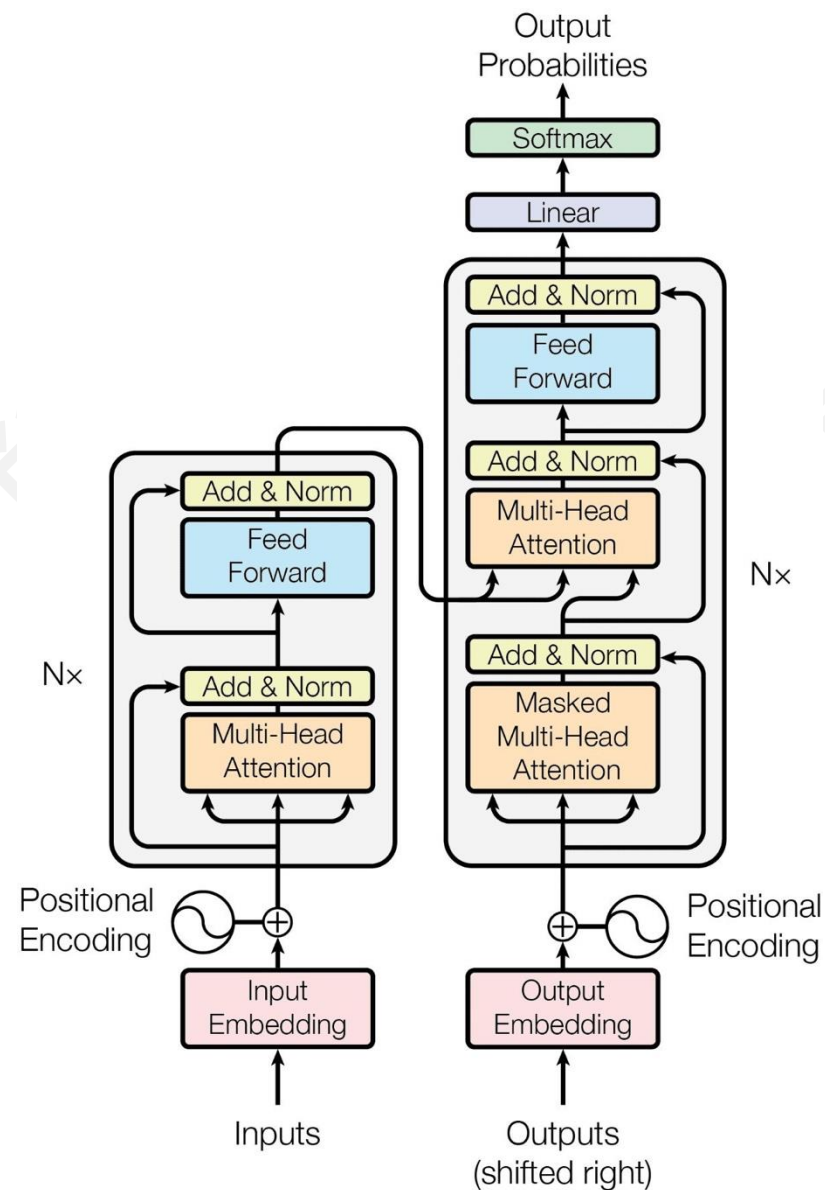
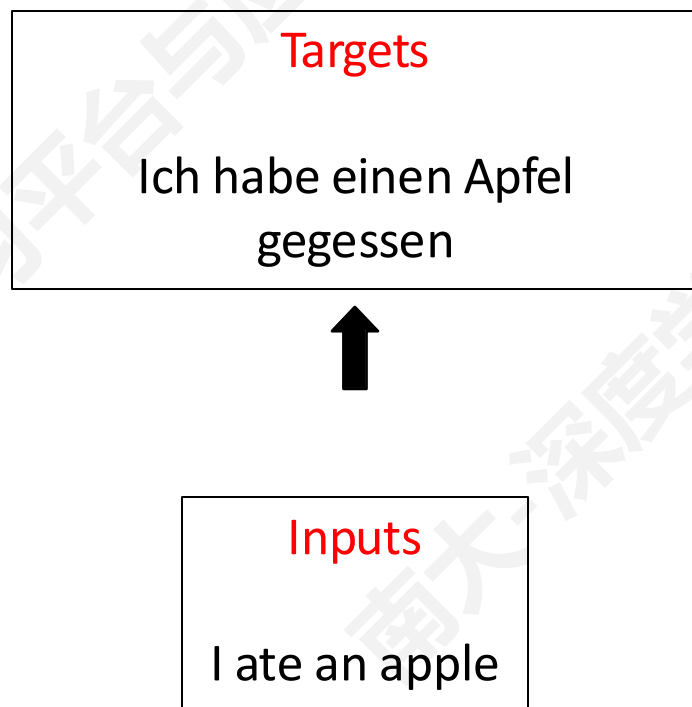
南大-深度学习平台与应用

Transformer 组成要素

- Tokenization
- Input Embeddings
- Position Encodings
- Query, Key, & Value
- Attention
- Self Attention
- Multi-Head Attention
- Feed Forward
- Add & Norm
- Encoders
- Masked Attention
- Encoder Decoder Attention
- Linear
- Softmax
- Decoders
- Encoder-Decoder Models



■ 例子：机器翻译

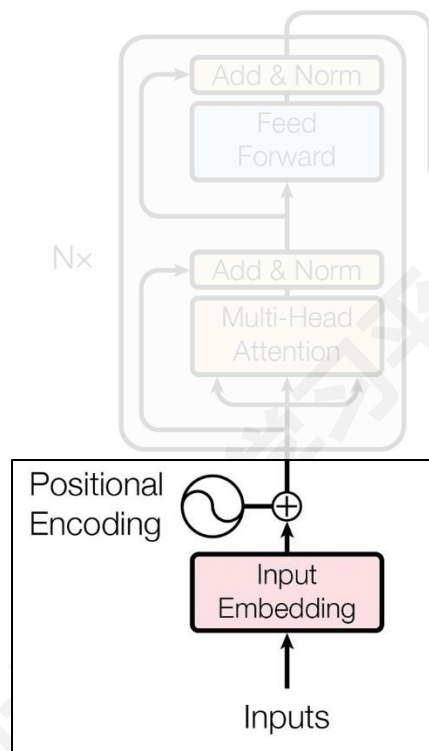


■ 处理输入：Tokenization

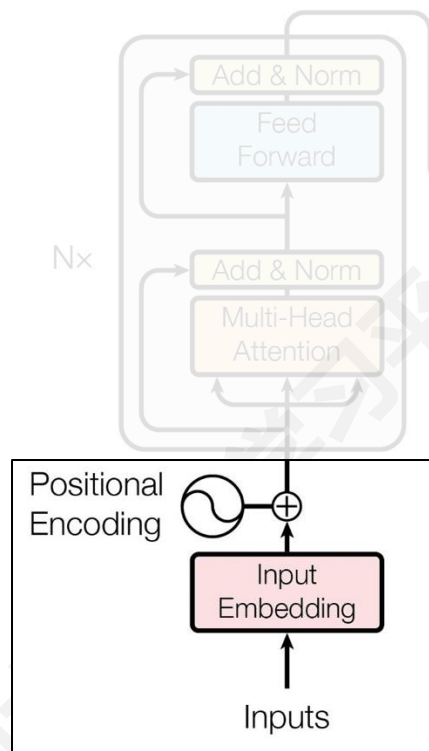
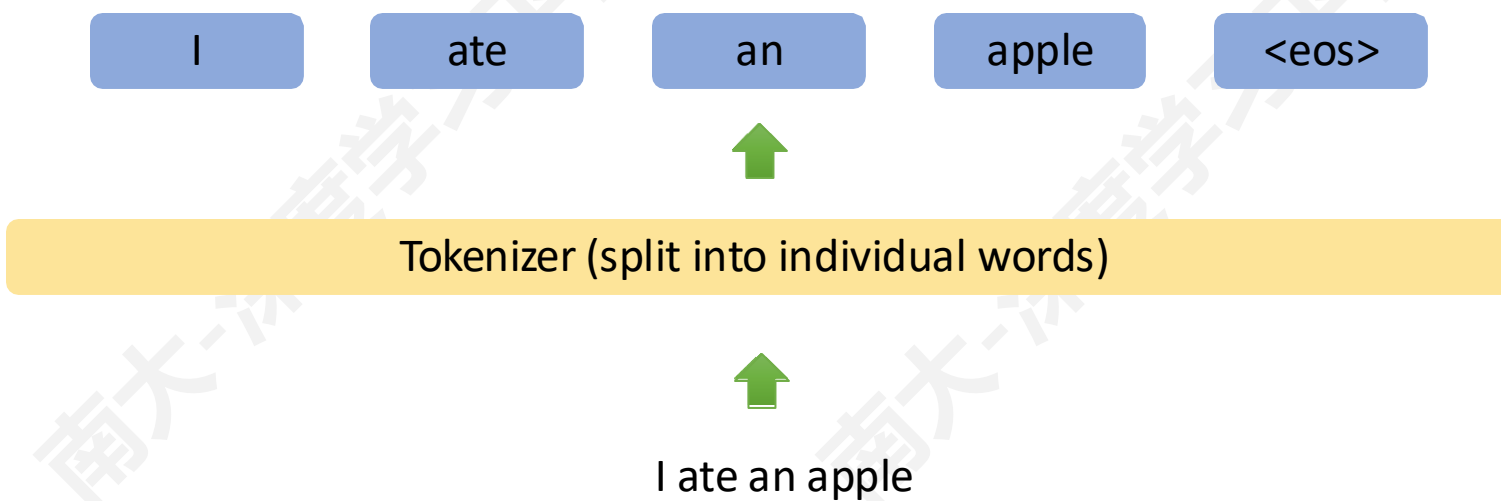
Tokenizer (split into individual words)



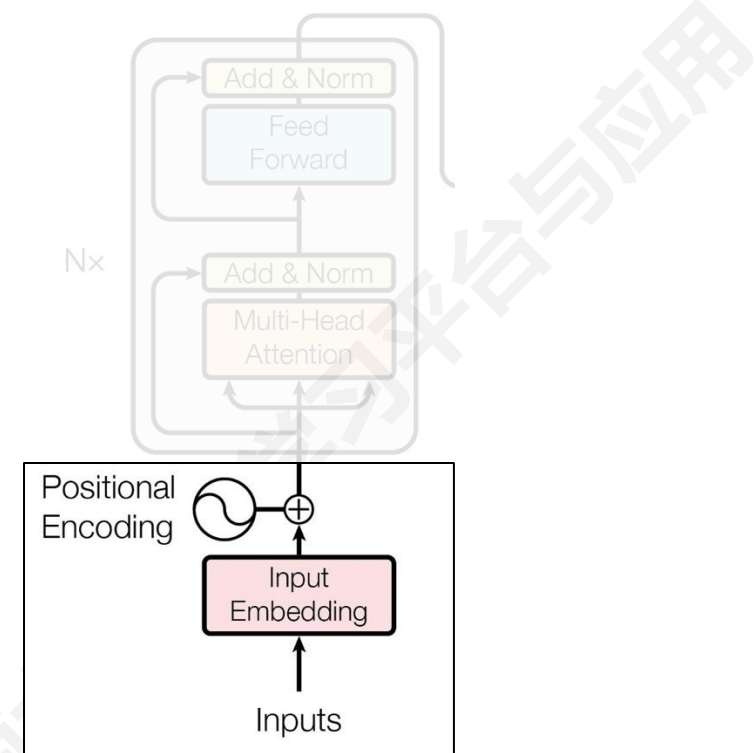
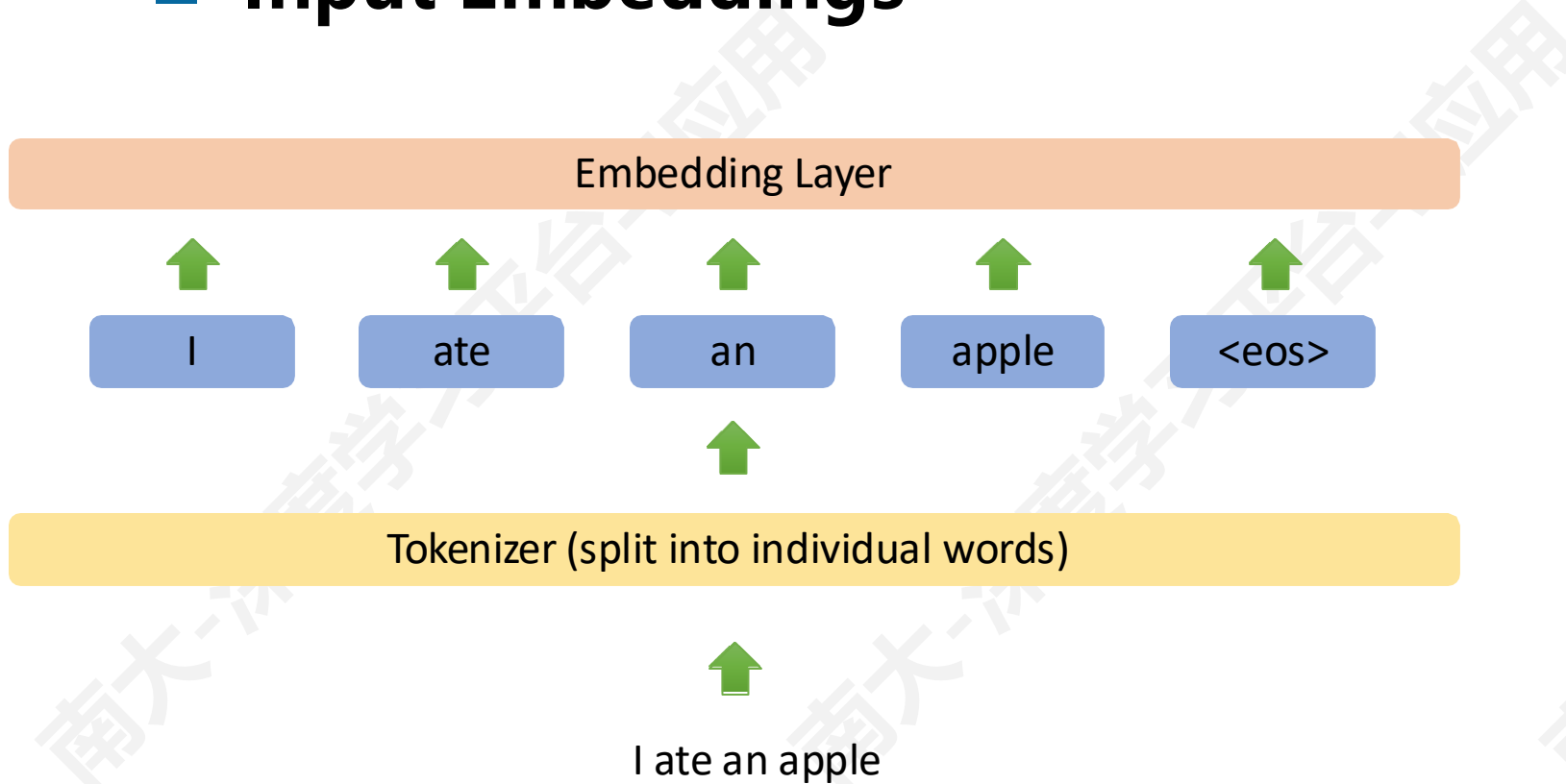
I ate an apple



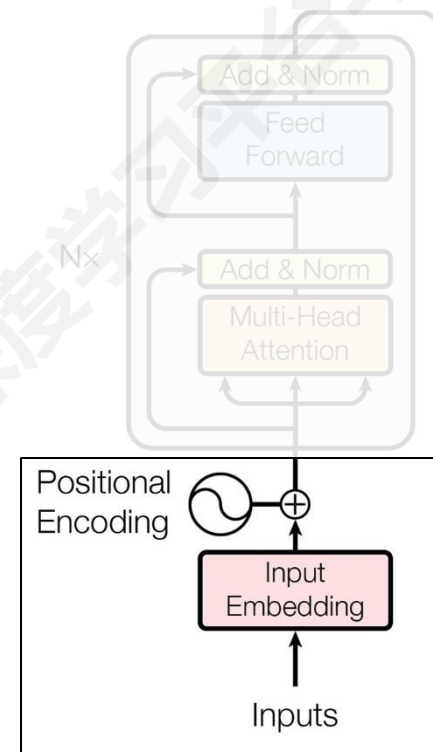
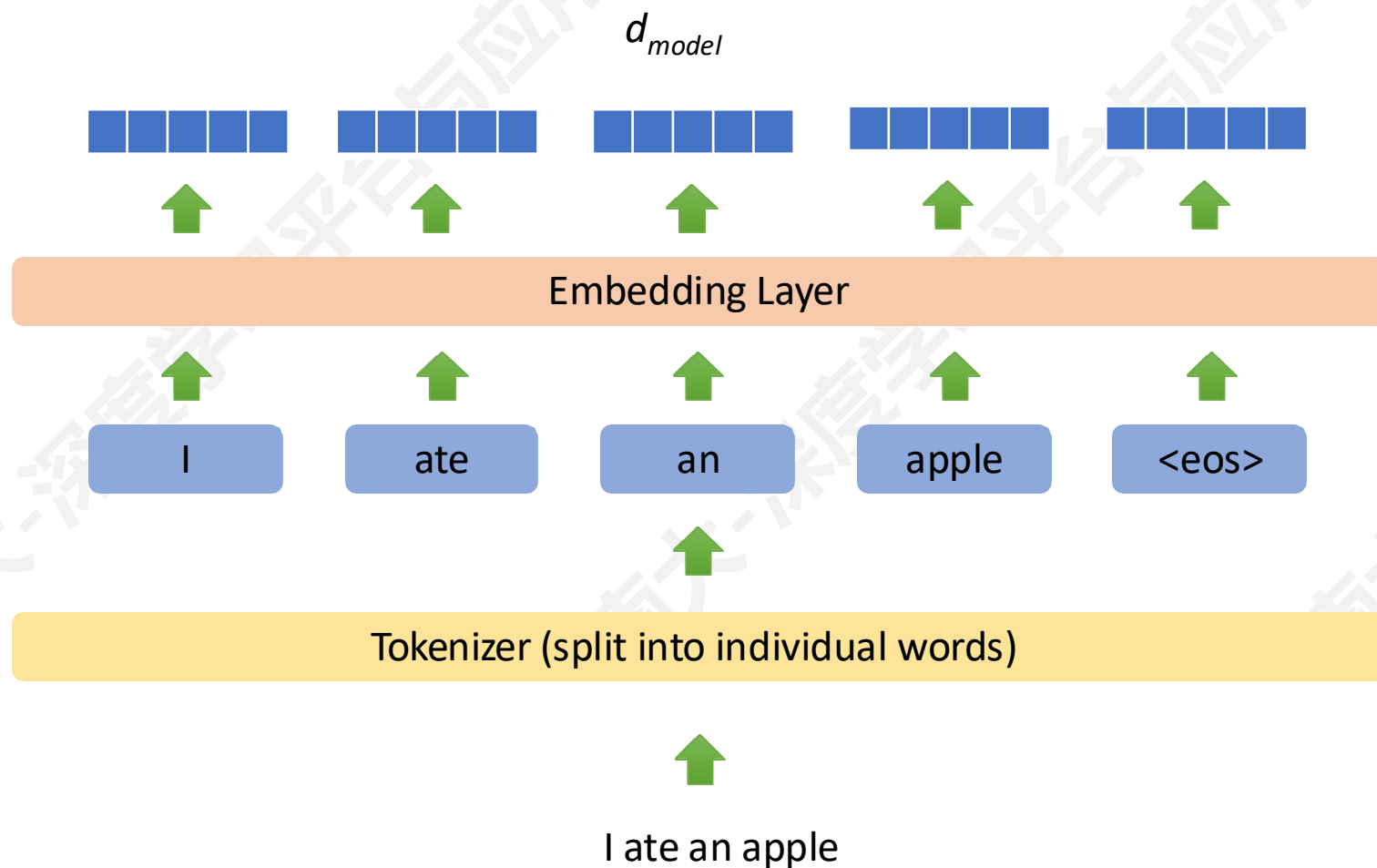
■ 处理输入：Tokenization



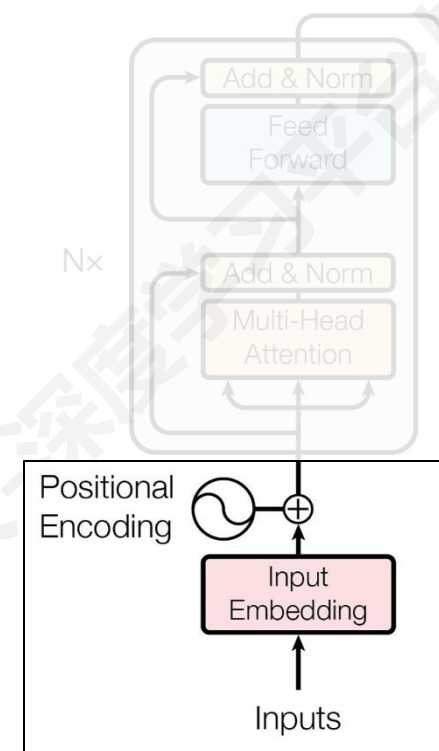
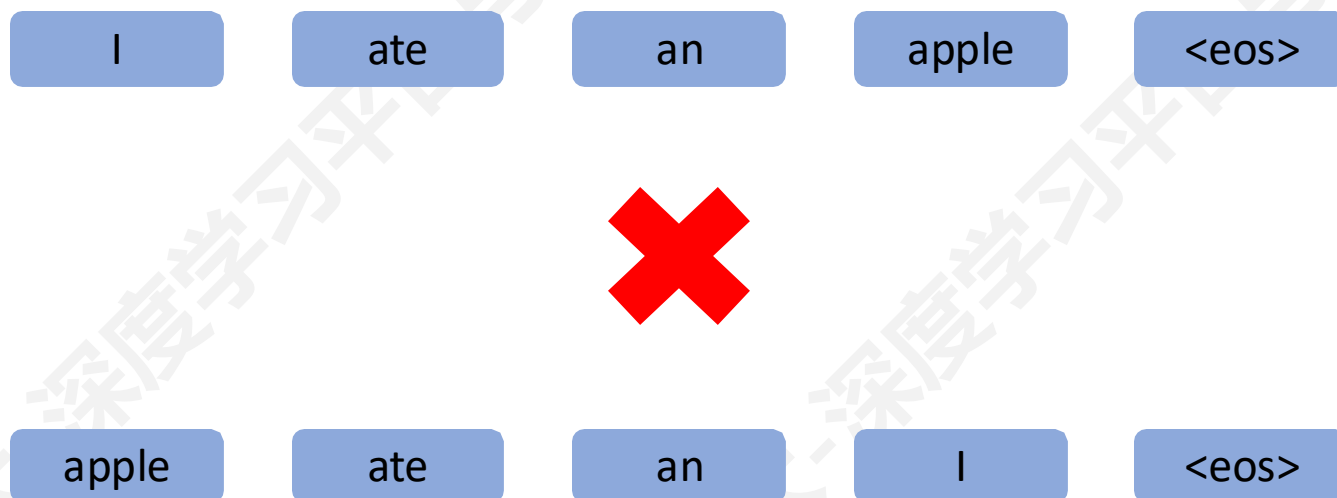
■ Input Embeddings



Input Embeddings



■ Position Encodings



■ Position Encodings

- 可以表示序列关系
- 每个位置都应该是独一无二的
- 值域有限

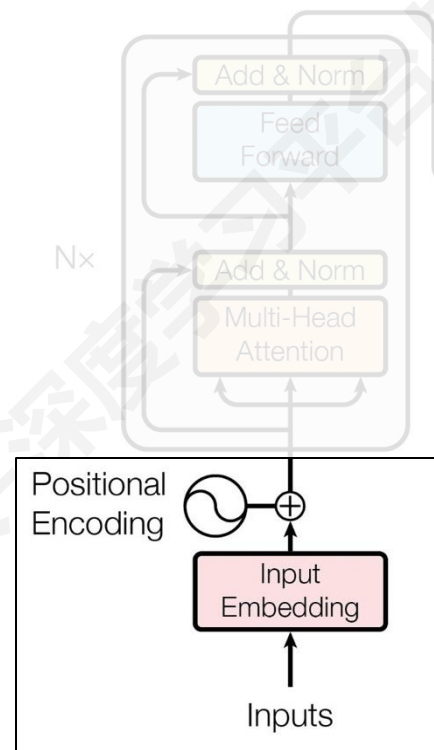
$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

pos -> idx of the token in input sentence

i -> i^{th} dimension out of d

d model -> embedding dimension of each token



Transformer

Position Encodings

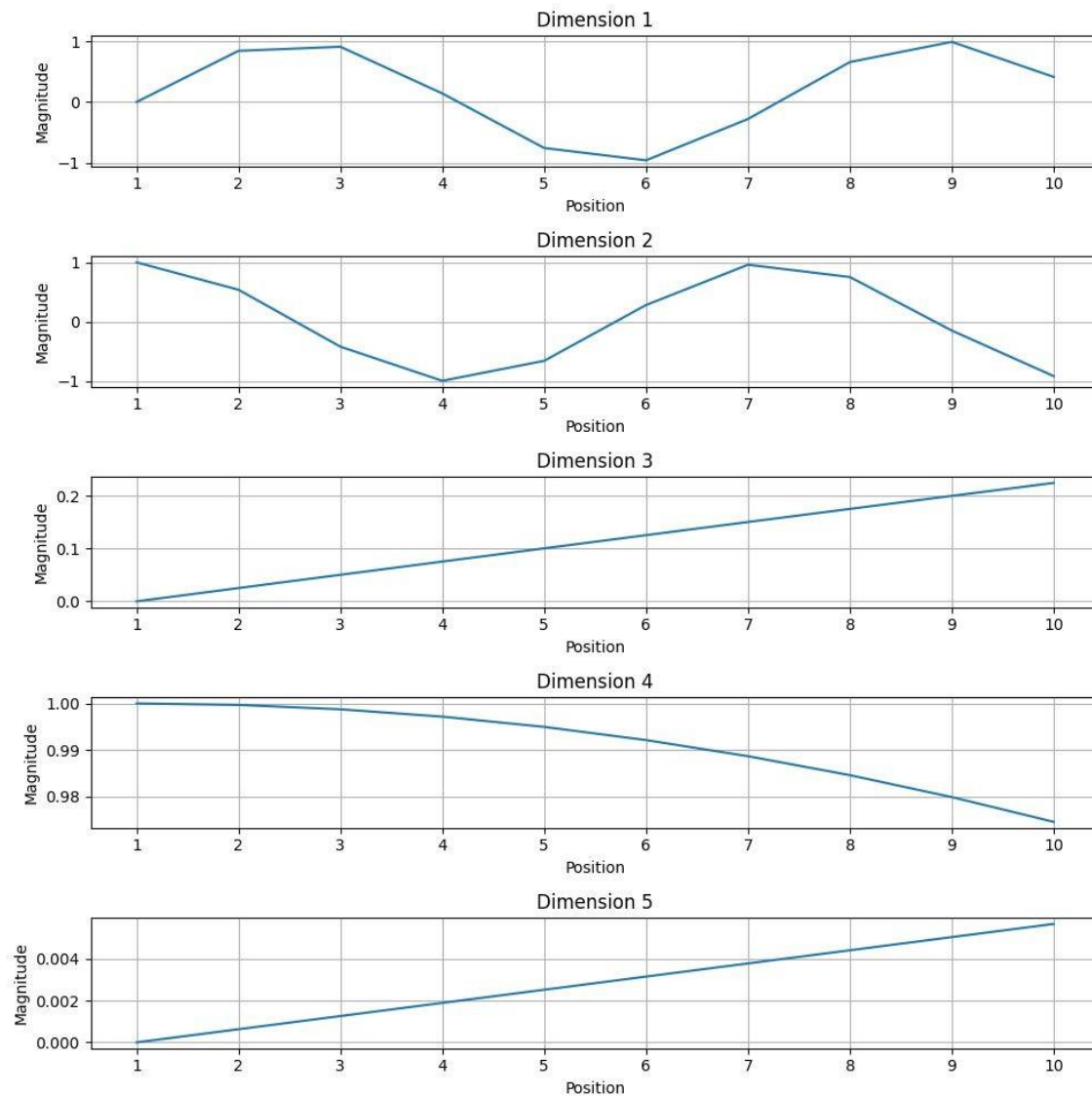
$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

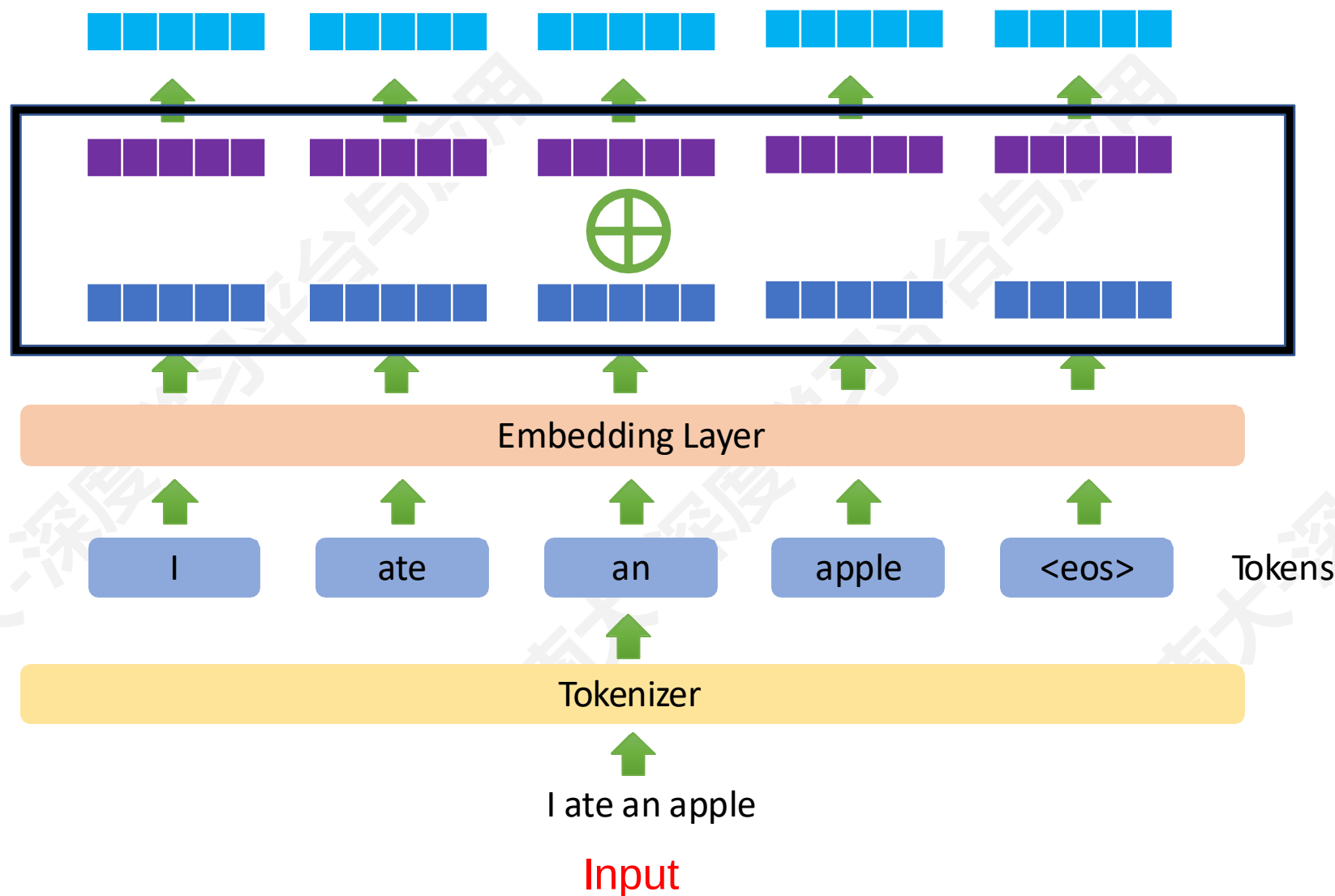


Positional Encoding:

	0	1	2	3	4
Dim 1	0.000	0.841	0.909	0.141	-0.757
Dim 2	1.000	0.540	-0.416	-0.990	-0.654
Dim 3	0.000	0.025	0.050	0.075	0.100
Dim 4	1.000	1.000	0.999	0.997	0.995
Dim 5	0.000	0.001	0.001	0.002	0.003



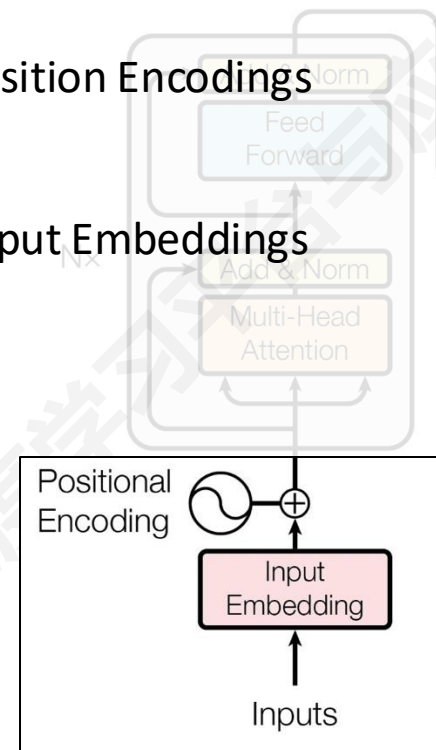
Transformer



Final Input Embeddings

Position Encodings

Input Embeddings



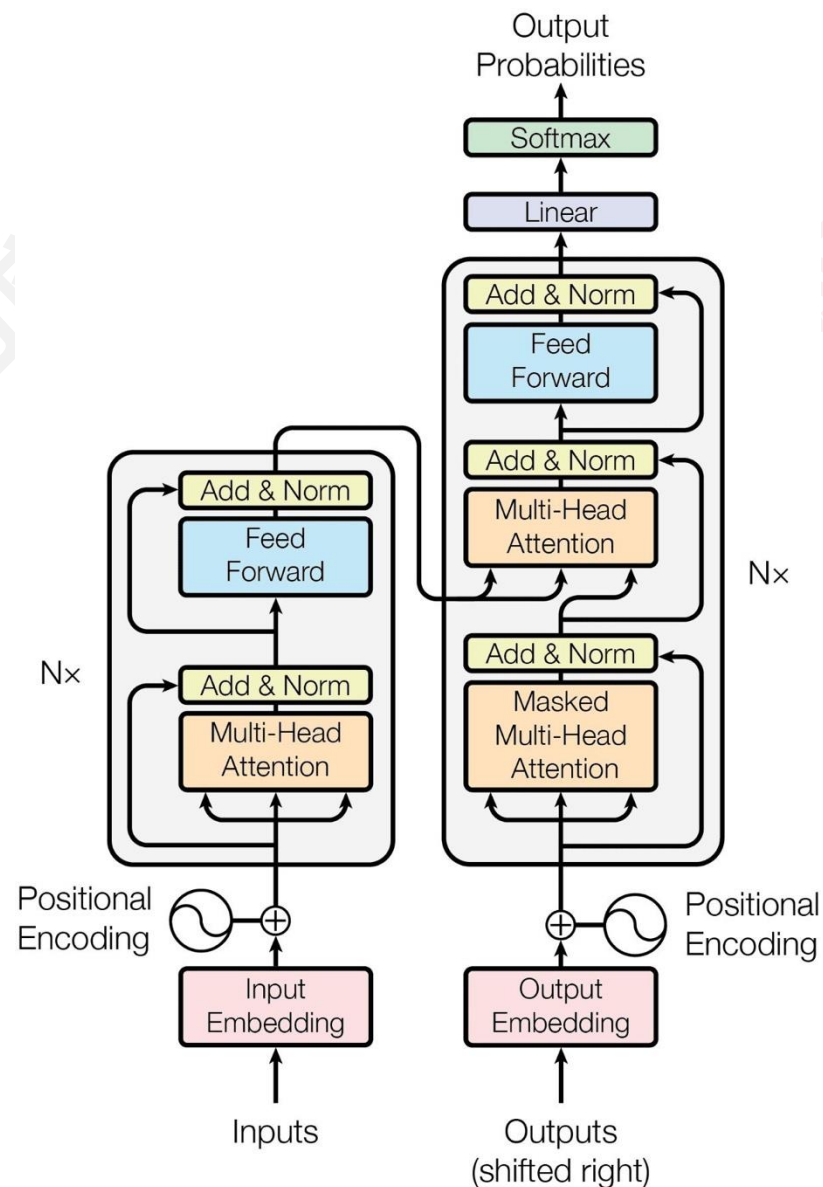
Transformer

✓ **Tokenization**

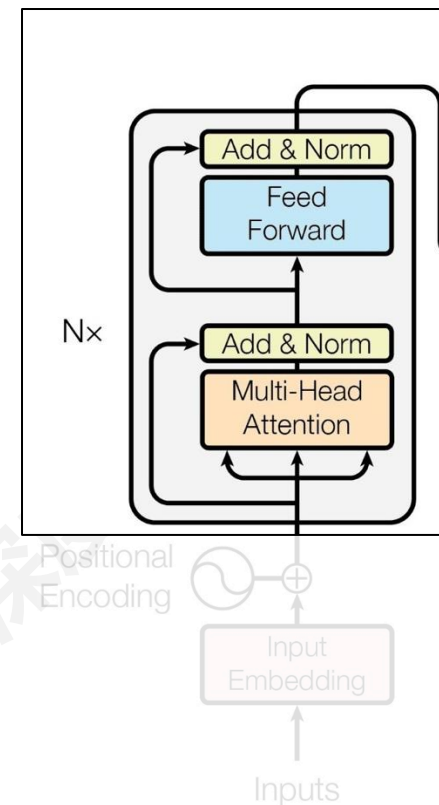
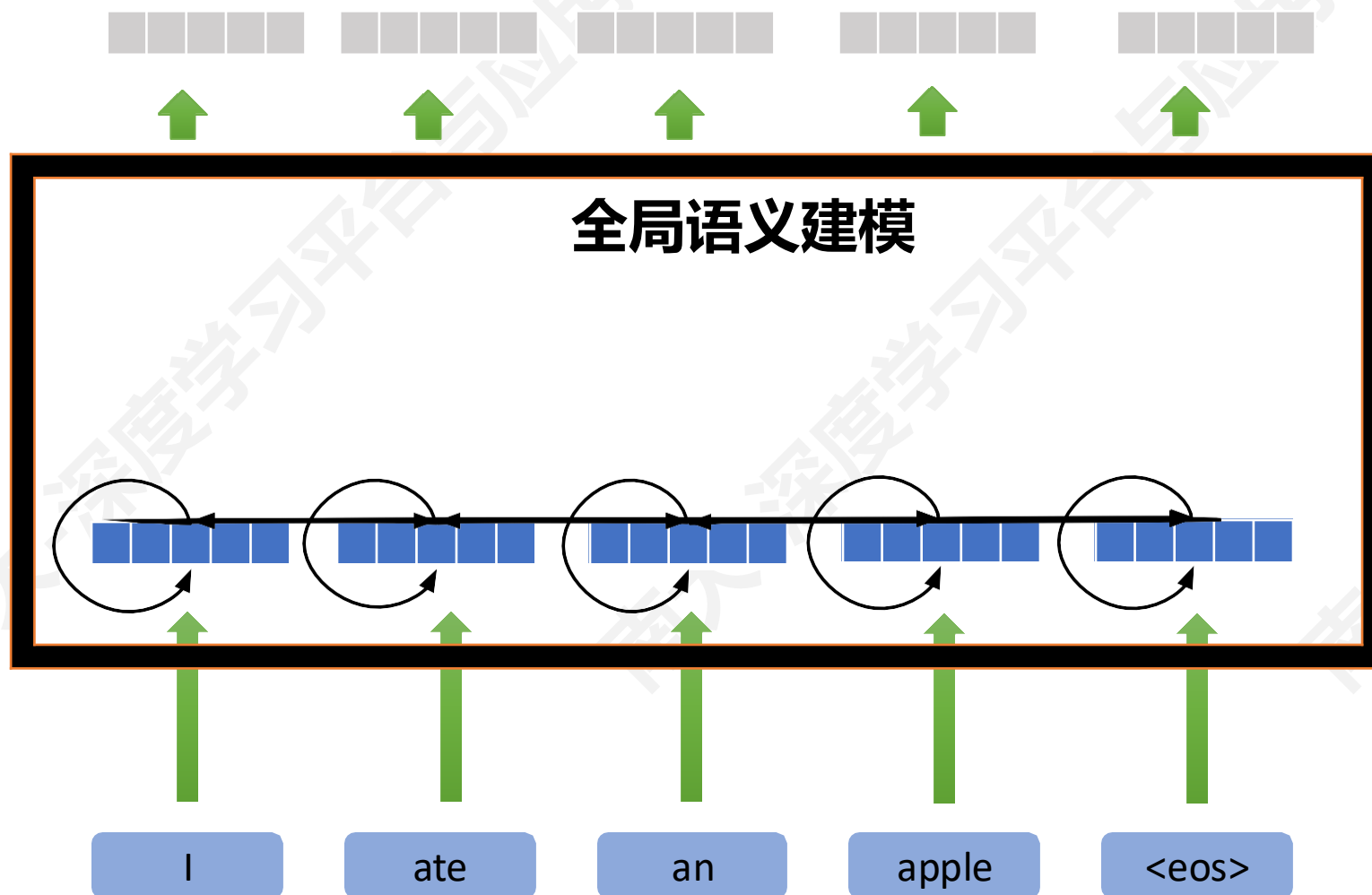
✓ **Input Embeddings**

✓ **Position Encodings**

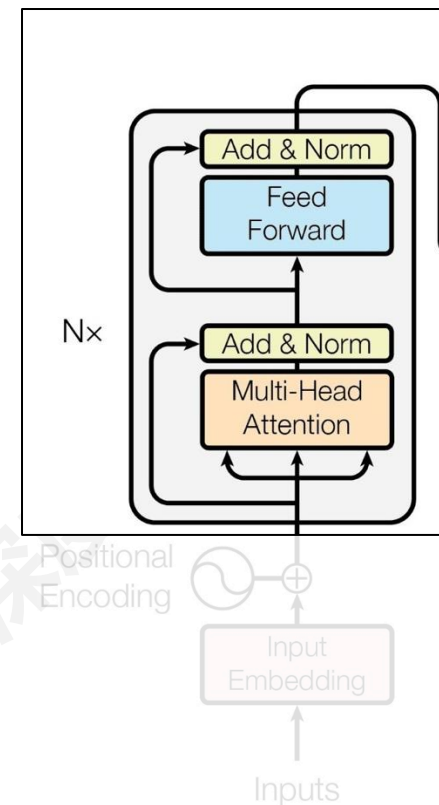
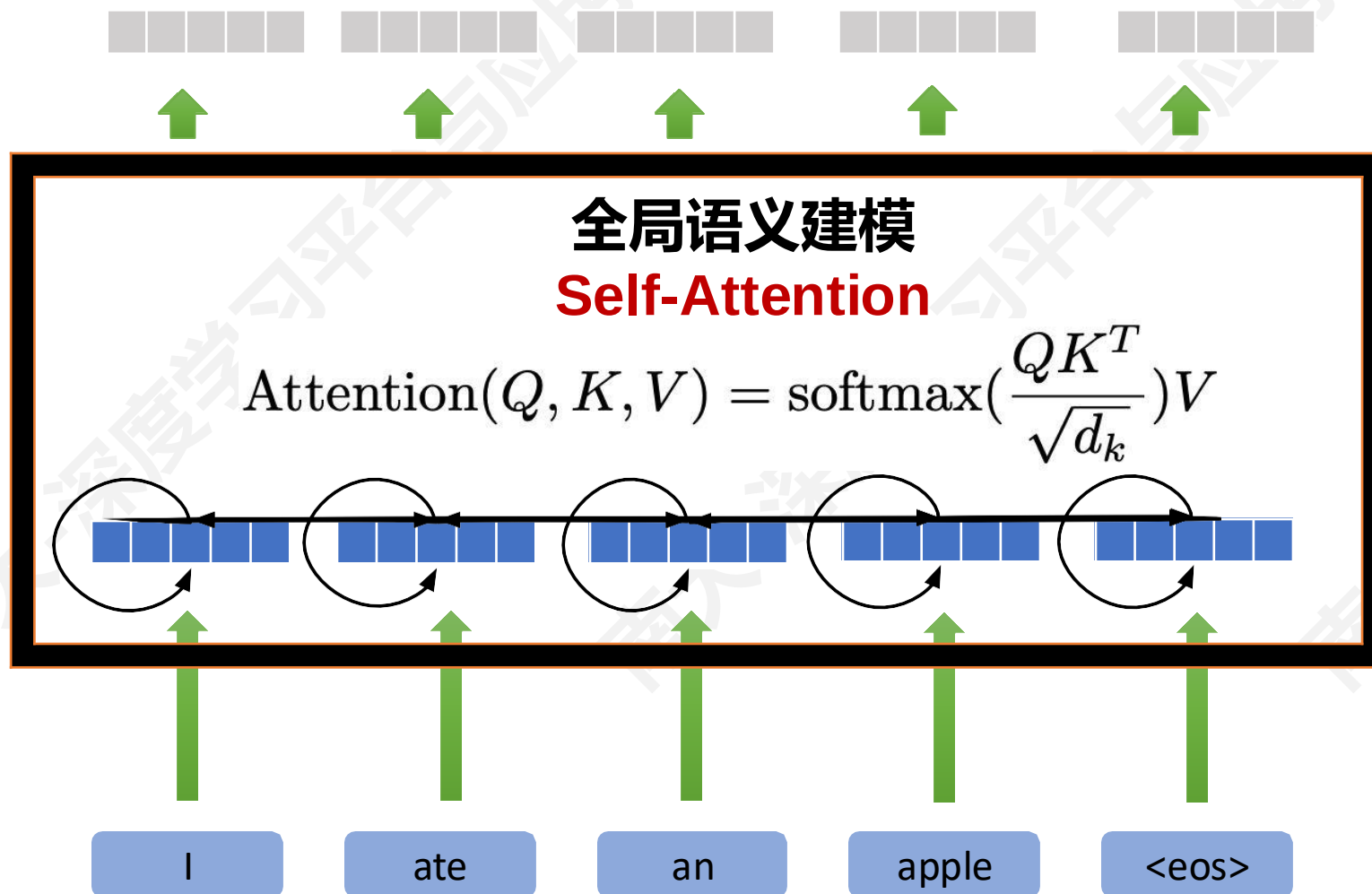
- Query, Key, & Value
- Attention
- Self Attention
- Multi-Head Attention
- Feed Forward
- Add & Norm
- Encoders
- Masked Attention
- Encoder Decoder Attention
- Linear
- Softmax
- Decoders
- Encoder-Decoder Models



■ 编码器 Encoder: 全局语义建模



■ 编码器 Encoder: 全局语义建模



■ Attention



I_1

I



I_2

ate



I_3

an



I_4

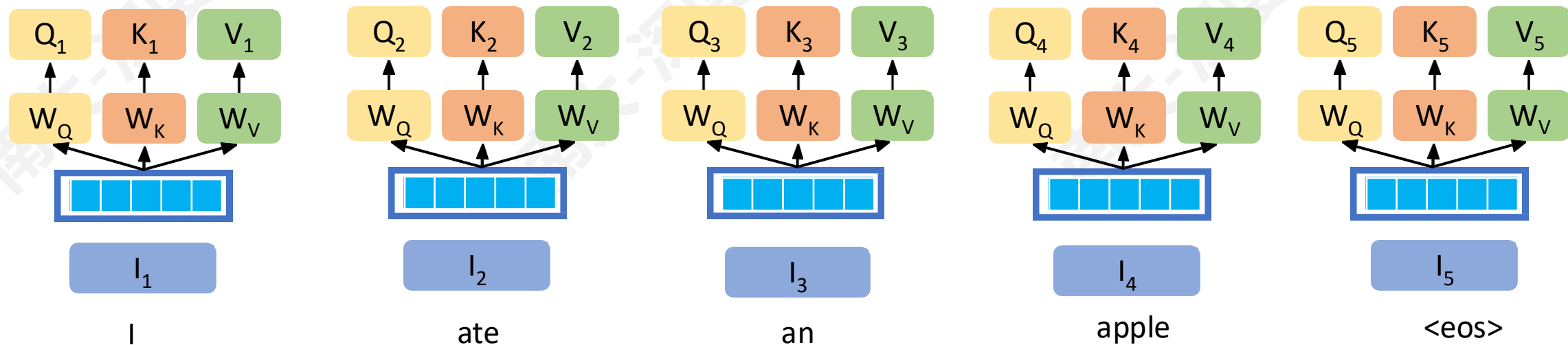
apple



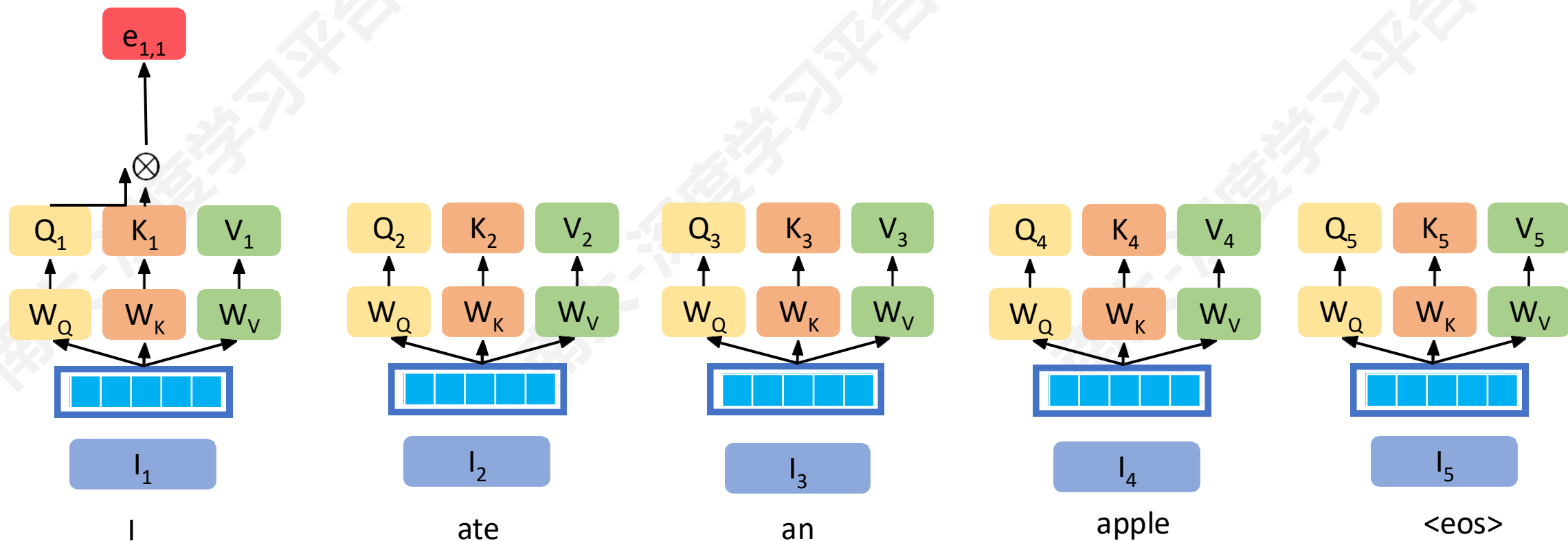
I_5

<eos>

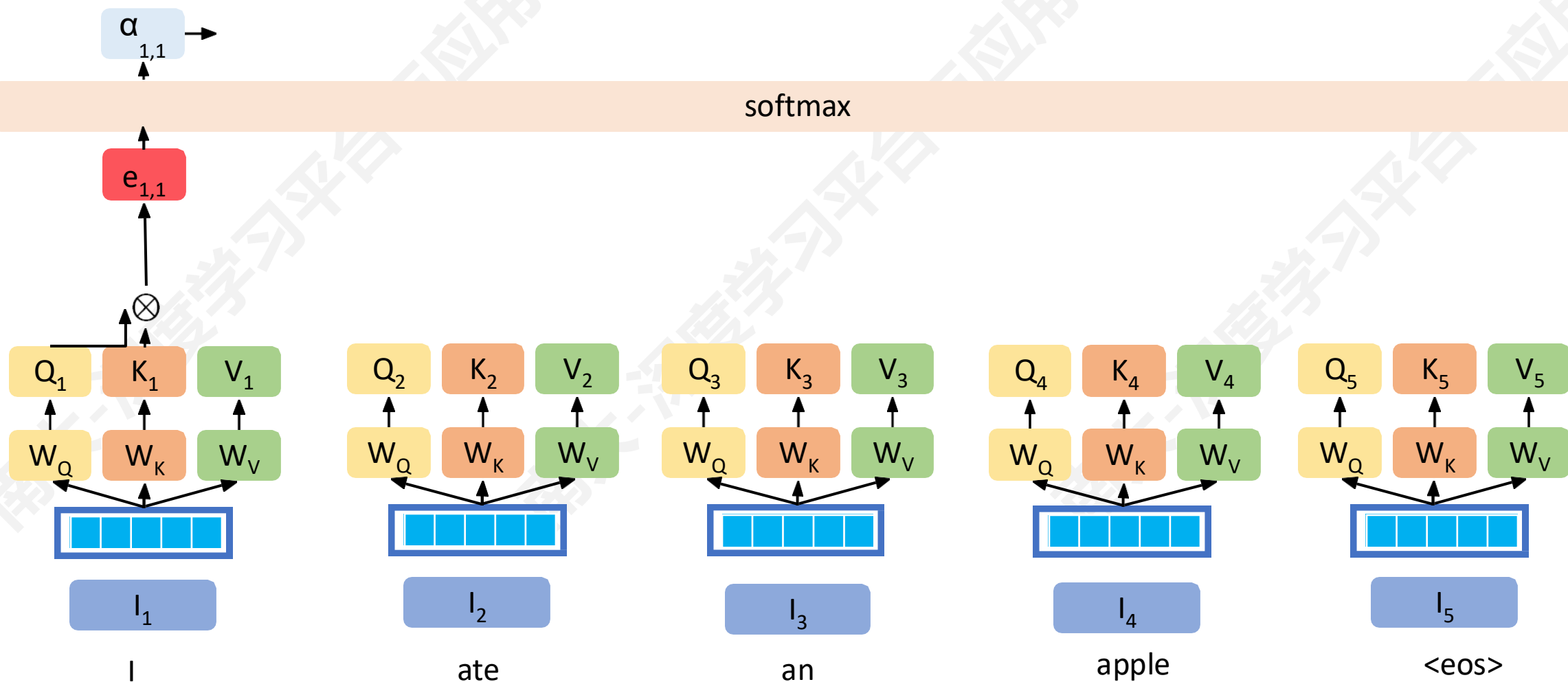
■ Attention



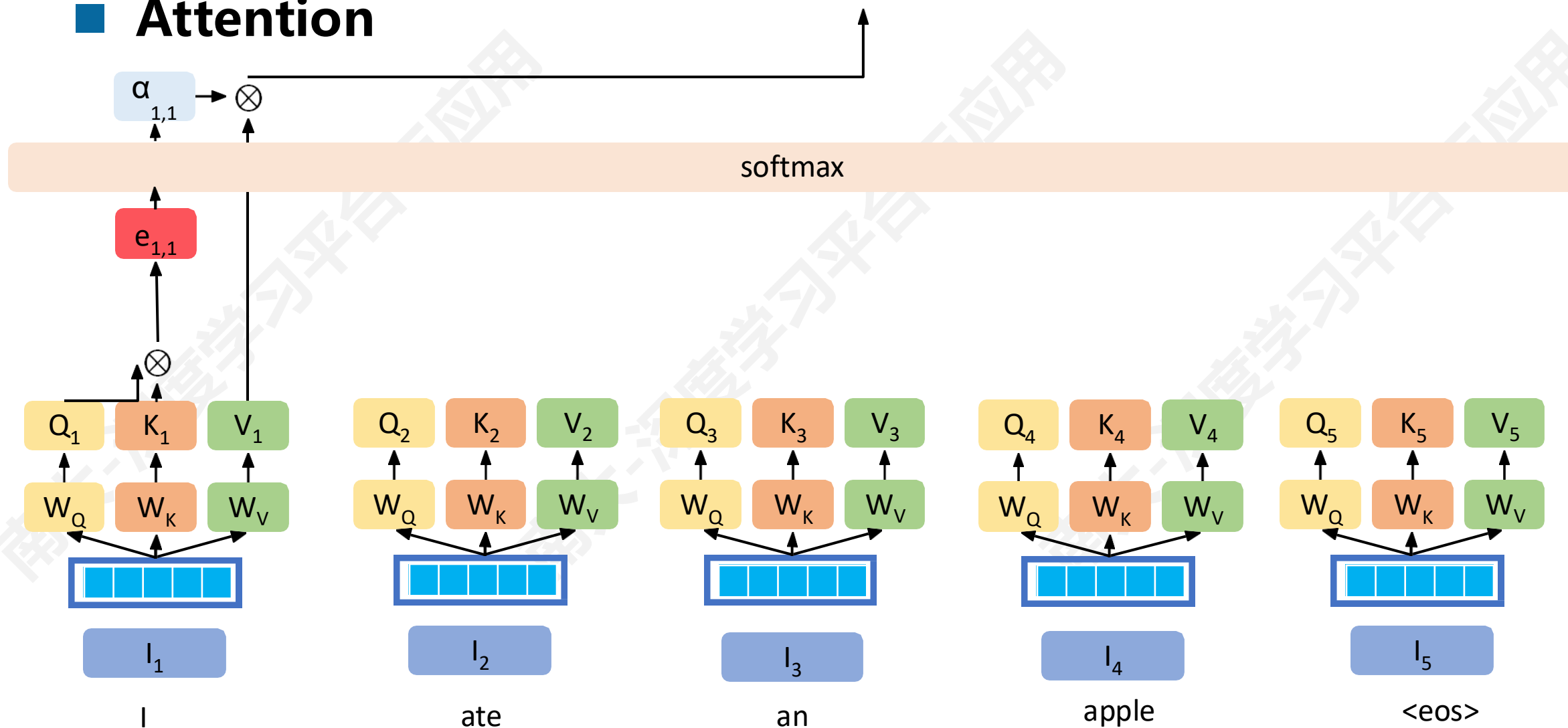
■ Attention



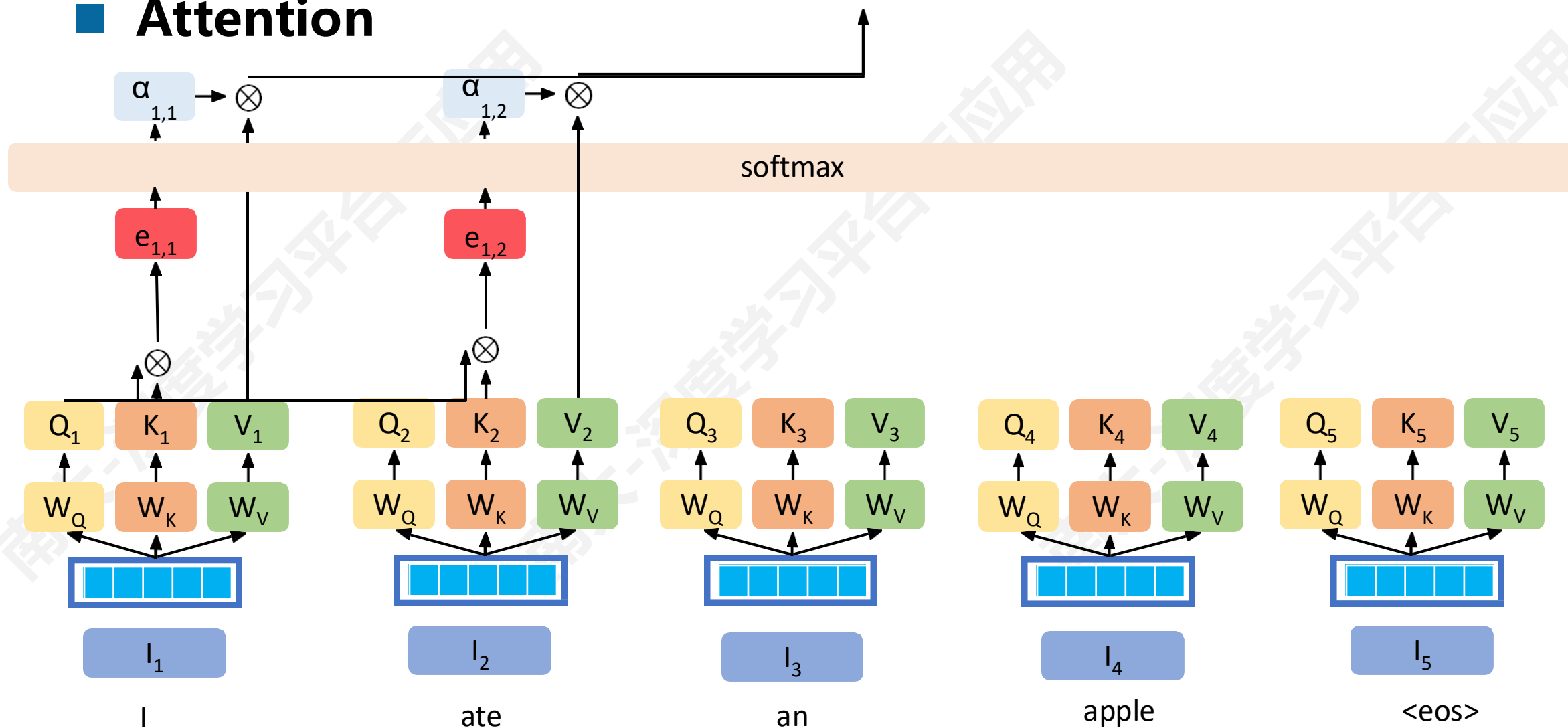
■ Attention



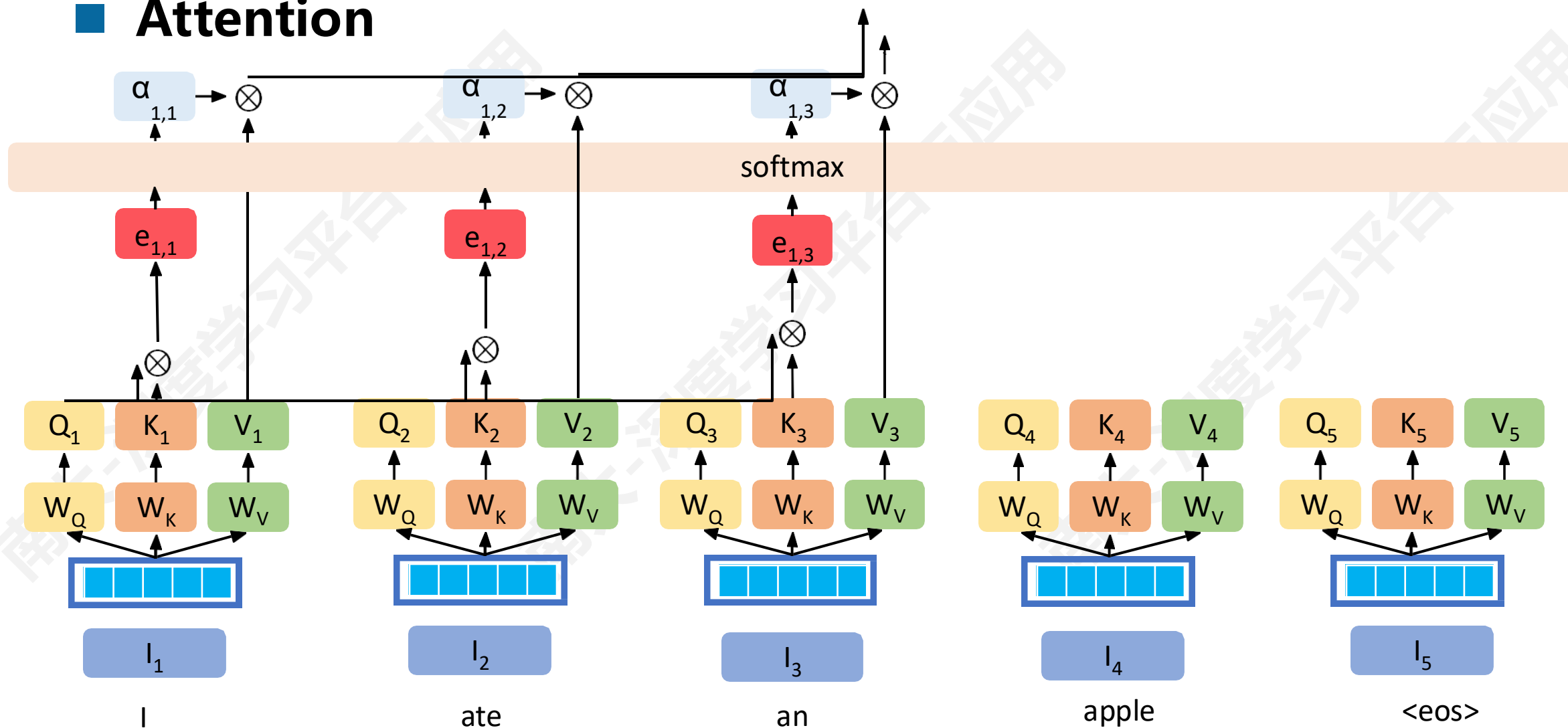
■ Attention



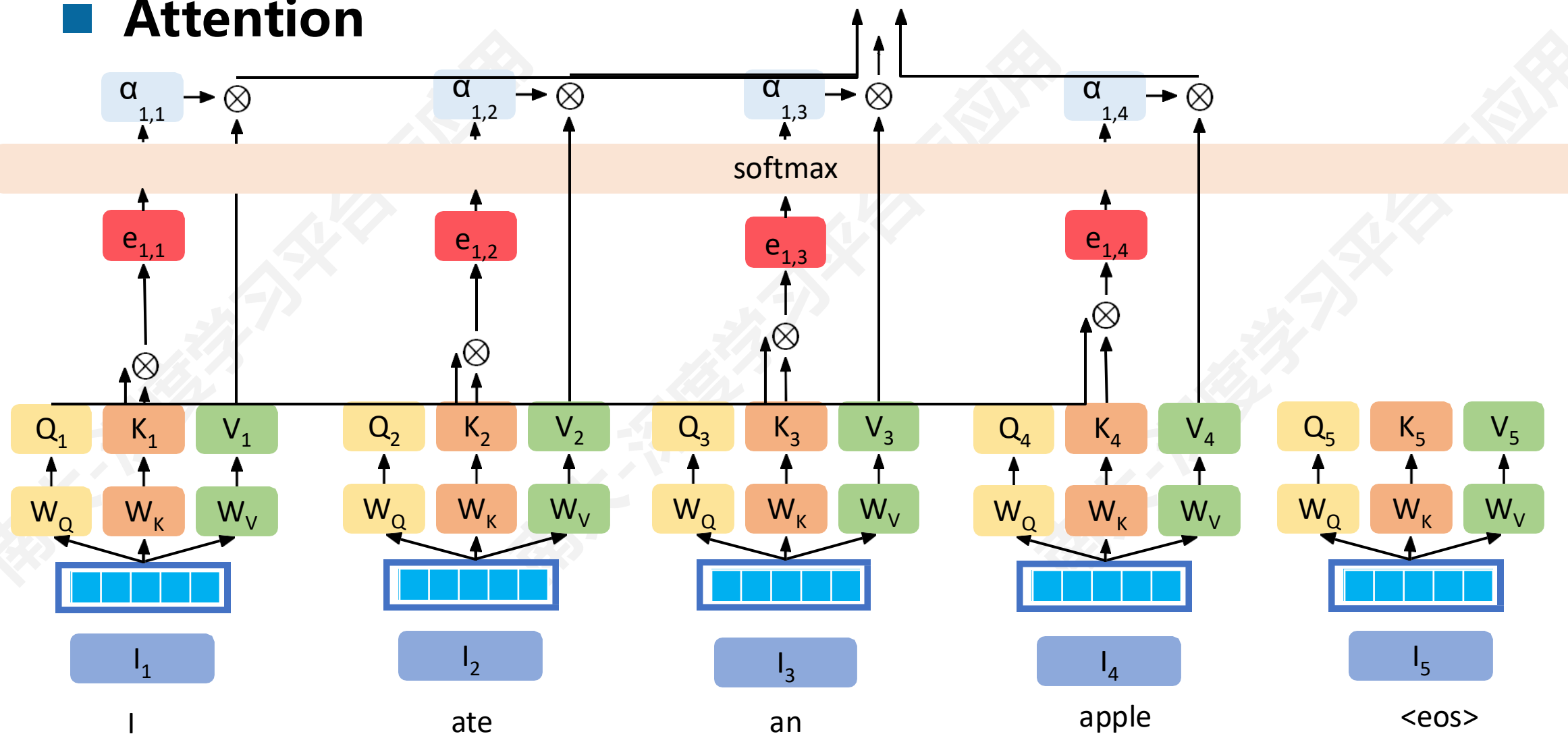
■ Attention



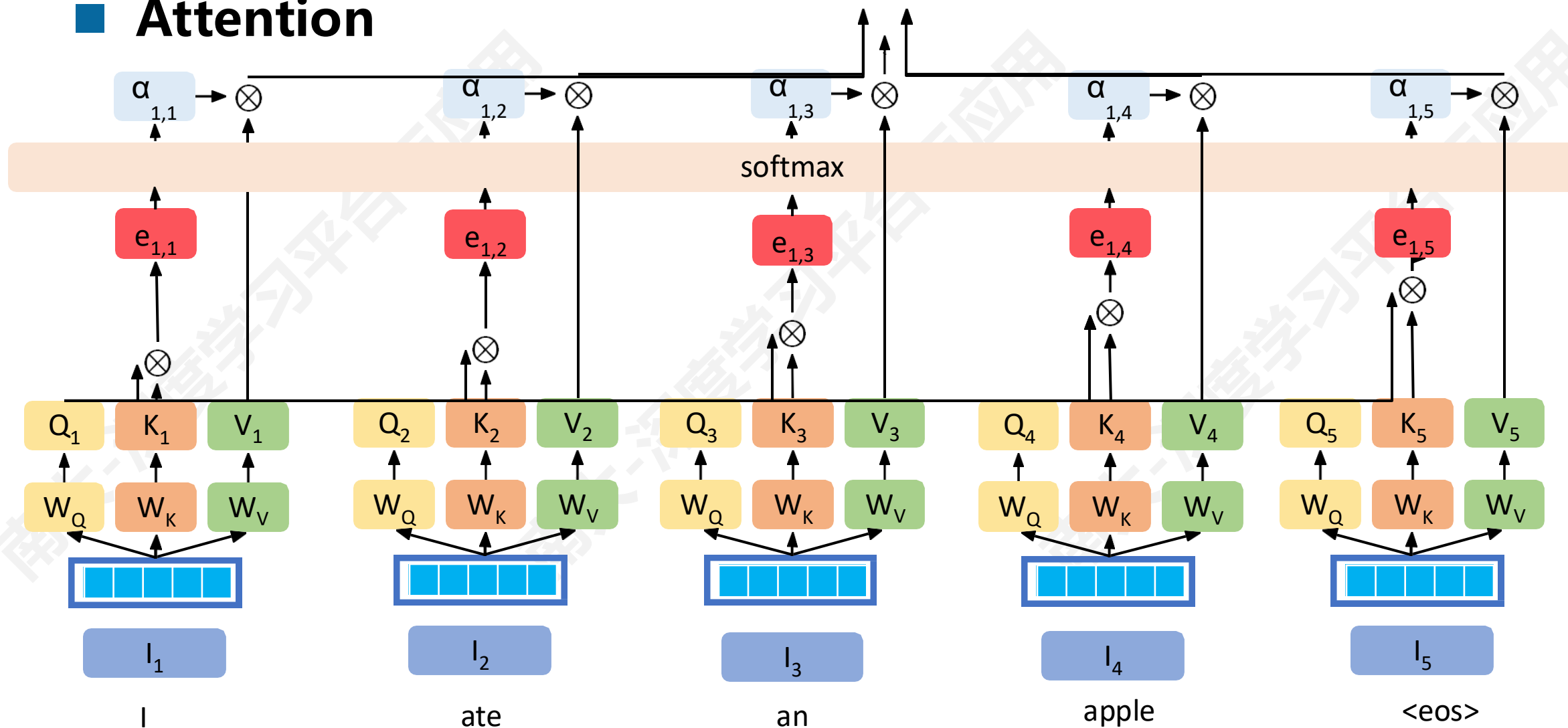
■ Attention



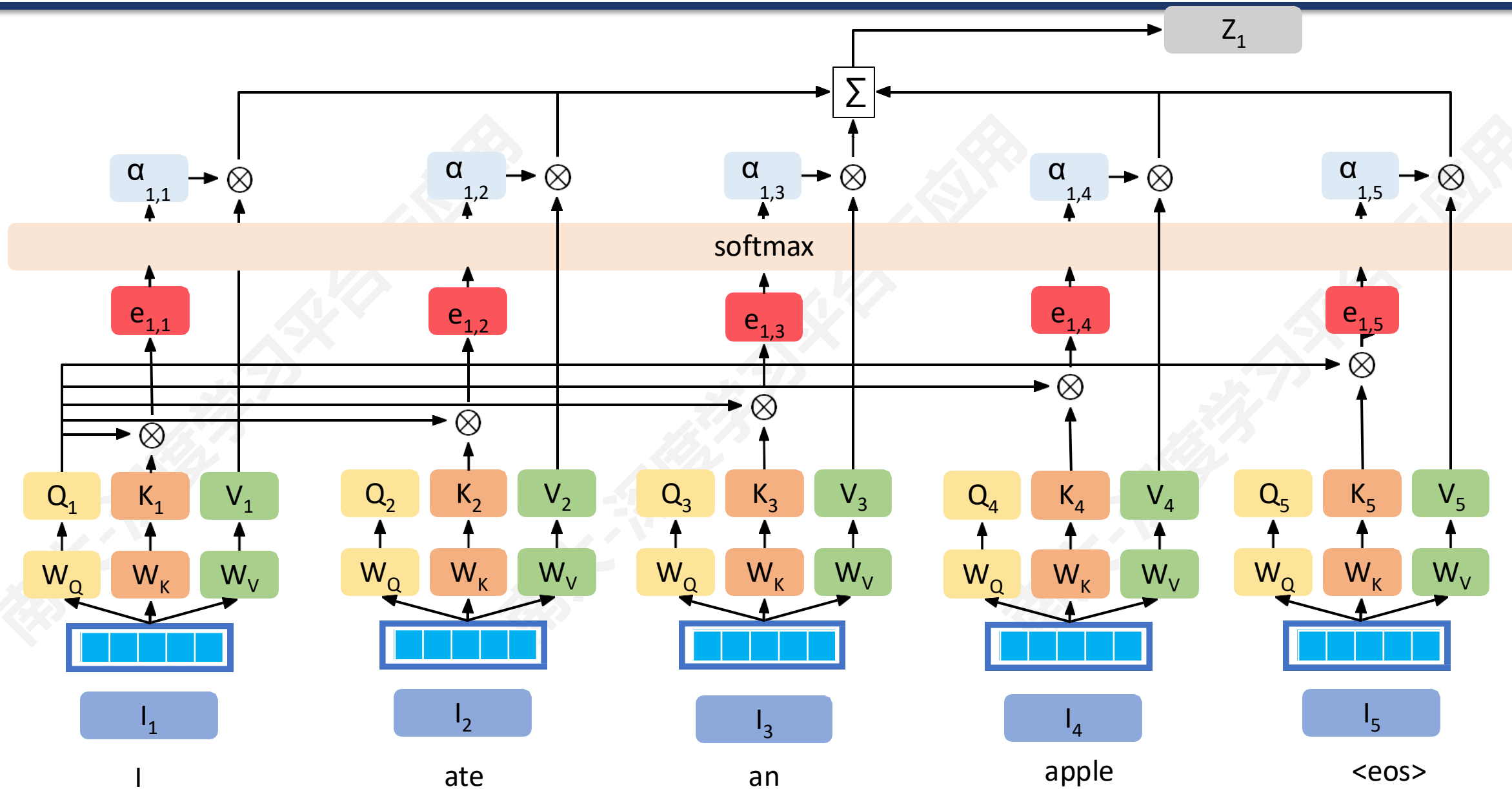
■ Attention



■ Attention

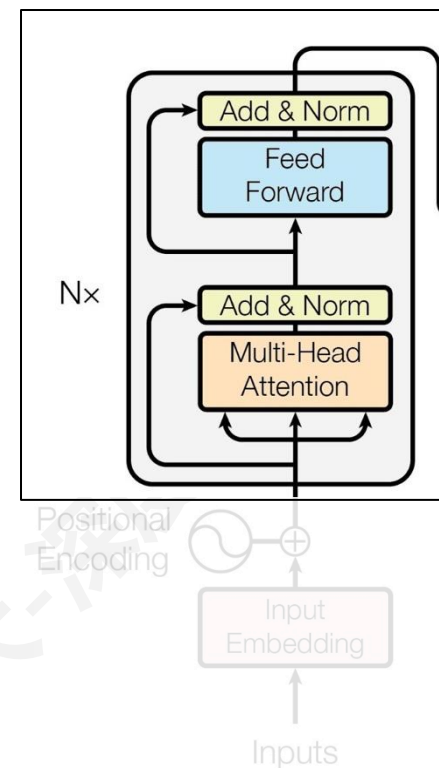


Transformer



■ Self Attention

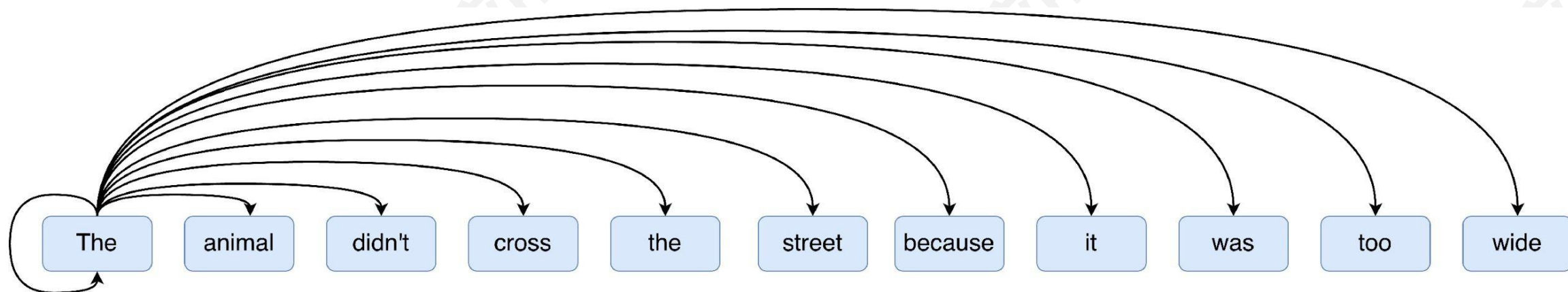
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



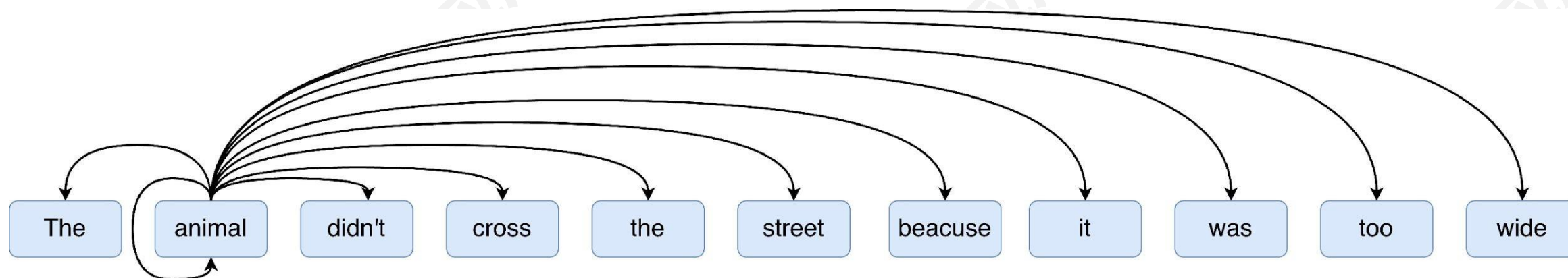
■ Self Attention

The animal didn't cross the street because it was too wide

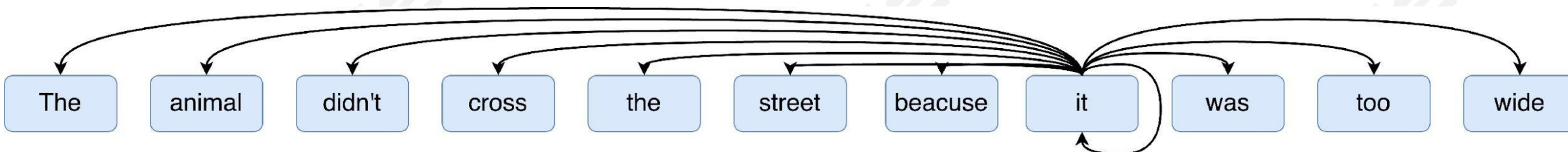
■ Self Attention



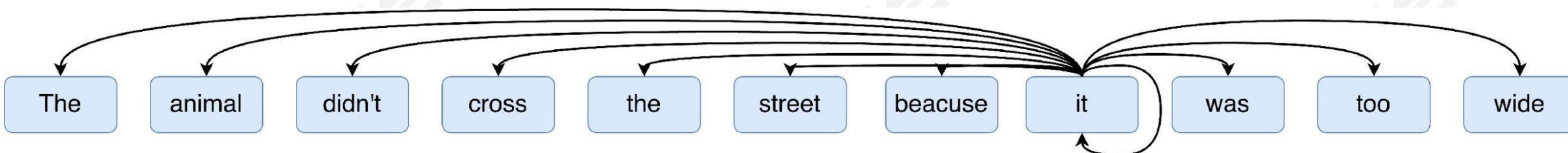
■ Self Attention



■ Self Attention



■ Self Attention



SELF

Query Inputs

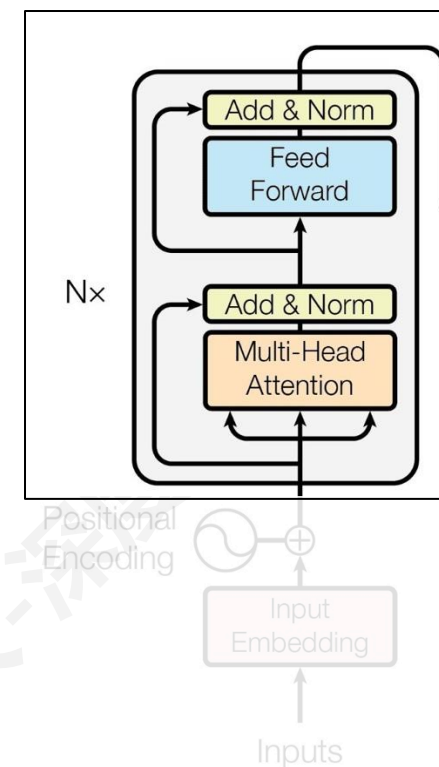
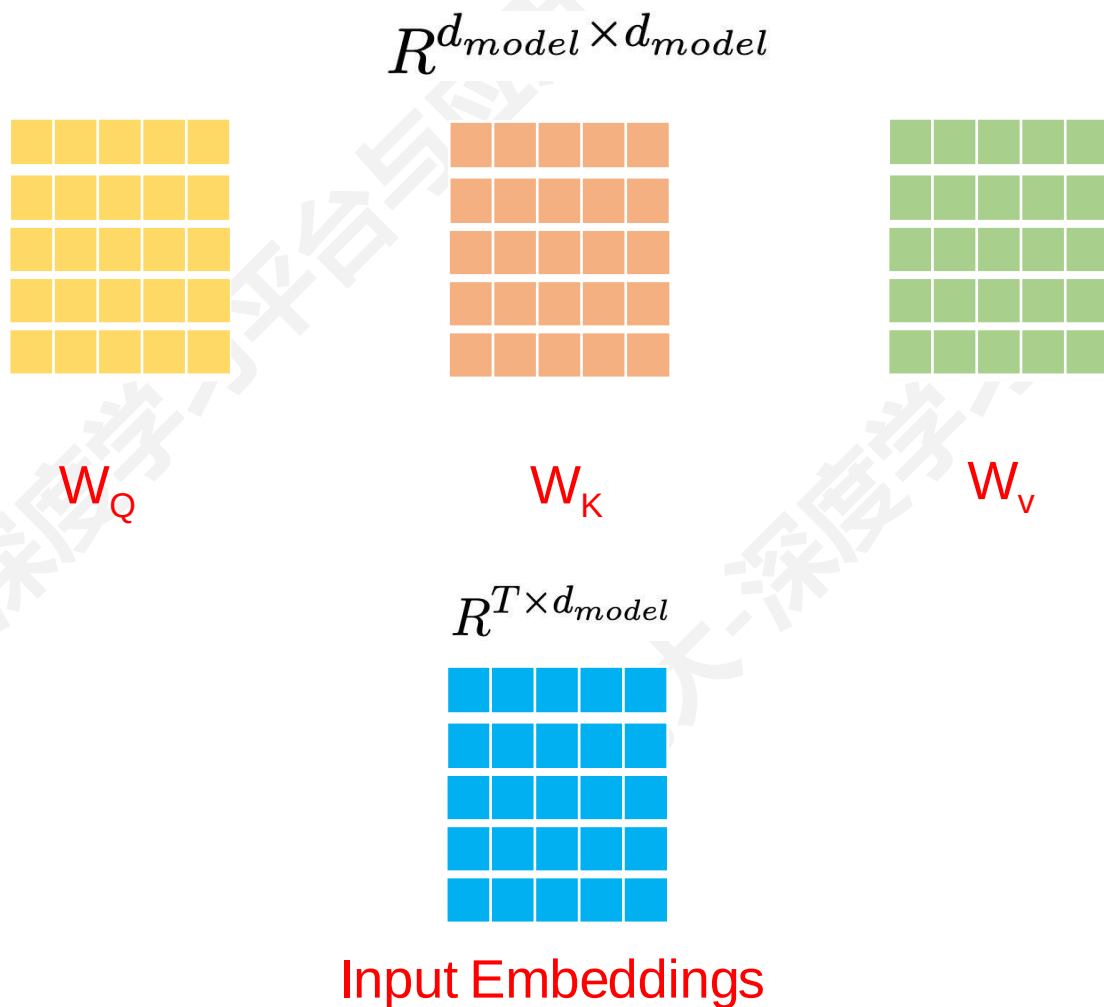
=

Key Inputs

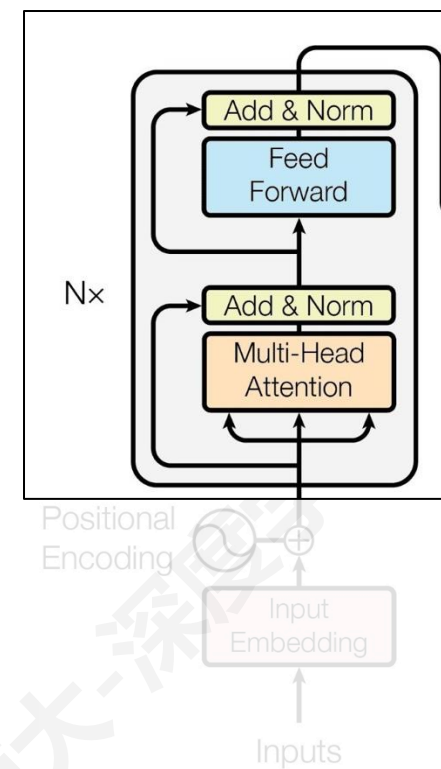
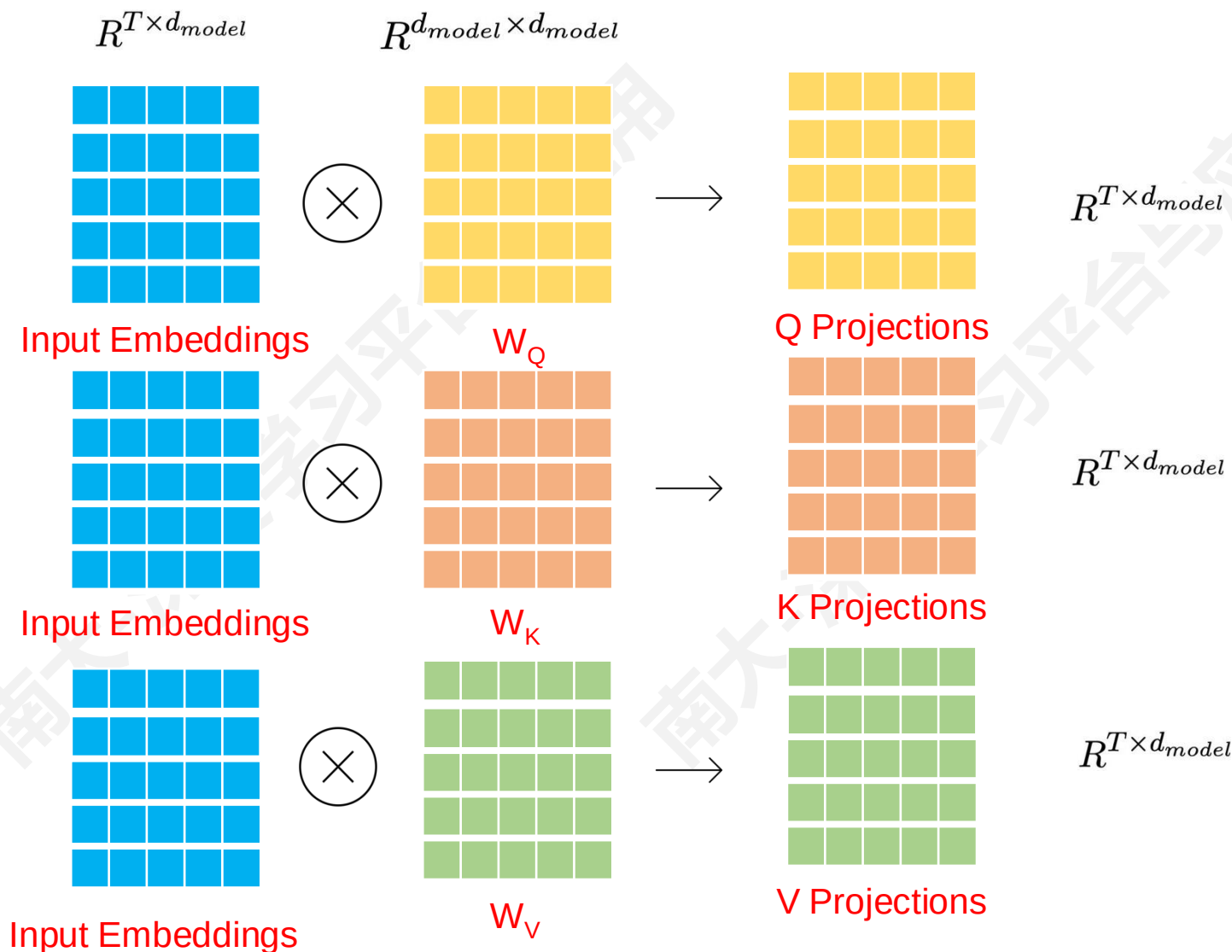
=

Value Inputs

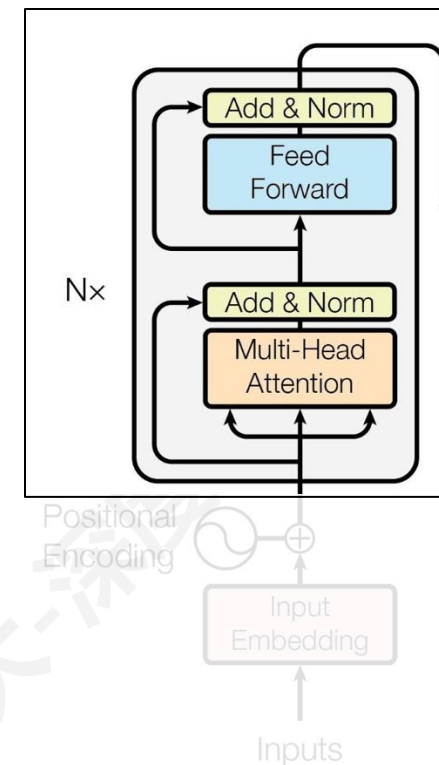
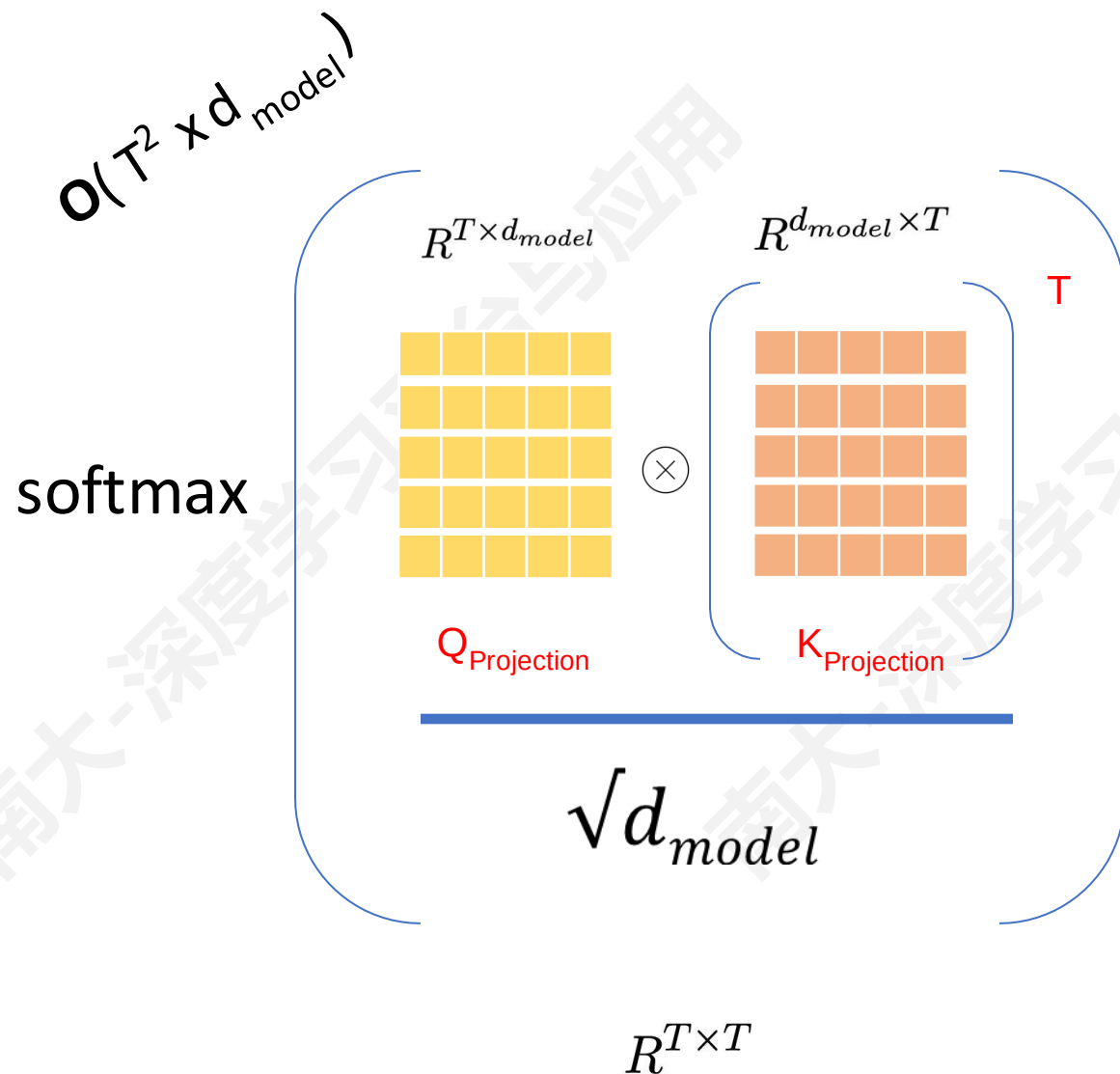
■ Self Attention



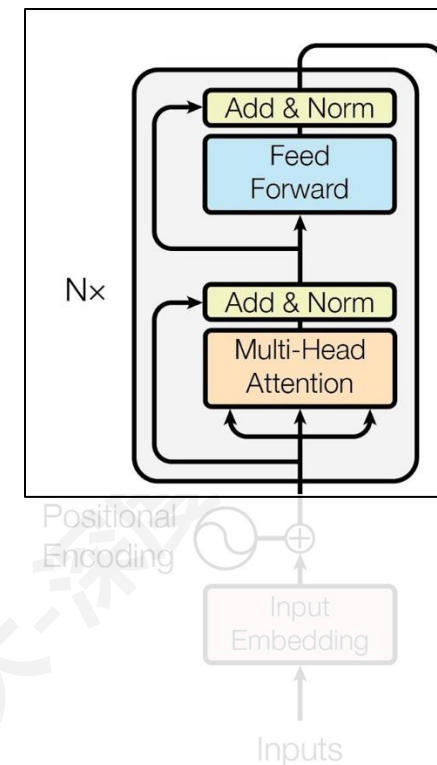
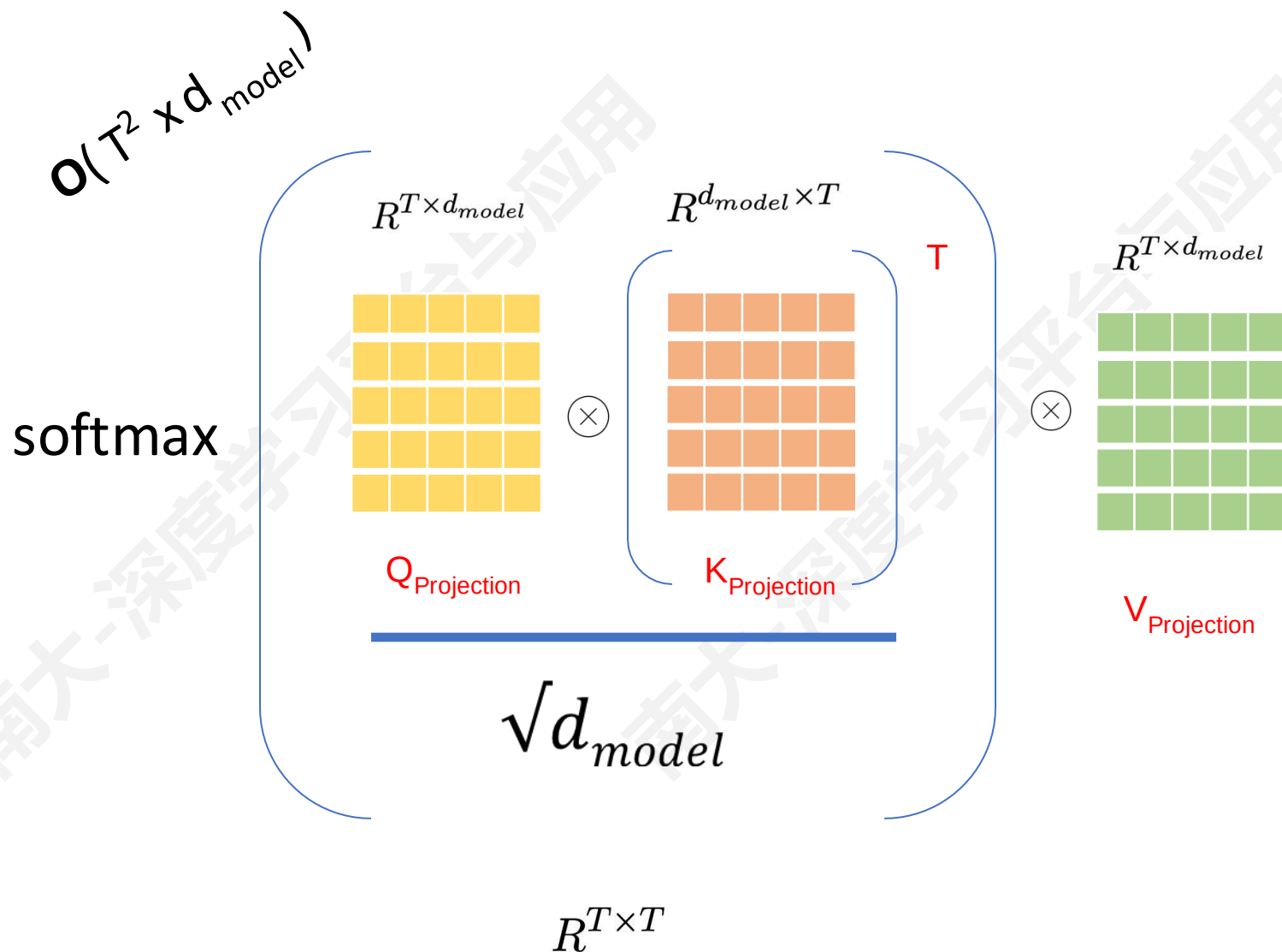
Transformer



Transformer

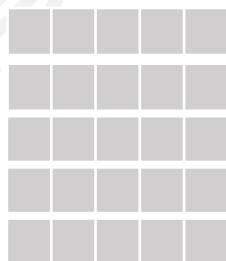


Transformer

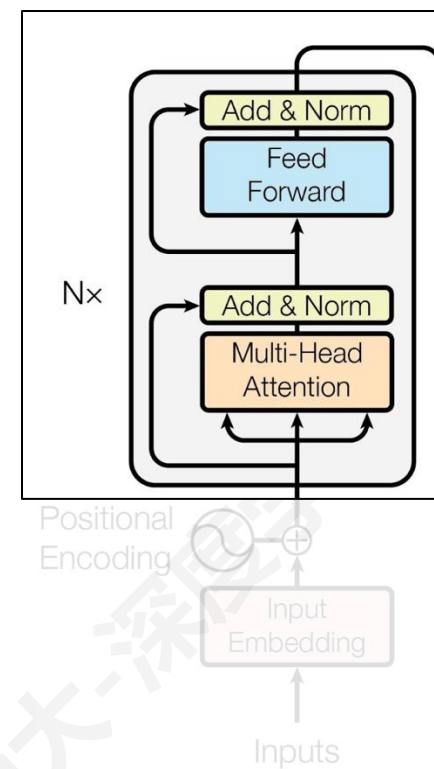


■ Self Attention

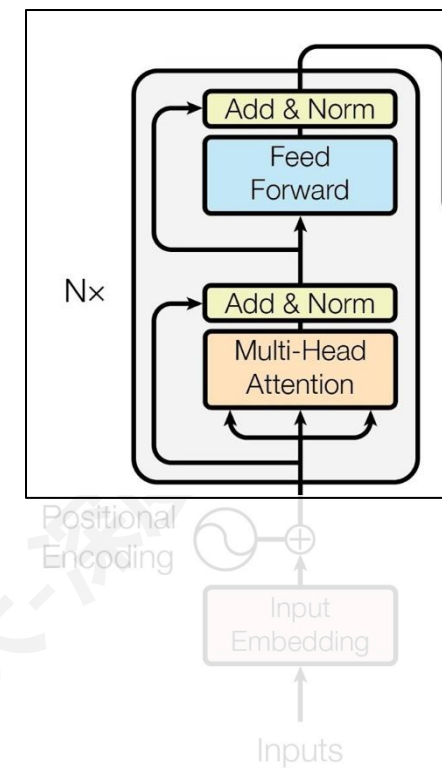
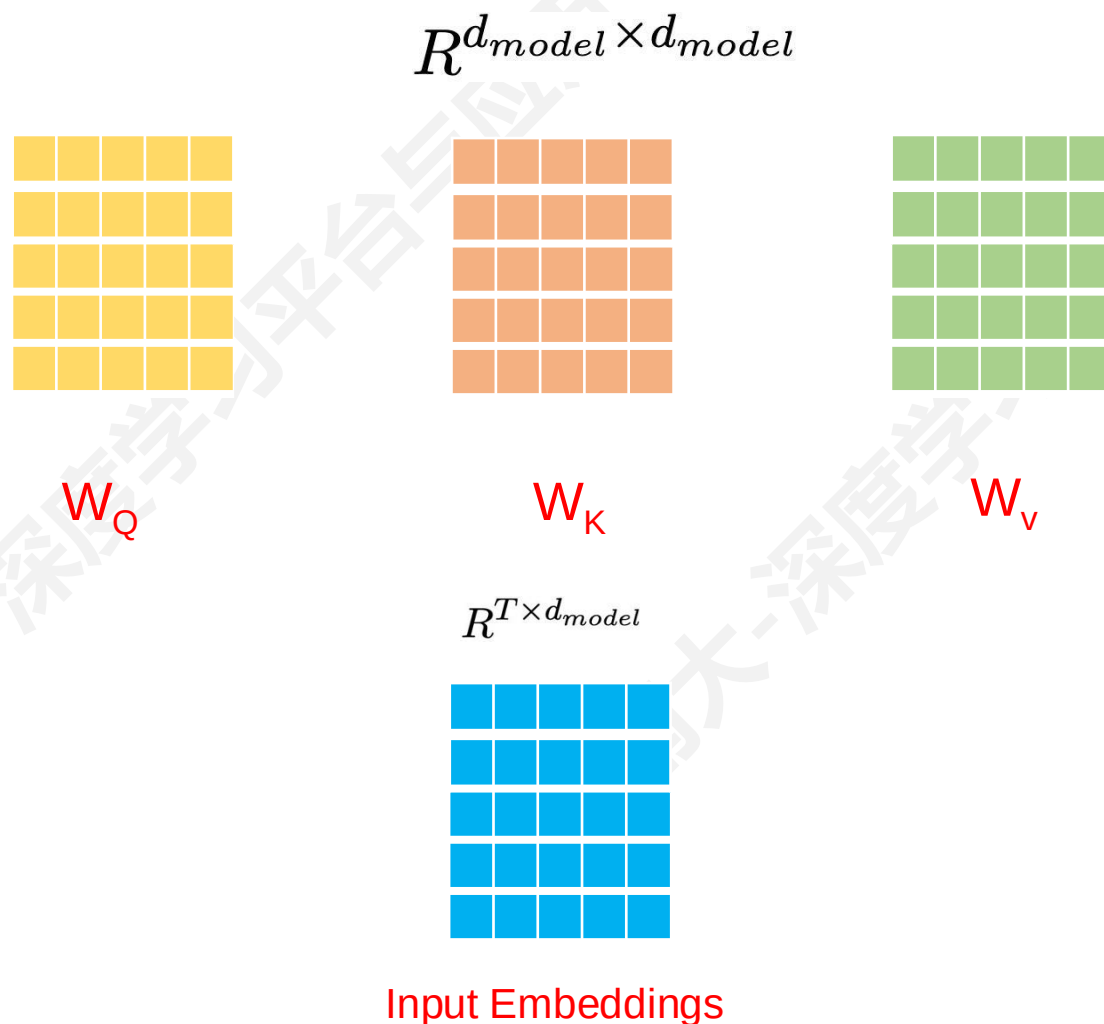
$$R^{T \times d_{model}}$$



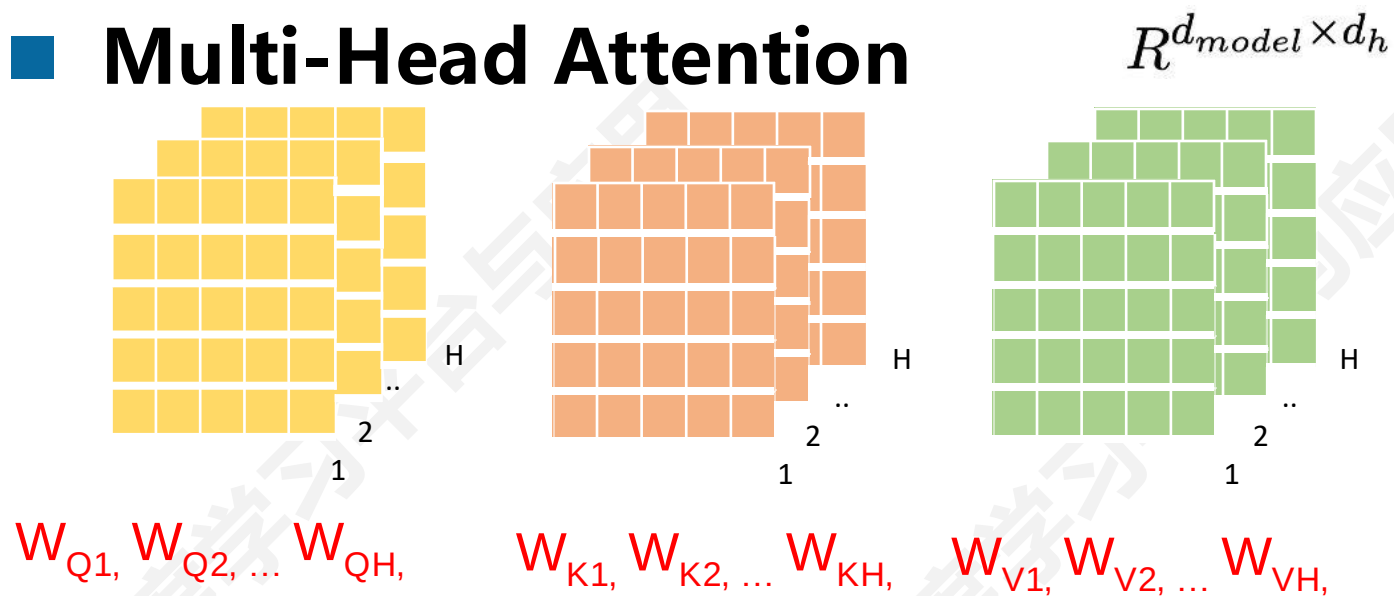
Attention: Z



■ Multi-Head Attention



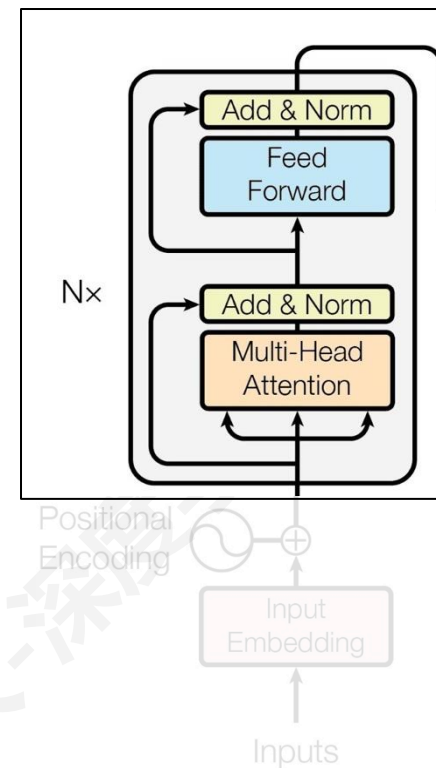
Multi-Head Attention



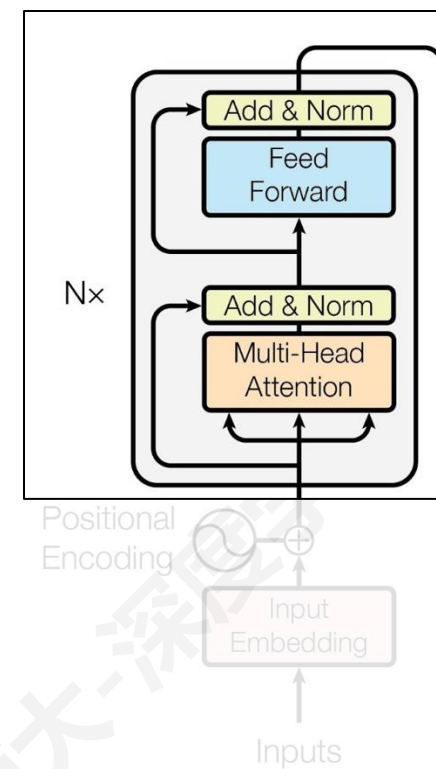
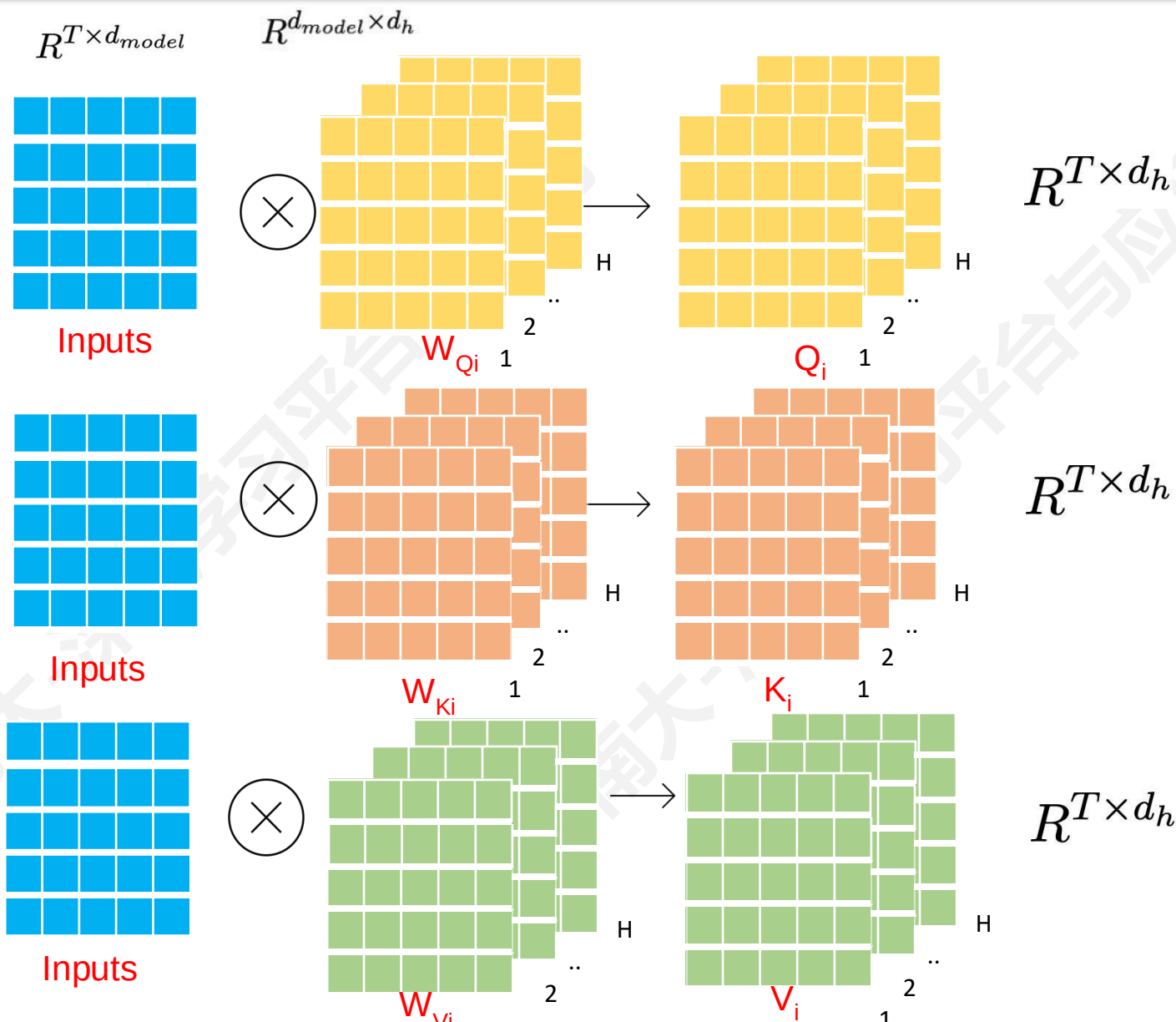
Input Embeddings

$$R^{T \times d_{model}}$$

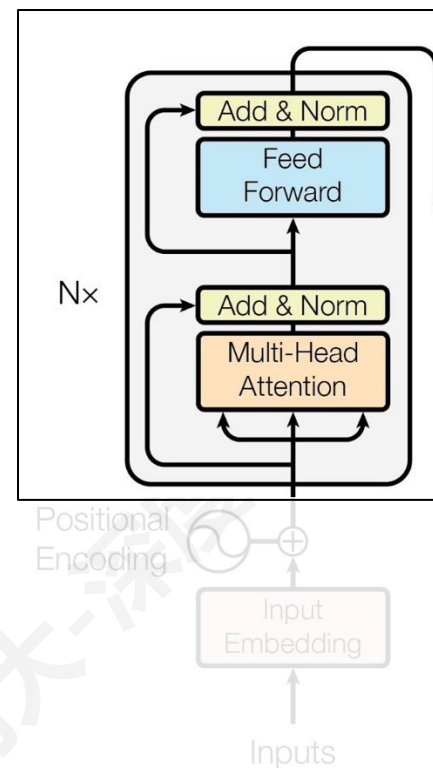
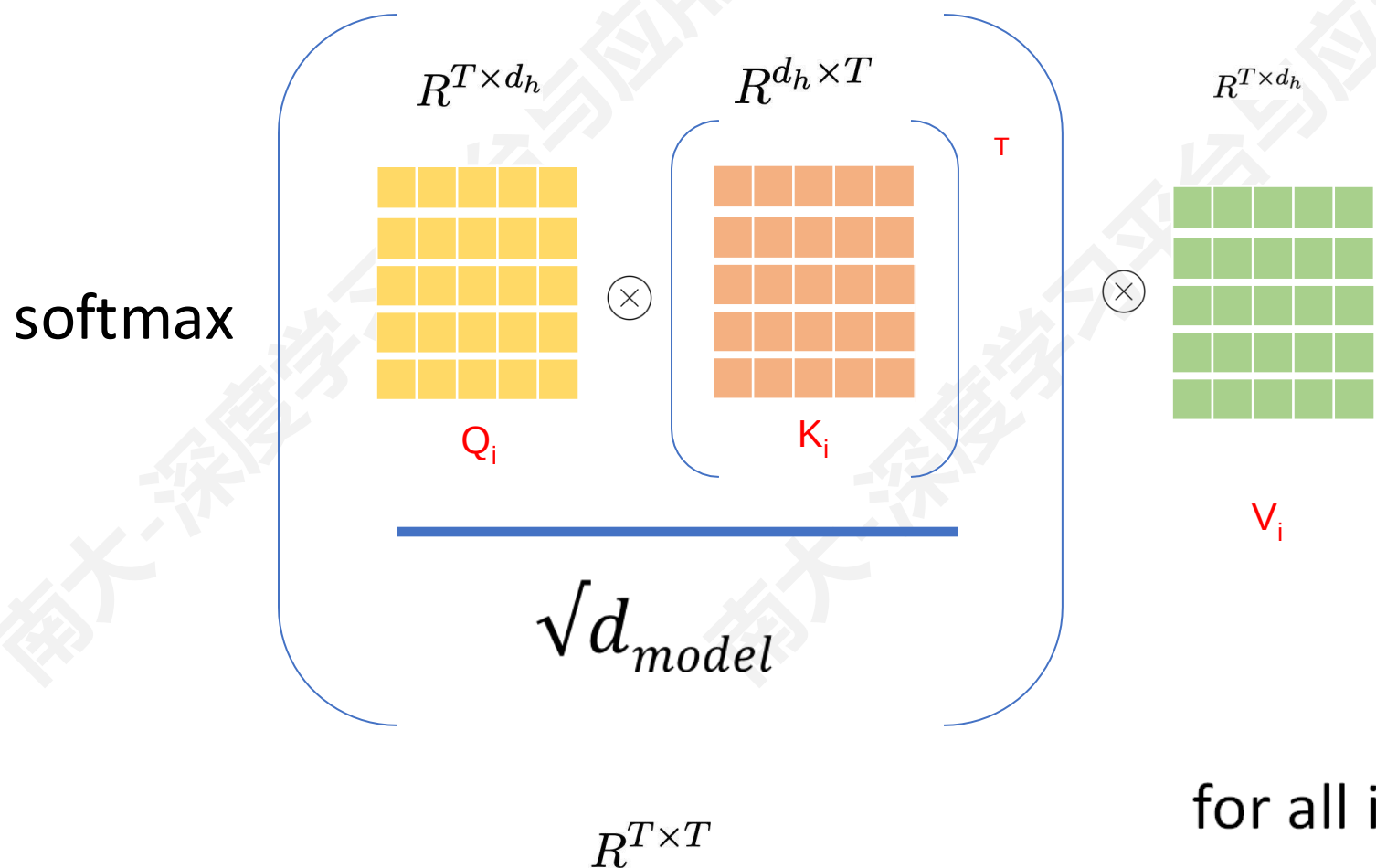
$$d_h = \frac{d_{model}}{h}$$



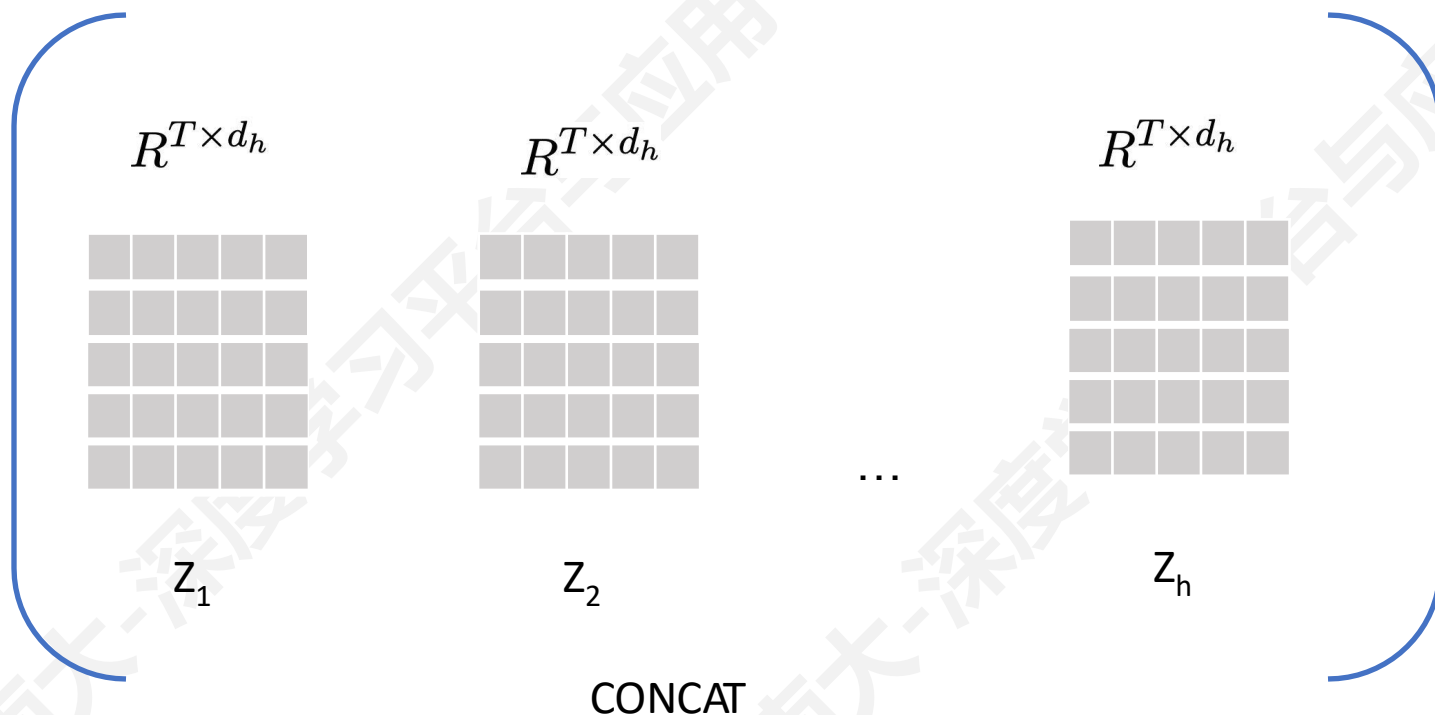
Transformer



■ Multi-Head Attention



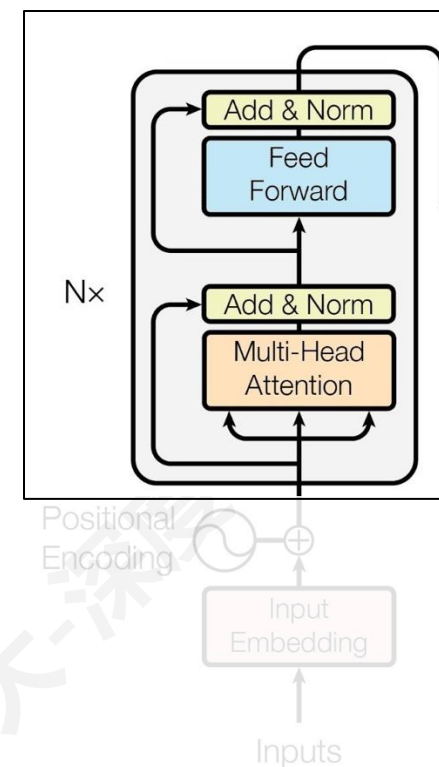
■ Multi-Head Attention



Multi Head Attention : Z

$$R^{T \times d_{model}}$$

$$d_h = \frac{d_{model}}{h}$$

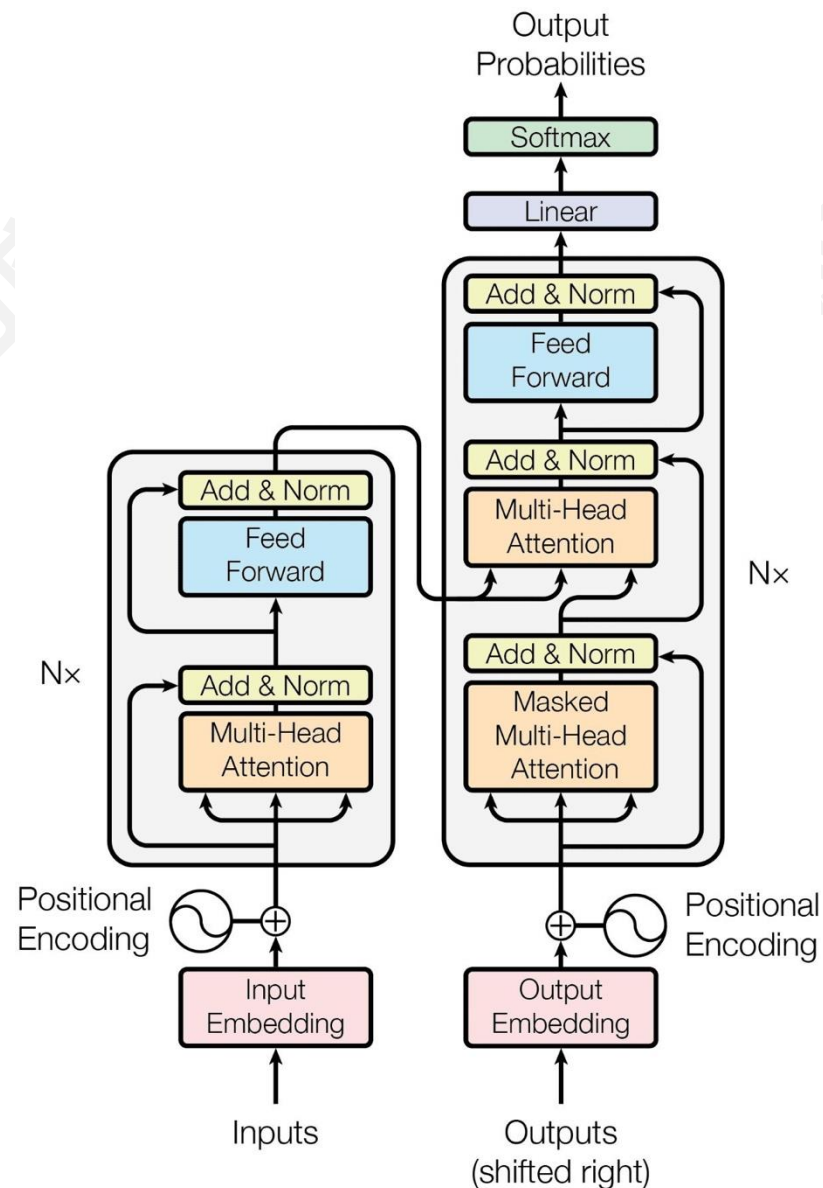


Transformer

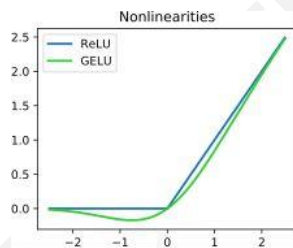
- ✓ Tokenization
- ✓ Input Embeddings
- ✓ Position Encodings
- ✓ Query, Key, & Value
- ✓ Attention

- Self Attention
- Multi-Head Attention
- Feed Forward
- Add & Norm
- Encoders

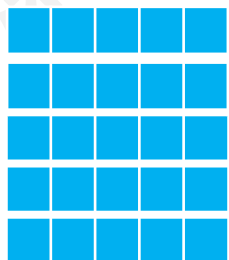
- Masked Attention
- Encoder Decoder Attention
- Linear
- Softmax
- Decoders
- Encoder-Decoder Models



■ Feed Forward



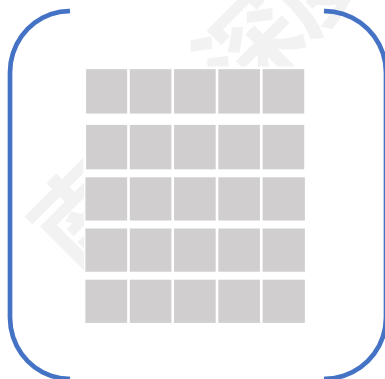
Feed Forward



Input



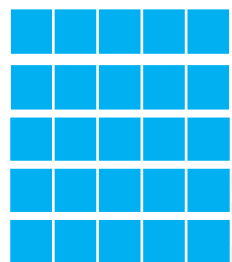
Residuals



Norm(Z)

```
class FeedForward(nn.Module):  
    def __init__(self, dim, hidden_dim):  
        super().__init__()  
        self.net = nn.Sequential(  
            nn.LayerNorm(dim),  
            nn.Linear(dim, hidden_dim),  
            nn.GELU(),  
            nn.Linear(hidden_dim, dim),  
        )  
    def forward(self, x):  
        return self.net(x)
```

■ Add & Norm



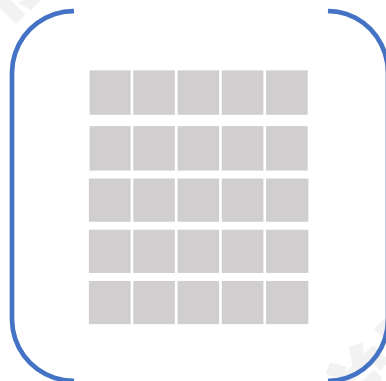
Input

Normalization

Mean 0, Std dev 1

稳定训练

正则化

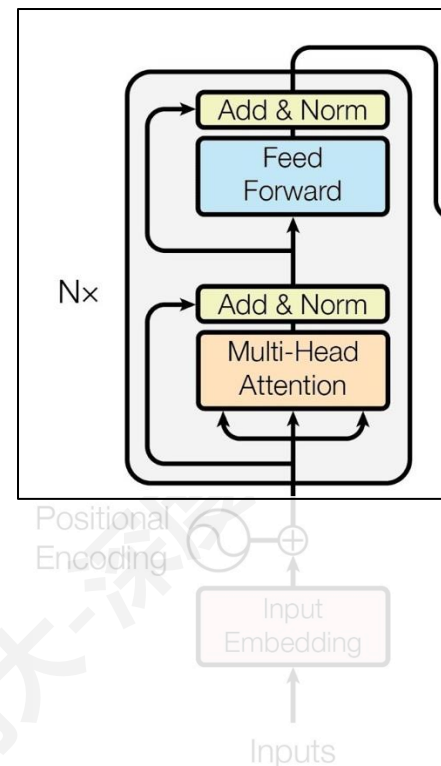


Norm(Z)

Add Residuals

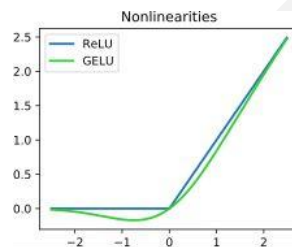
避免梯度消失

训练更深的网络

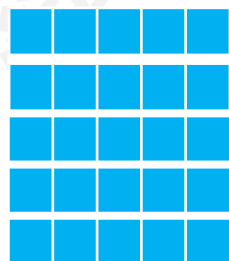


Transformer

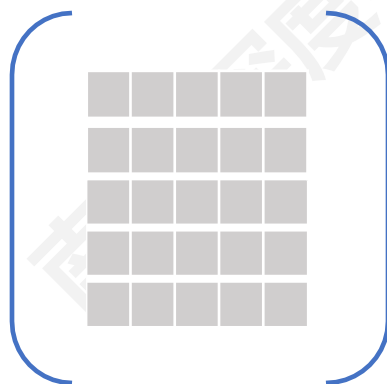
Add & Norm



Feed Forward

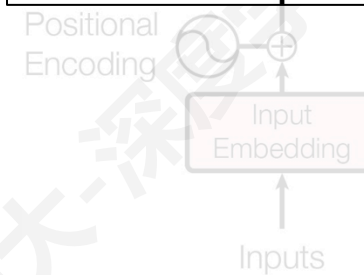
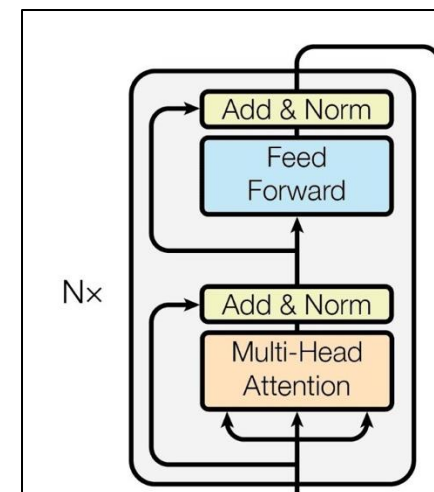


Residuals



Input

Norm(Z)



```
class Attention(nn.Module):
    def __init__(self, dim, heads = 8, dim_head = 64):
        super().__init__()
        inner_dim = dim_head * heads
        self.heads = heads
        self.scale = dim_head ** -0.5
        self.norm = nn.LayerNorm(dim)

        self.attend = nn.Softmax(dim = -1)

        self.to_qkv = nn.Linear(dim, inner_dim * 3, bias = False)
        self.to_out = nn.Linear(inner_dim, dim, bias = False)

    def forward(self, x):
        x = self.norm(x)

        qkv = self.to_qkv(x).chunk(3, dim = -1)
        q, k, v = map(lambda t: rearrange(t, 'b n (h d) -> b h n d', h = self.heads), qkv)

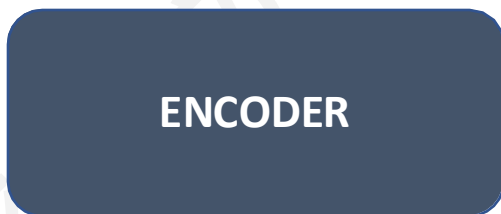
        dots = torch.matmul(q, k.transpose(-1, -2)) * self.scale

        attn = self.attend(dots)

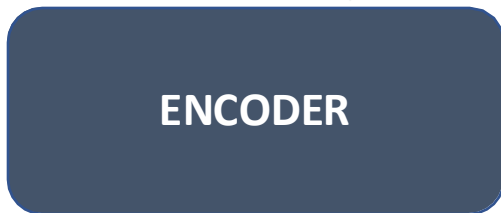
        out = torch.matmul(attn, v)
        out = rearrange(out, 'b h n d -> b n (h d)')
        return self.to_out(out)
```

■ Encoder

Encoder



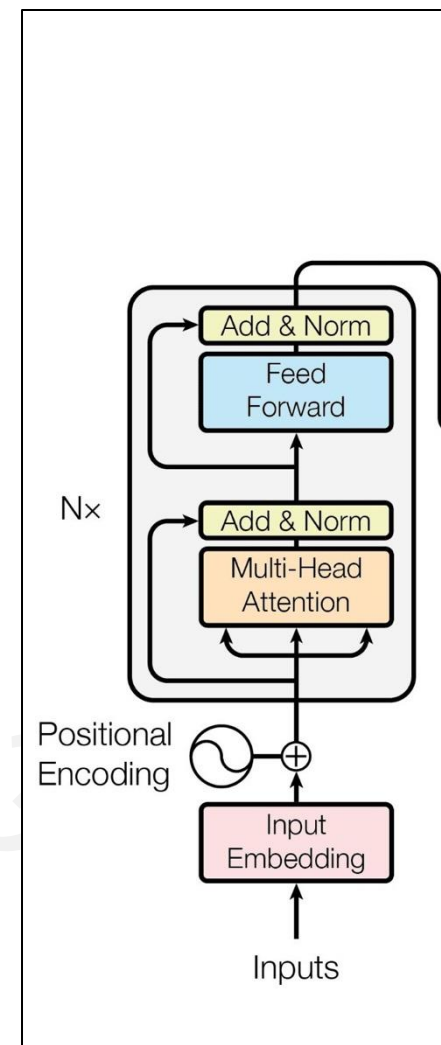
⋮



Input to Encoder _{$i+1$}

↑

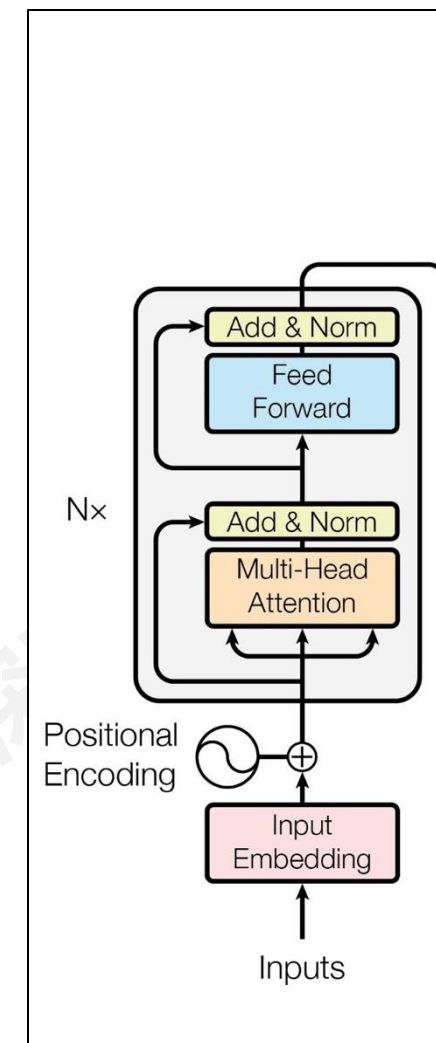
Output from Encoder _{i}



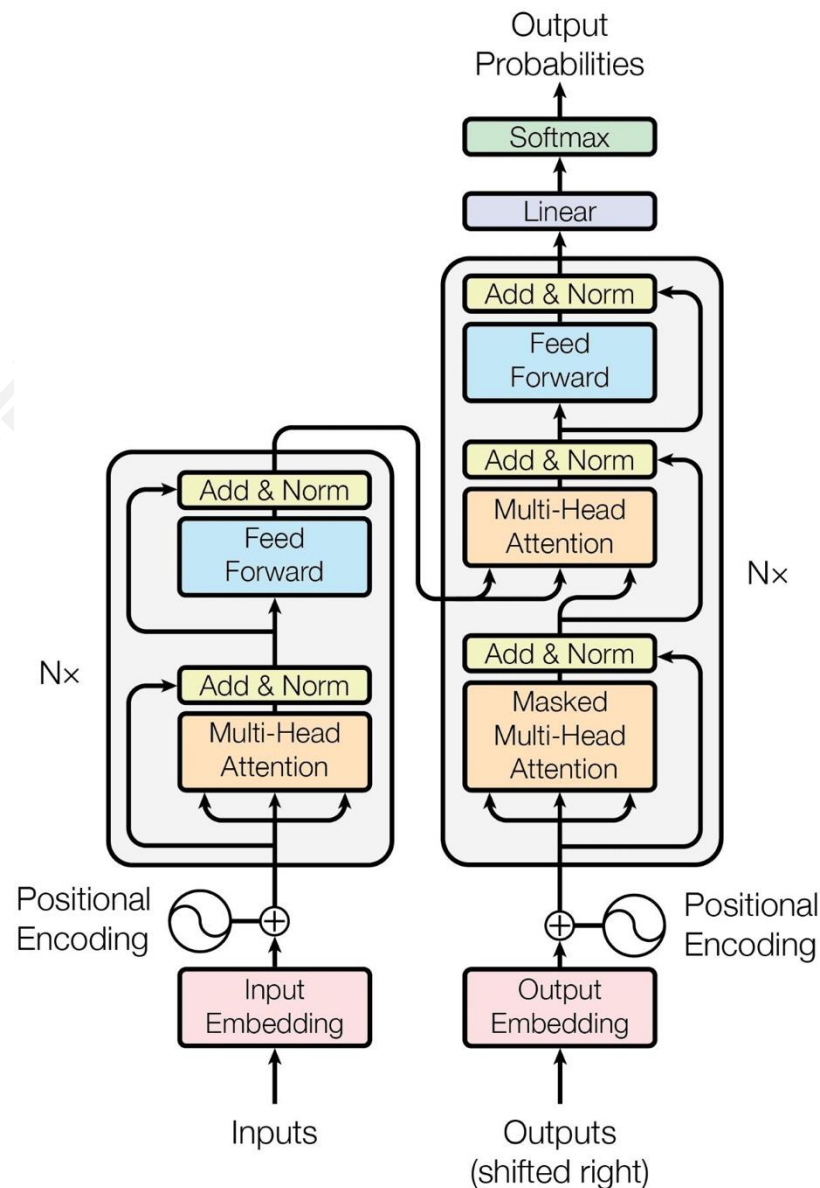
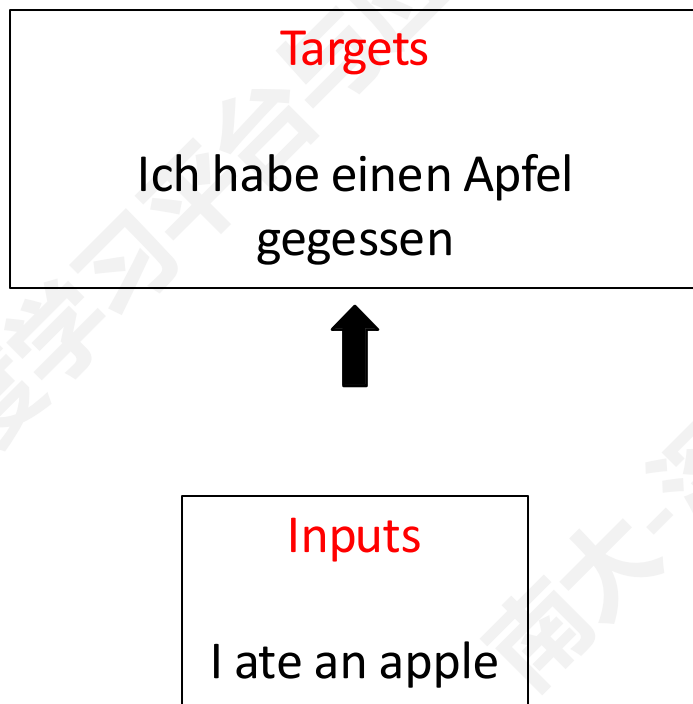
Transformer

- ✓ Tokenization
- ✓ Input Embeddings
- ✓ Position Encodings
- ✓ Query, Key, & Value
- ✓ Attention
- ✓ Self Attention
- ✓ Multi-Head Attention
- ✓ Feed Forward
- ✓ Add & Norm
- ✓ Encoders

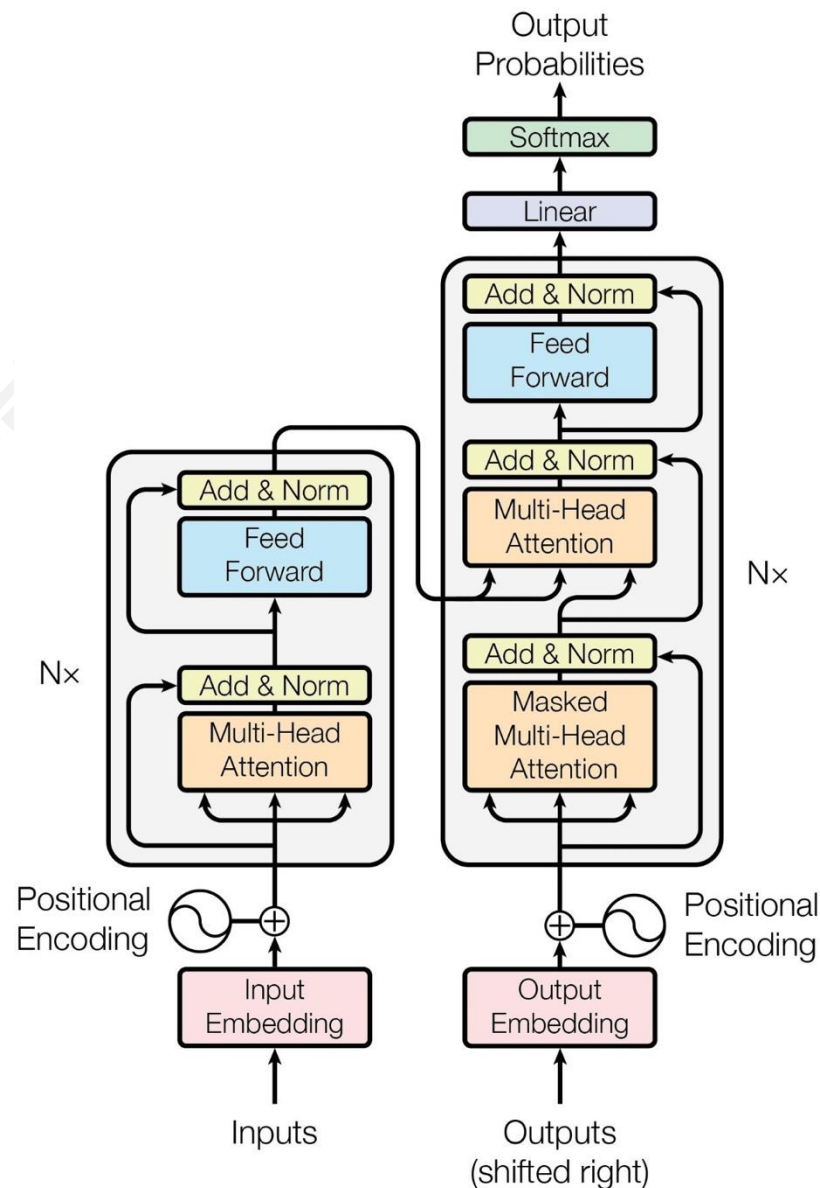
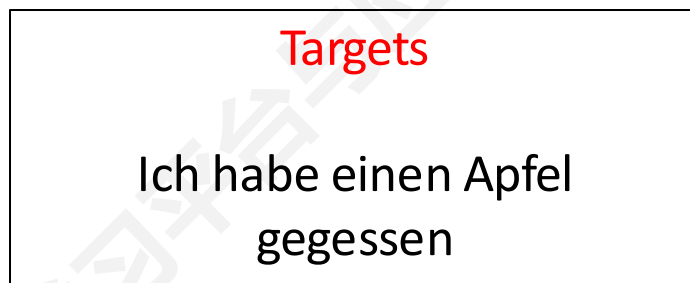
- Masked Attention
- Encoder Decoder Attention
- Linear
- Softmax
- Decoders
- Encoder-Decoder Models



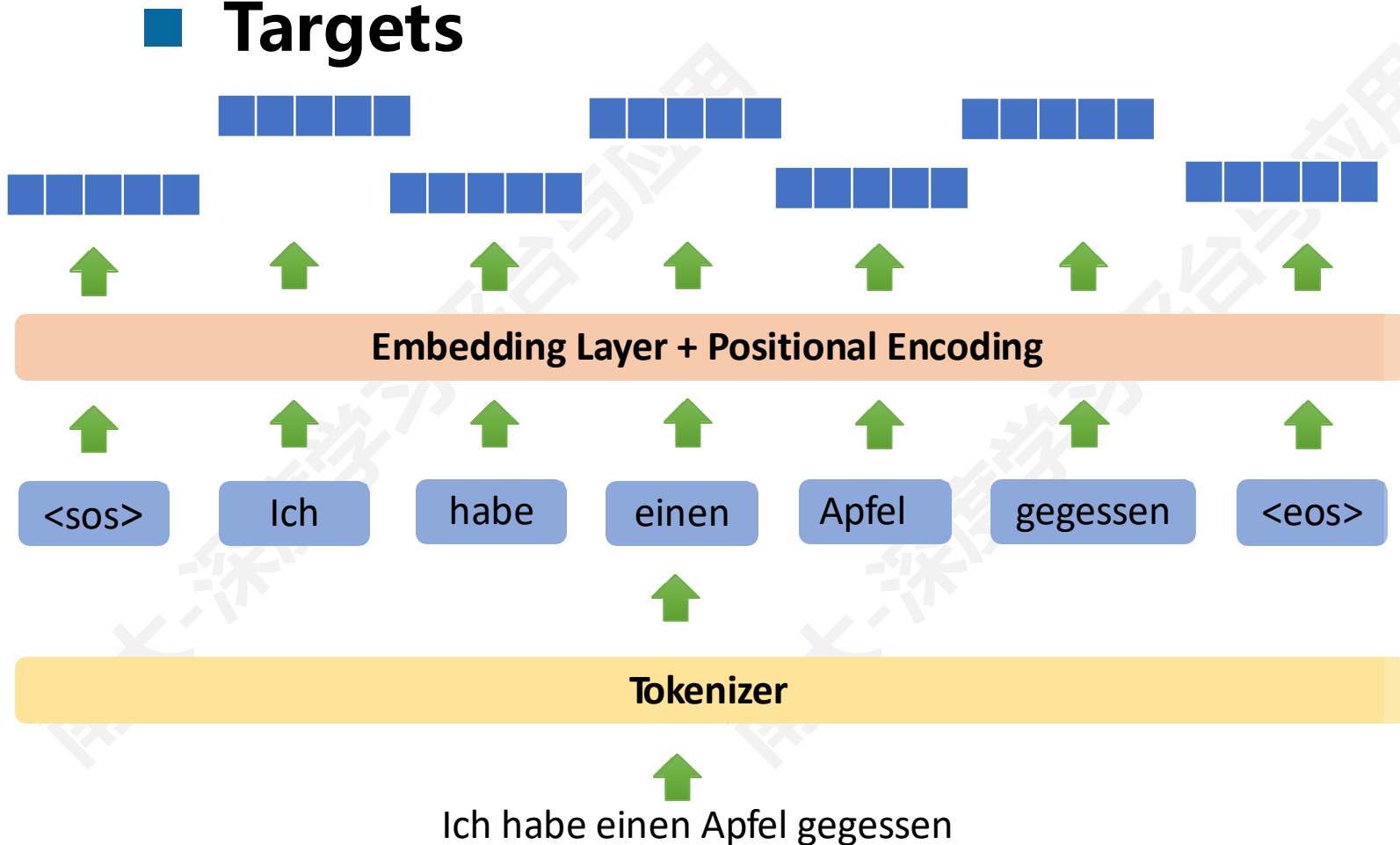
■ 例子：机器翻译



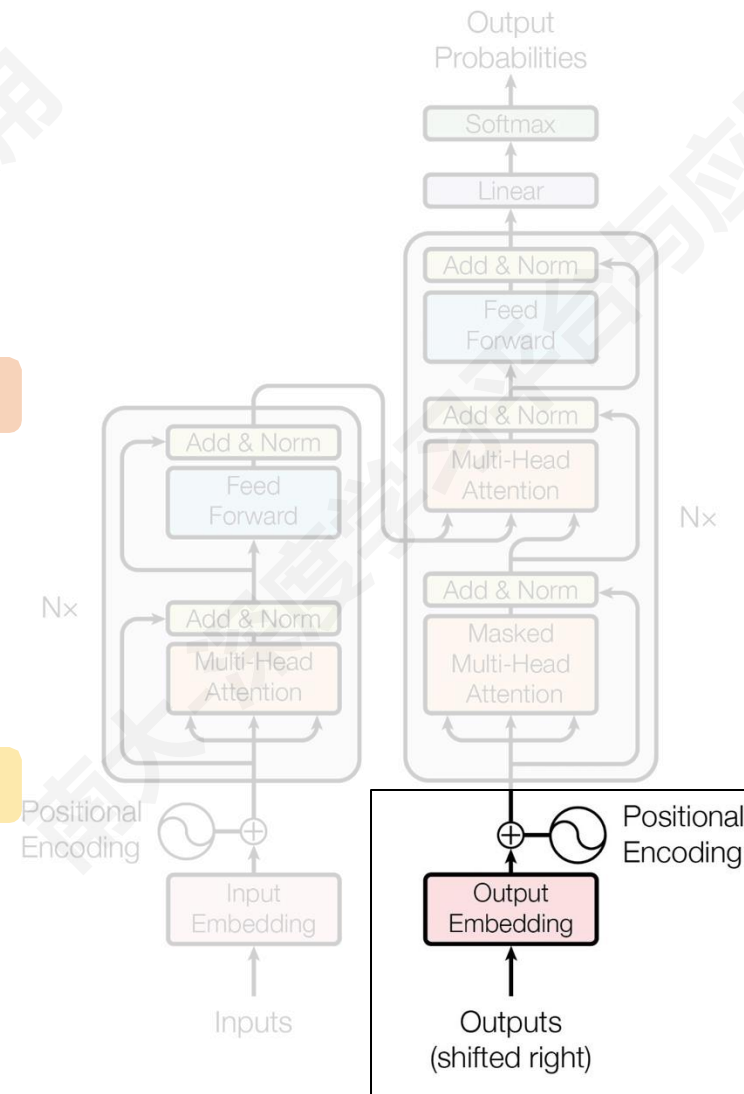
■ 例子：机器翻译



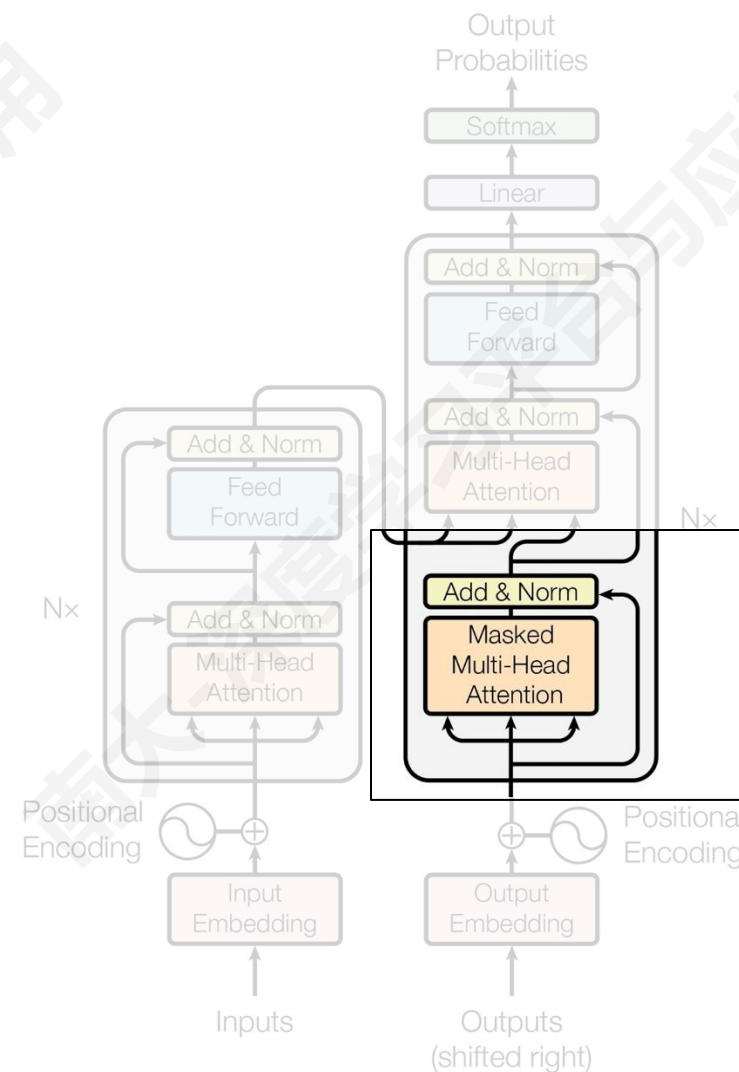
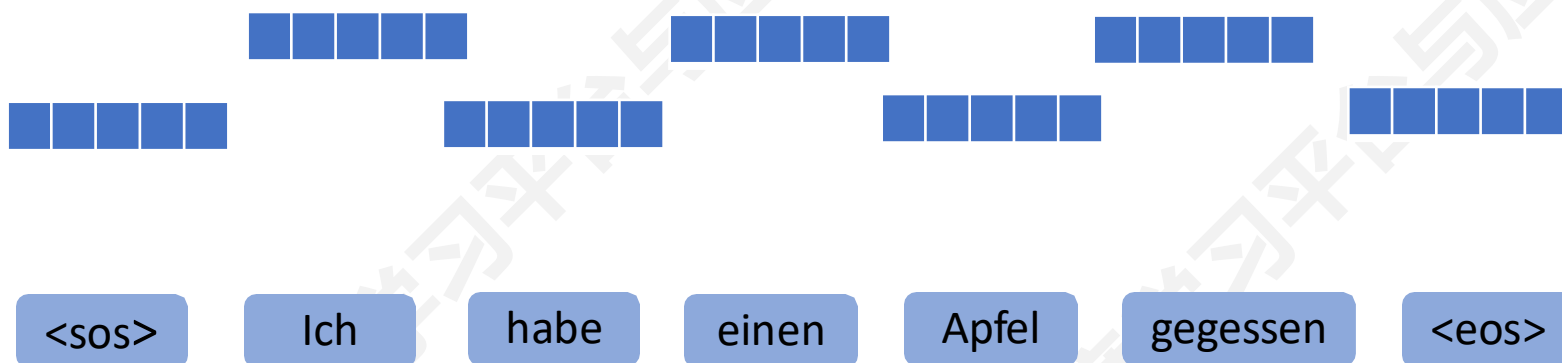
Targets



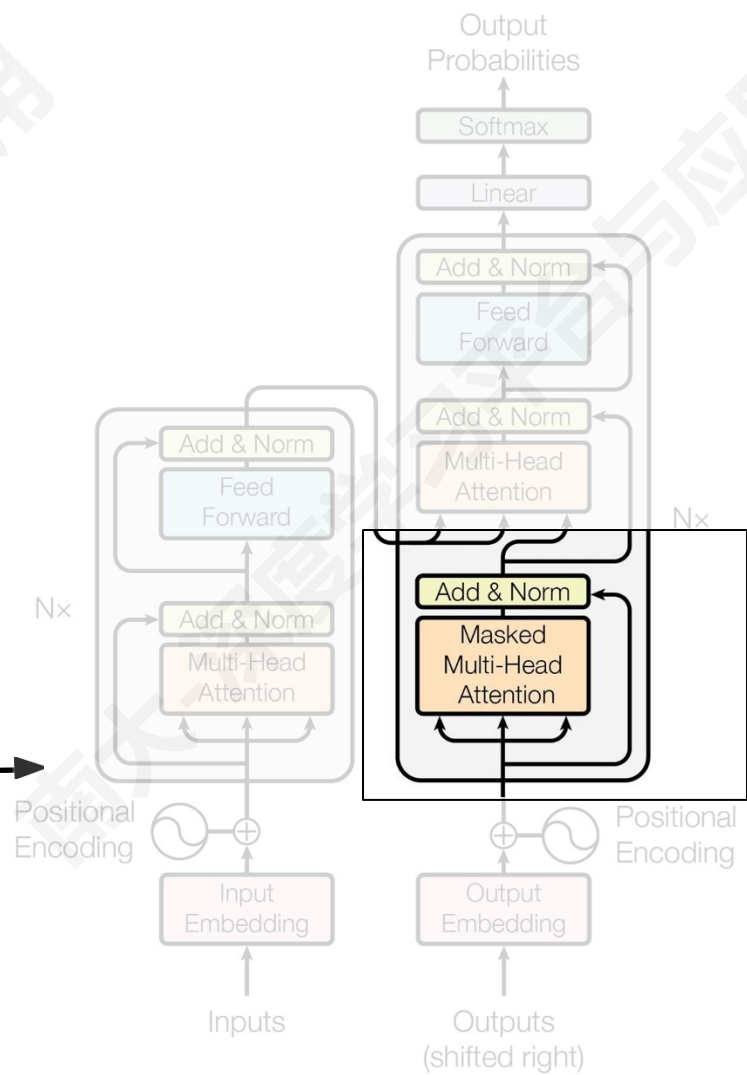
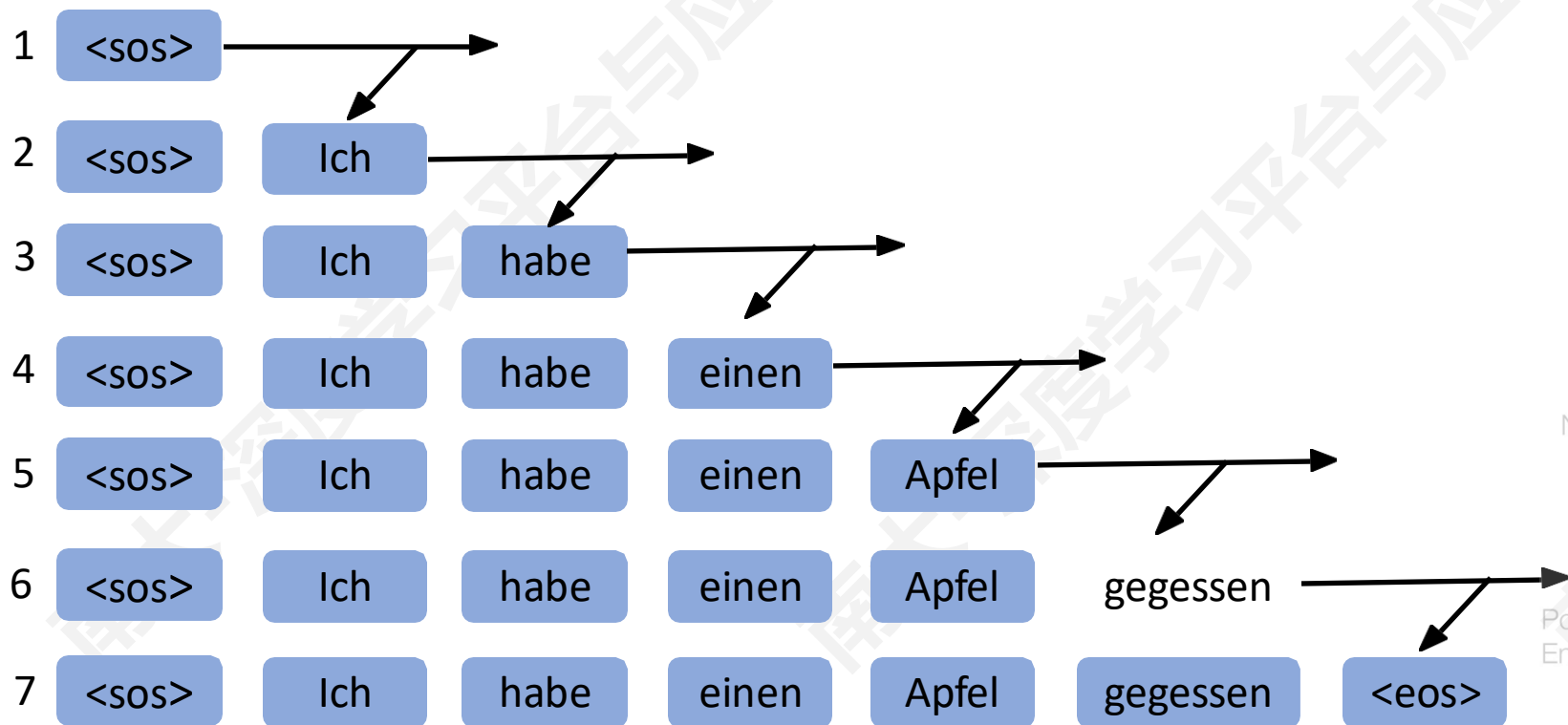
Generate Target Embeddings



■ Masked Multi Head Attention

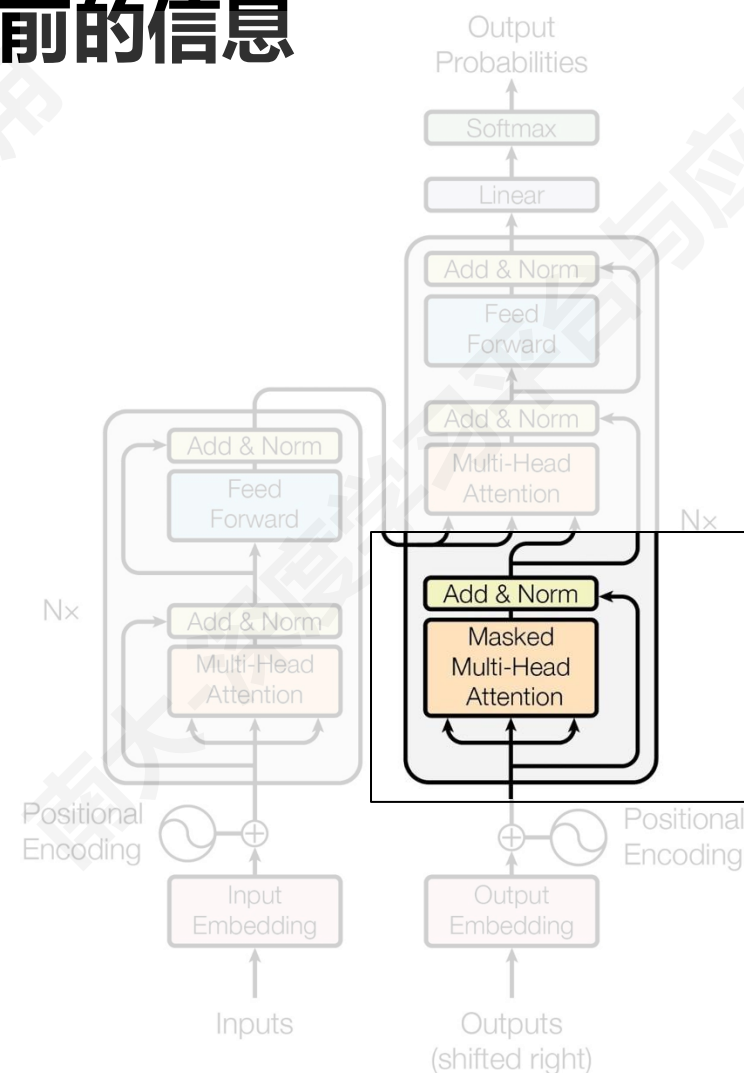


■ 测试阶段



■ 训练阶段：t 时刻只能看到 t-1 时刻及之前的信息

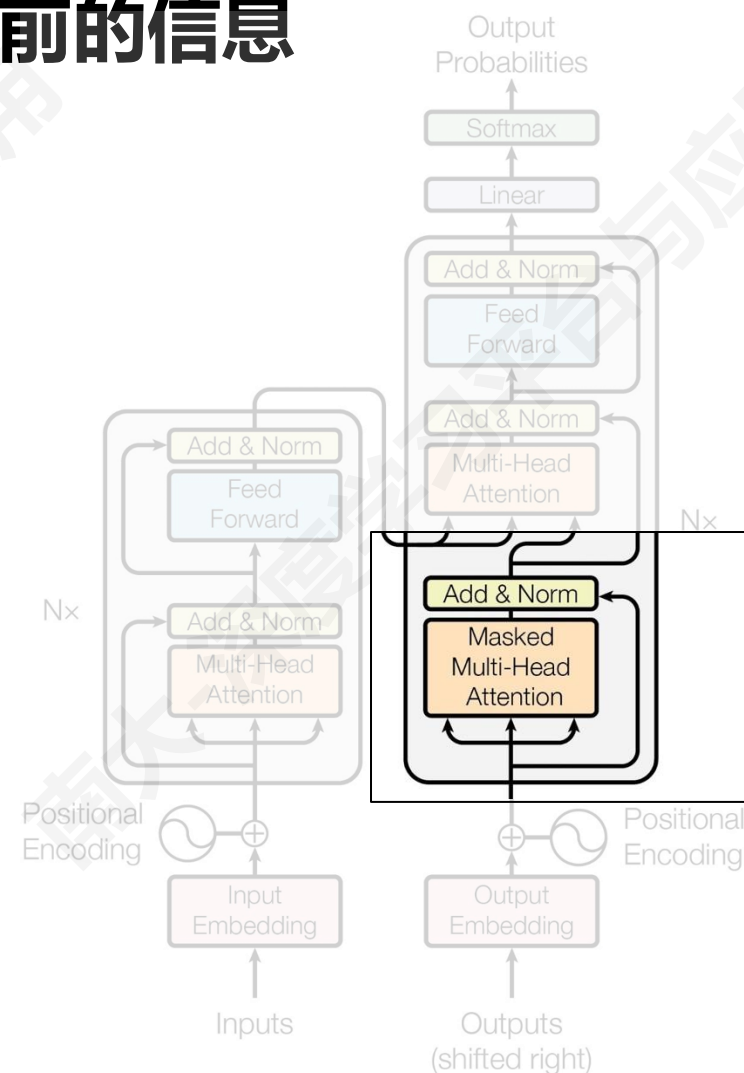
1	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
2	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
3	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
4	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
5	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
6	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>
7	<sos>	Ich	habe	einen	Apfel	gegessen	<eos>



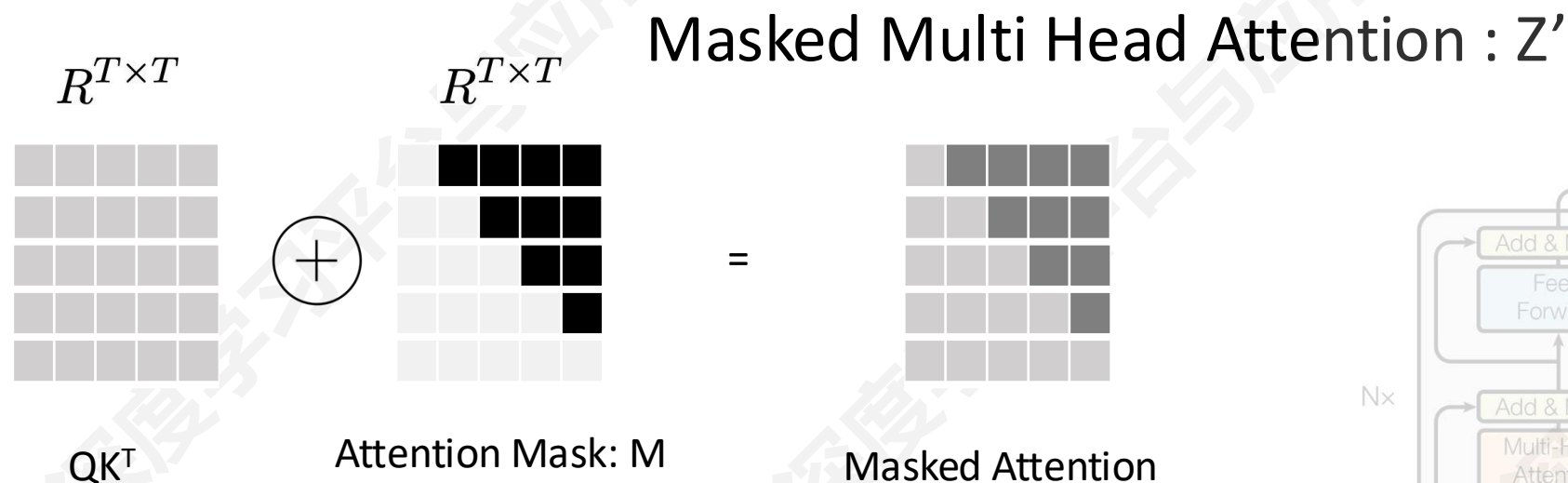
■ 训练阶段：t 时刻只能看到 t-1 时刻及之前的信息

1	<sos>	- ∞	- ∞	- ∞	- ∞	- ∞
2	<sos>	Ich	- ∞	- ∞	- ∞	- ∞
3	<sos>	Ich	habe	- ∞	- ∞	- ∞
4	<sos>	Ich	habe	einen	- ∞	- ∞
5	<sos>	Ich	habe	einen	Apfel	- ∞
6	<sos>	Ich	habe	einen	Apfel	gegessen
7	<sos>	Ich	habe	einen	Apfel	gegessen

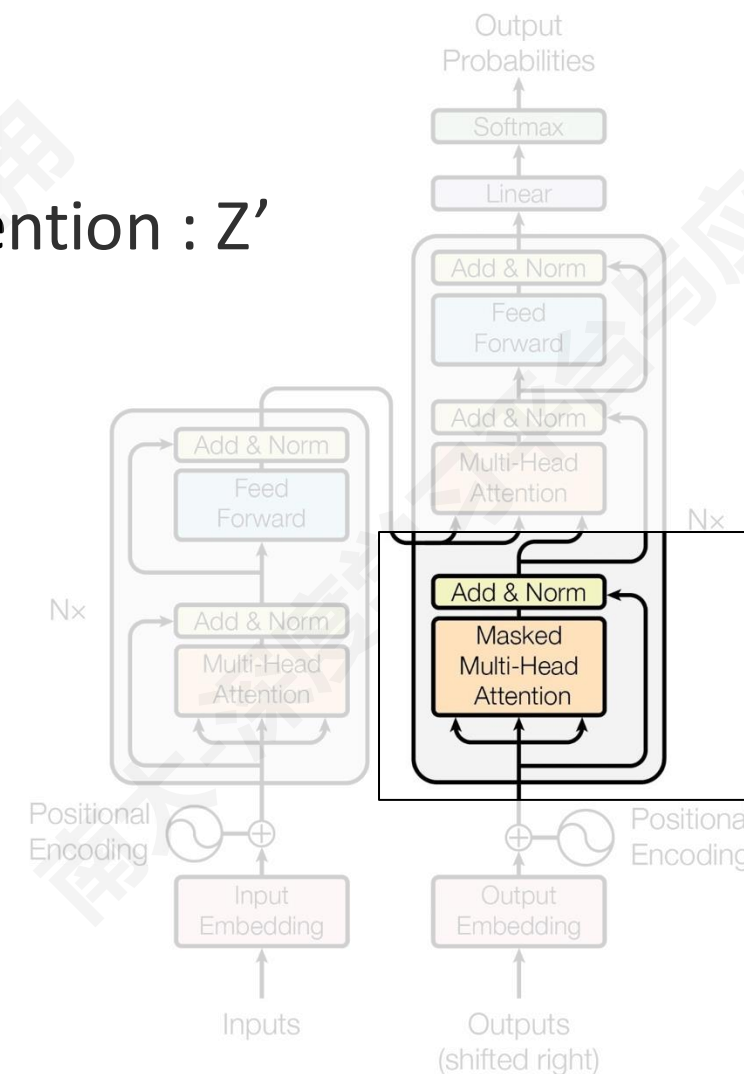
Softmax - ∞ -> 0



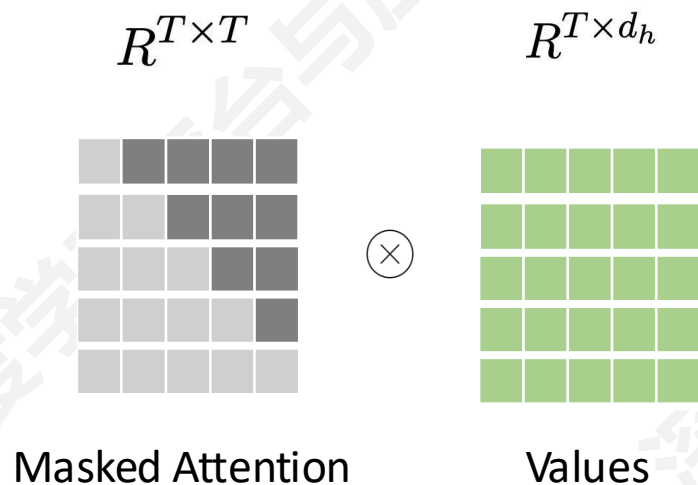
■ Masked Multi Head Attention



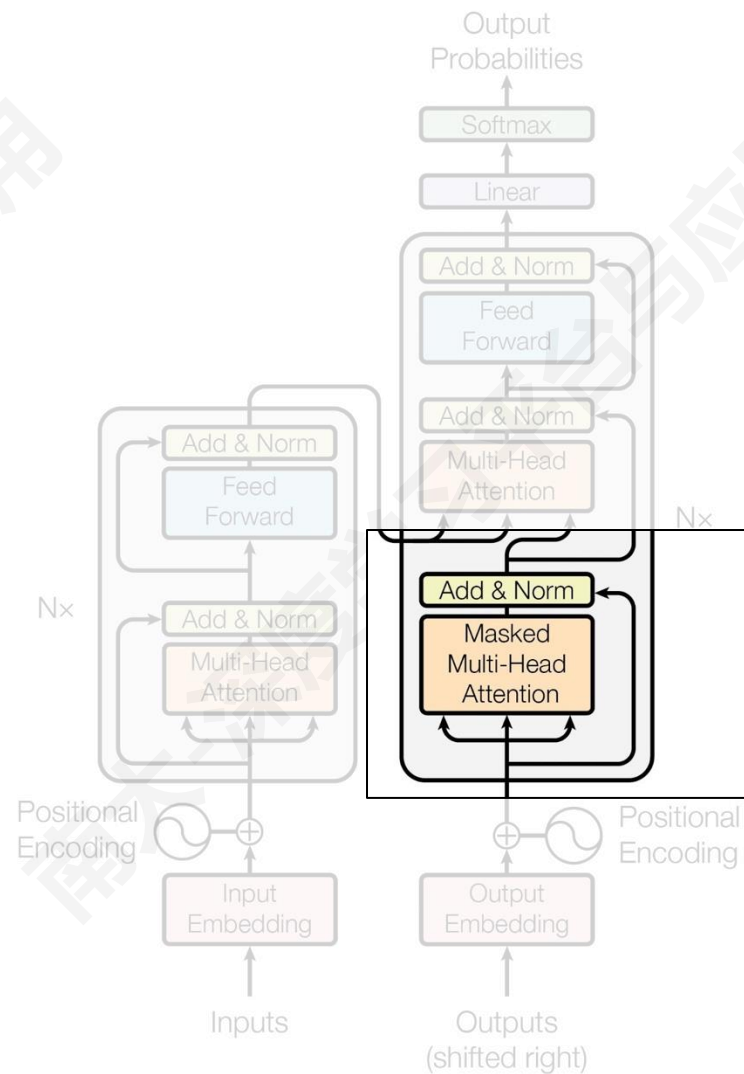
- **attn_mask** (*Optional*[*Tensor*]) – If specified, a 2D or 3D mask preventing attention to certain positions. Must be of shape (L, S) or $(N \cdot \text{num_heads}, L, S)$, where N is the batch size, L is the target sequence length, and S is the source sequence length. A 2D mask will be broadcasted across the batch while a 3D mask allows for a different mask for each entry in the batch. Binary and float masks are supported. For a binary mask, a **True** value indicates that the corresponding position is not allowed to attend. For a float mask, the mask values will be added to the attention weight. If both `attn_mask` and `key_padding_mask` are supplied, their types should match.



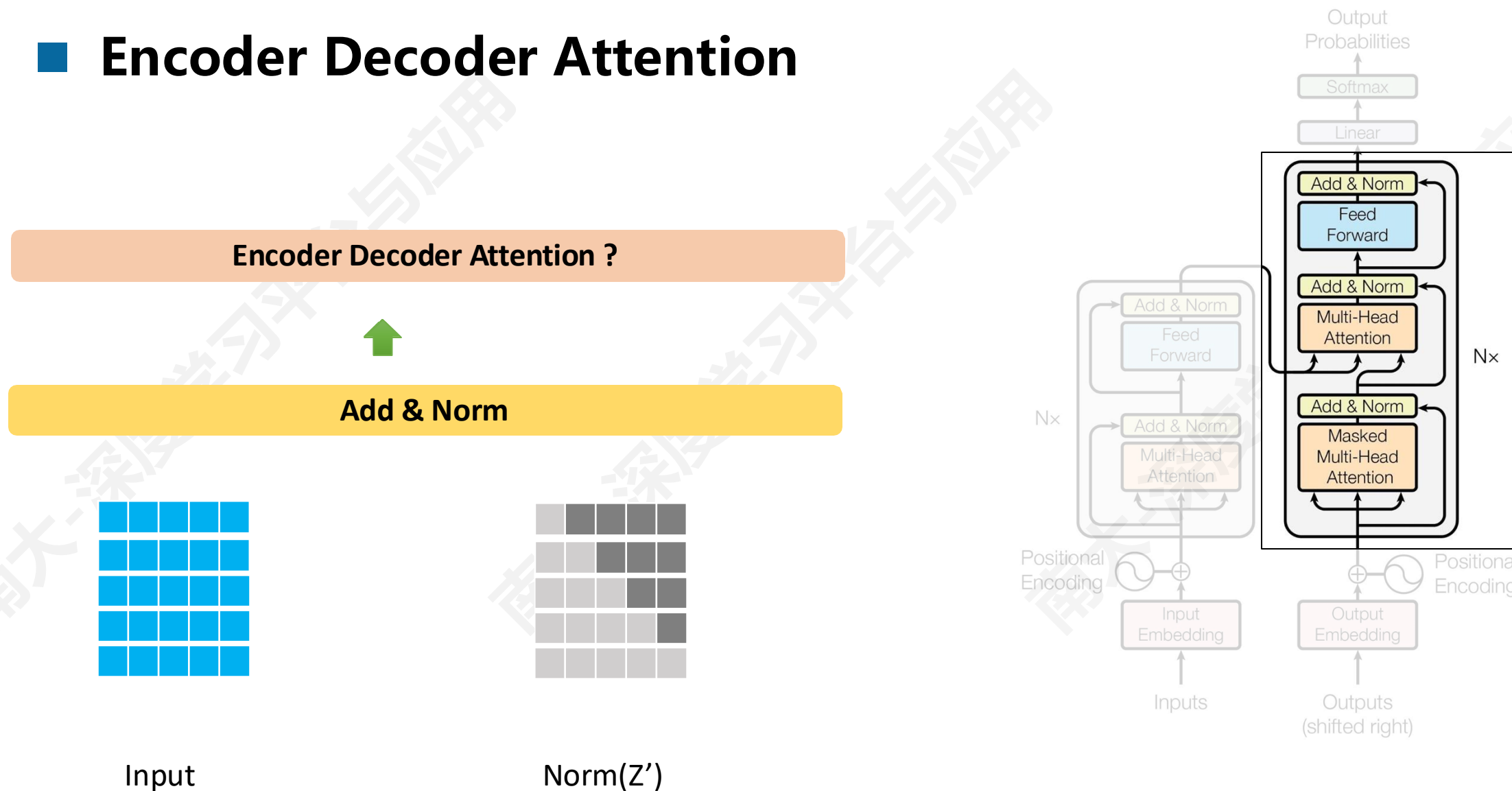
■ Masked Multi Head Attention



Masked Multi Head Attention : Z'

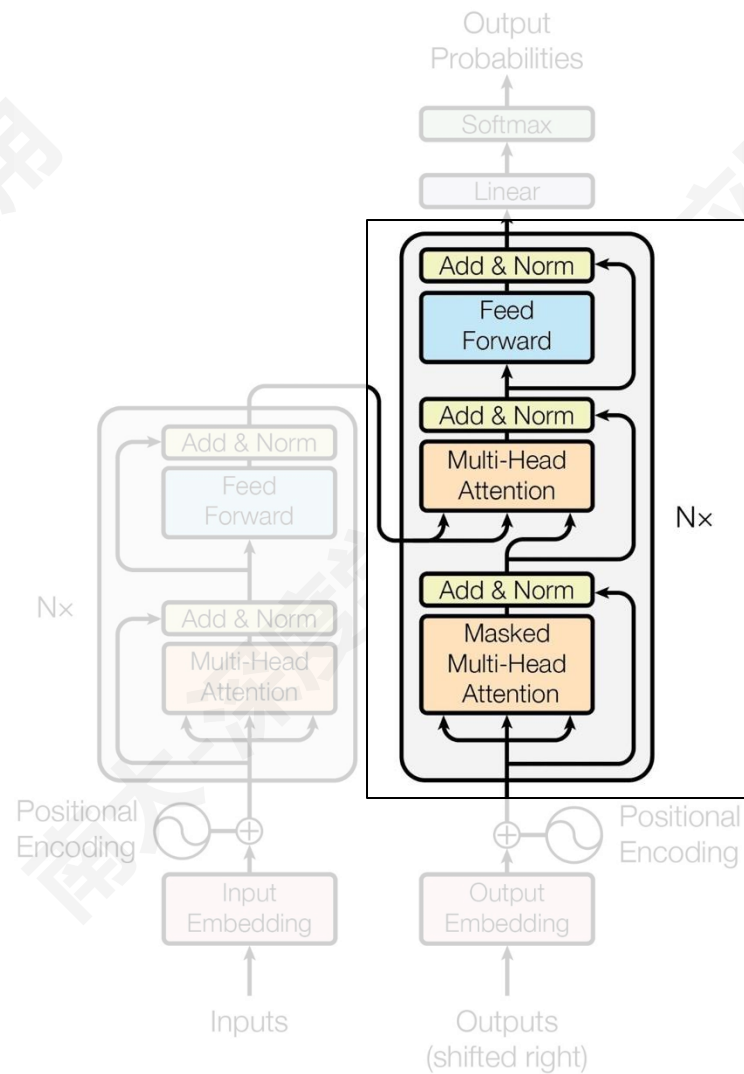


Encoder Decoder Attention

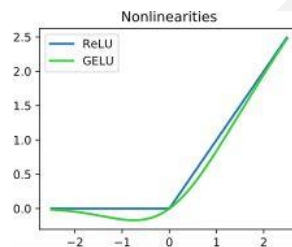


- Encoder Decoder Attention
- Key, Value, Query

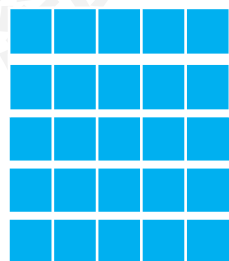
Encoder Decoder Attention ?



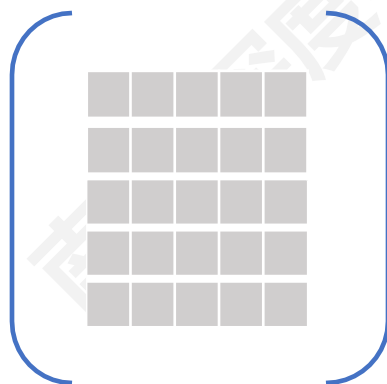
Encoder Decoder Attention



Feed Forward

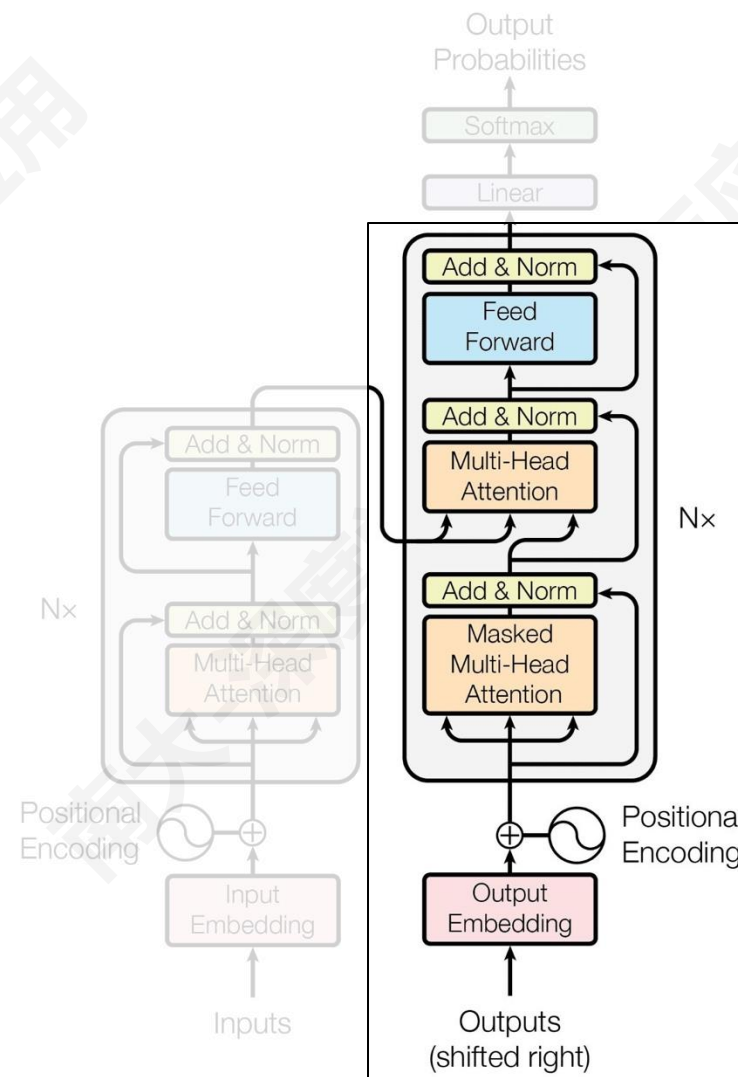


Residuals

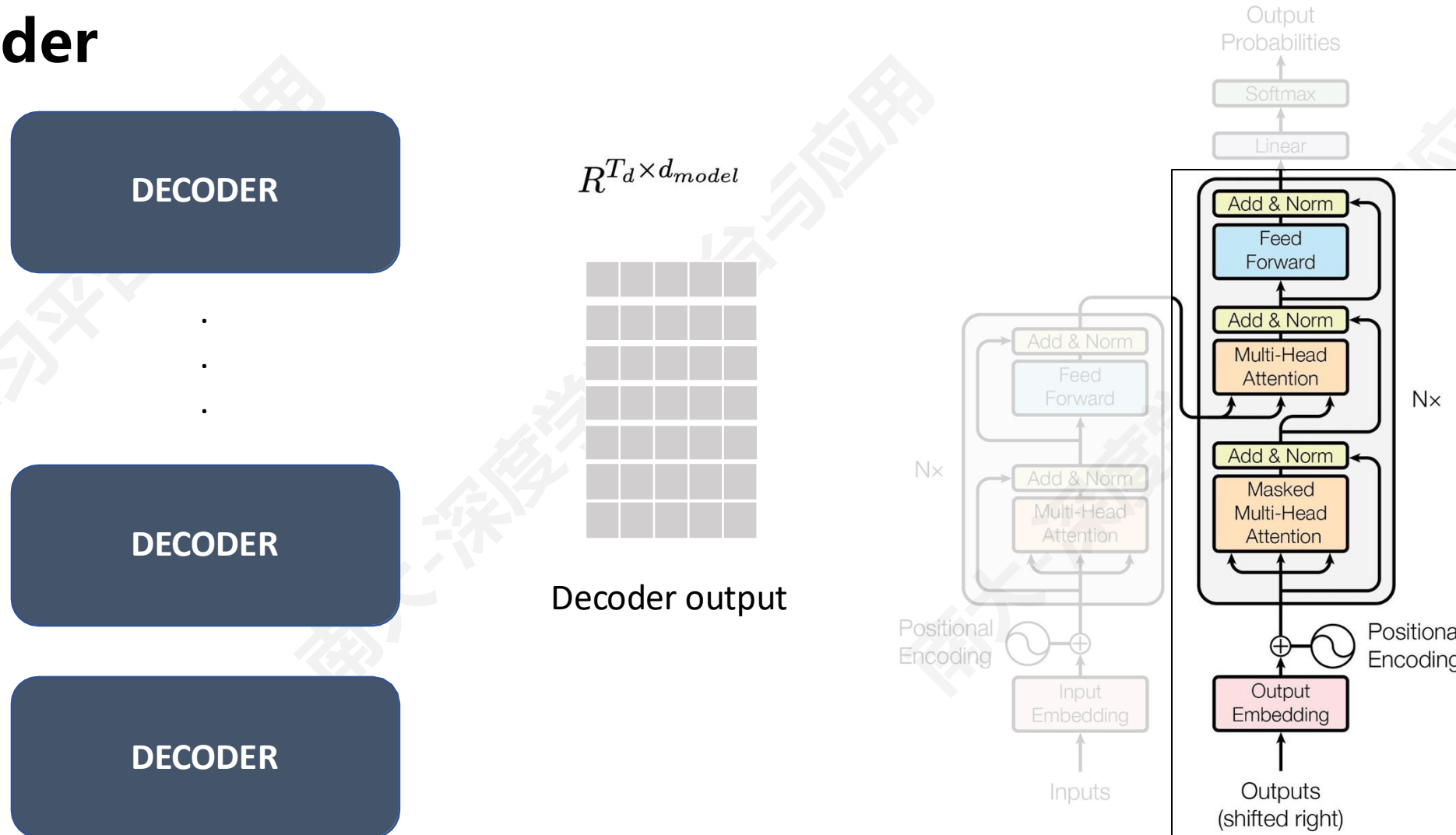


$\text{Norm}(Z'')$

Add n Norm Decoder Self Attn

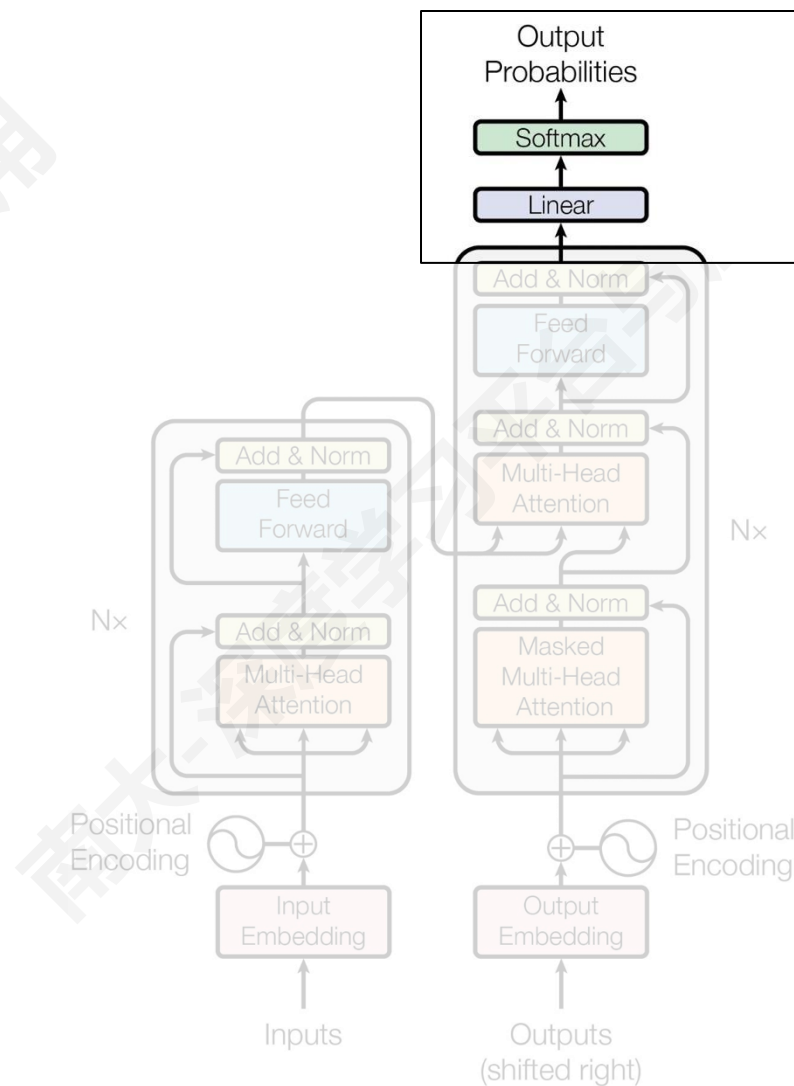
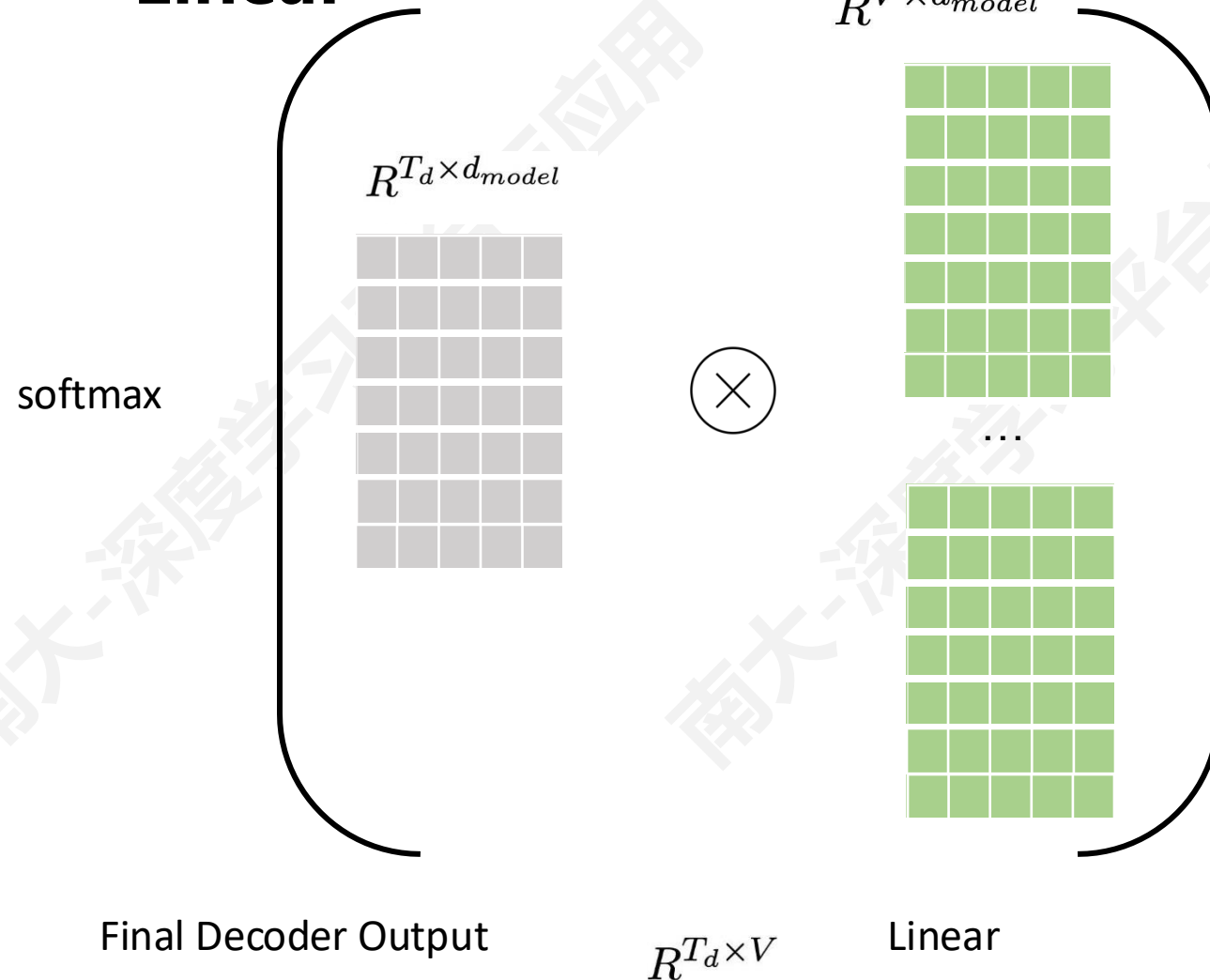


■ Decoder

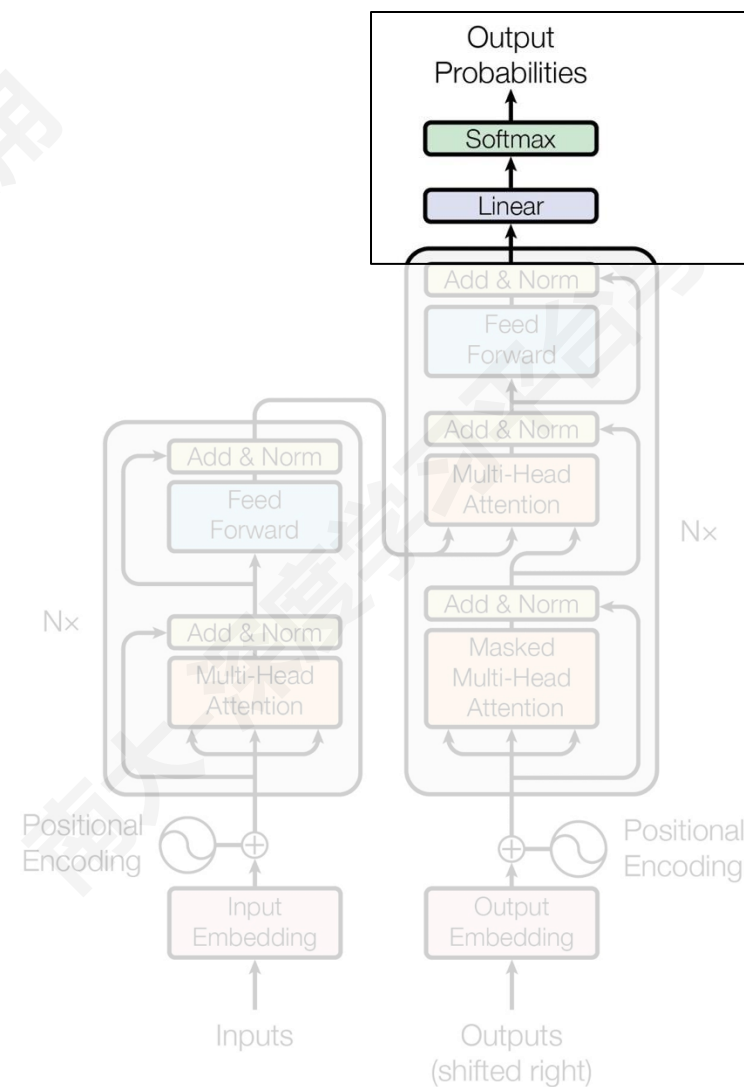
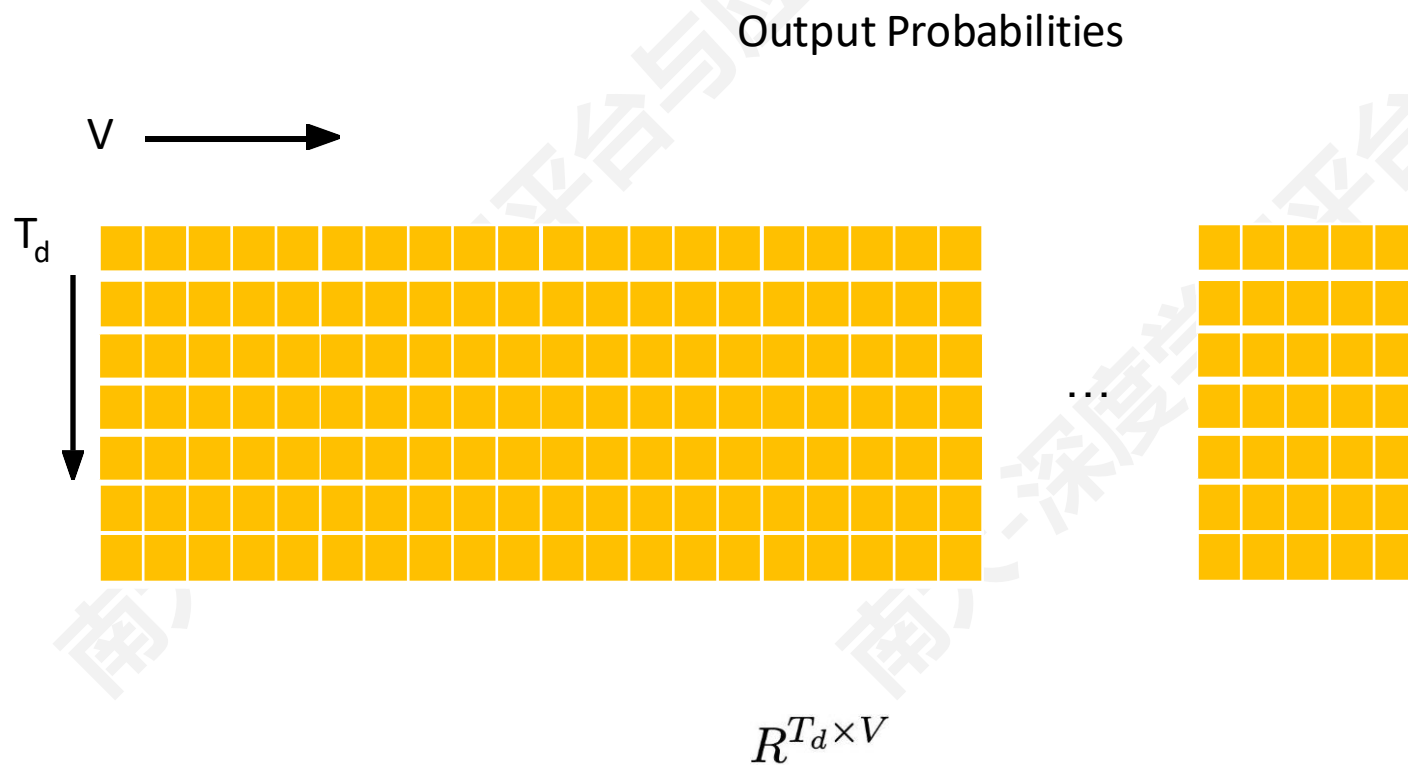


Transformer

■ Linear

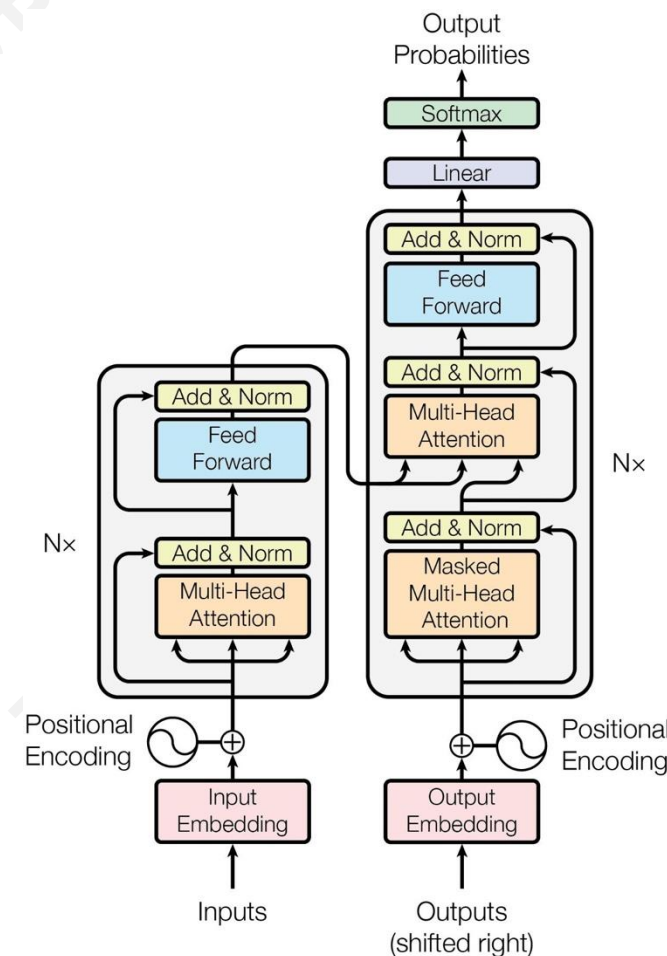


■ Softmax



Transformer

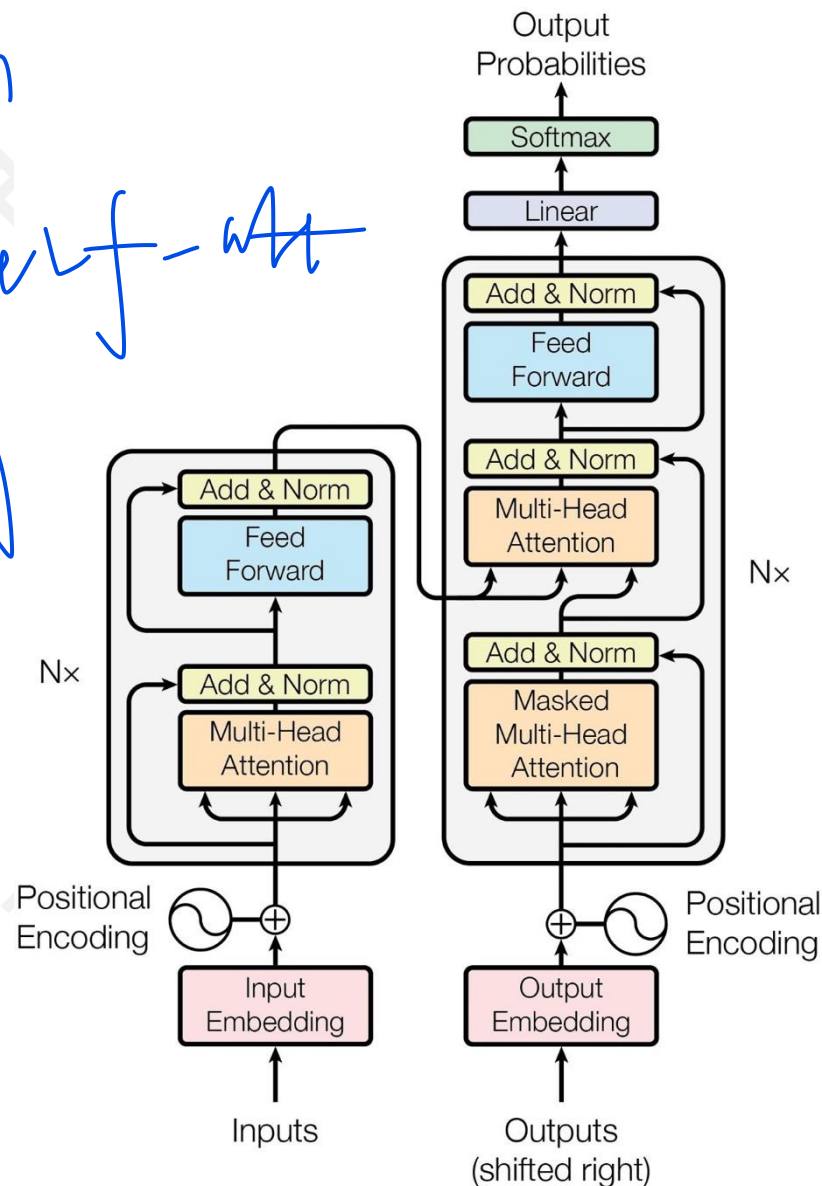
- ✓ Tokenization
- ✓ Input Embeddings
- ✓ Position Encodings
- ✓ Query, Key, & Value
- ✓ Attention
- ✓ Self Attention
- ✓ Multi-Head Attention
- ✓ Feed Forward
- ✓ Add & Norm
- ✓ Encoders
- ✓ Masked Attention
- ✓ Encoder Decoder Attention
- ✓ Linear
- ✓ Softmax
- ✓ Decoders
 - Encoder-Decoder Models



Transformer

fc 共负责每个 token

token 负责 self-att

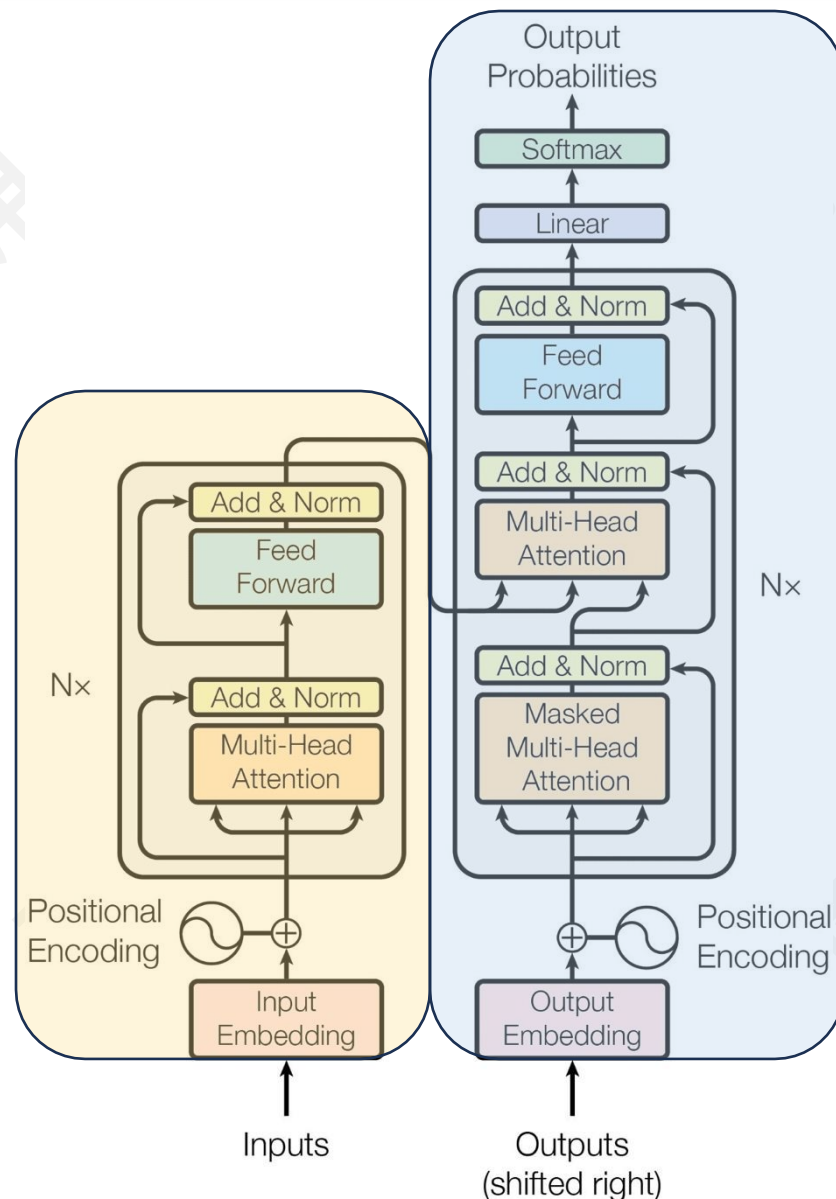


Transformer

Input – input tokens
Output – hidden states

模型可以看到所有时间步

Representation



Input – output tokens and hidden states*

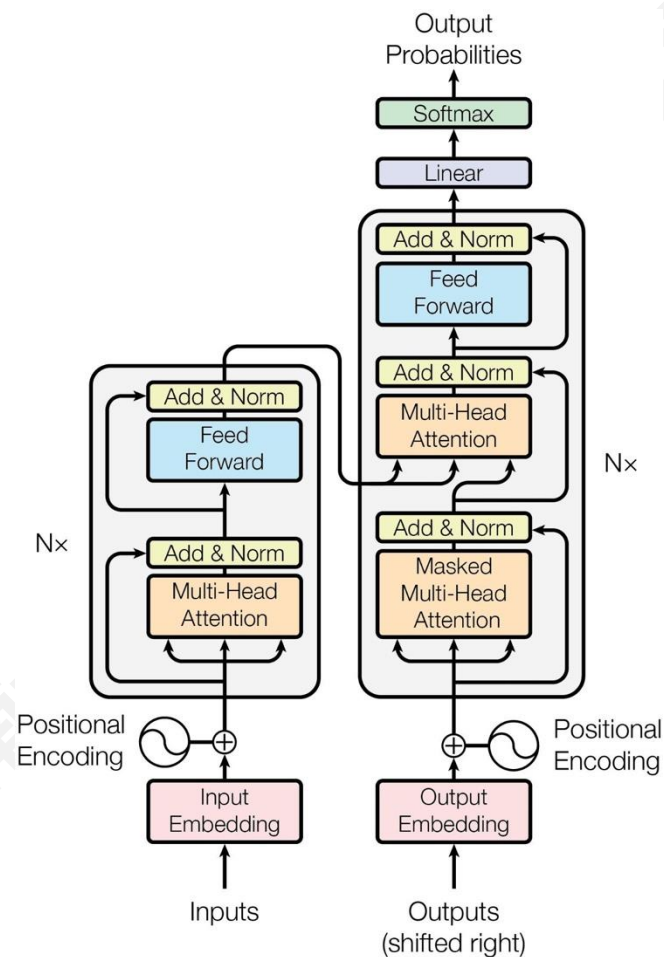
Output – output tokens

模型只能看到之前的时间步

Generation

■ 图像分类

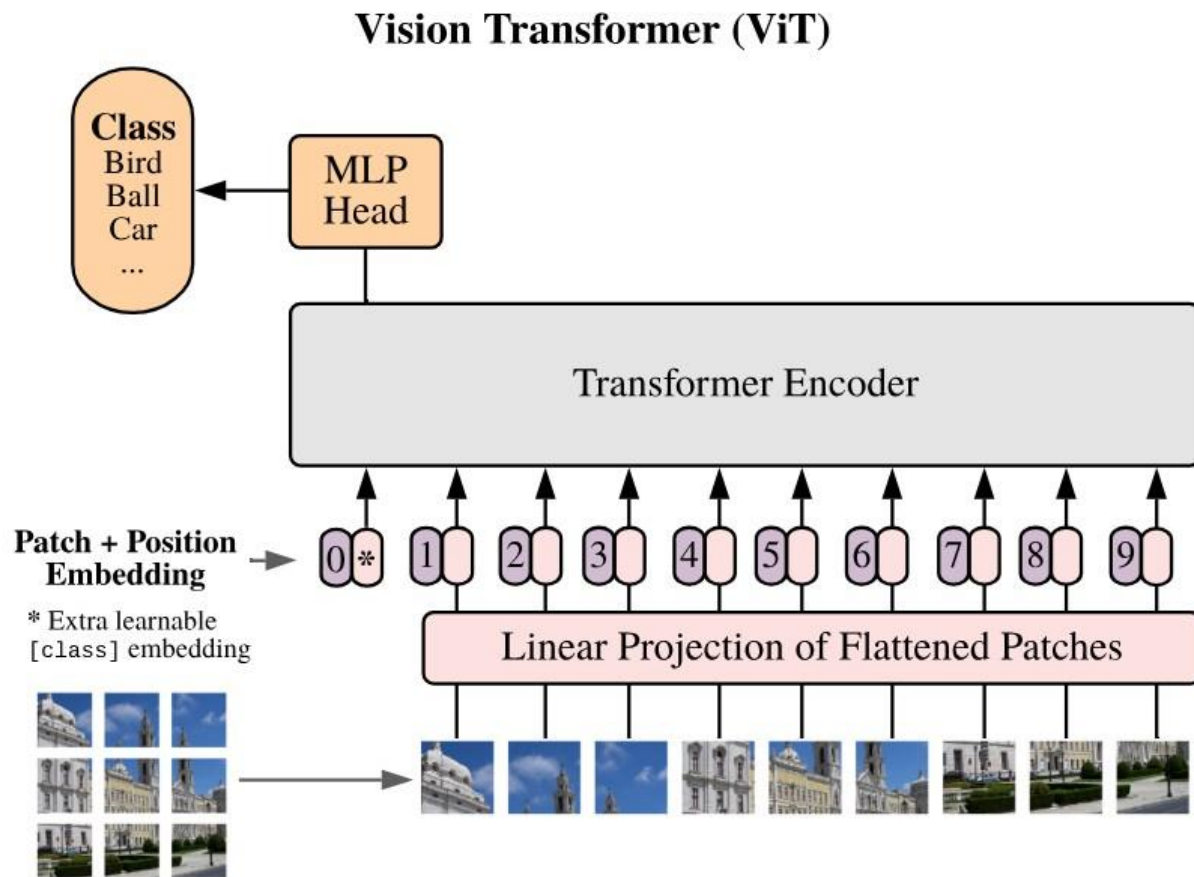
- ✓ Tokenization
- ✓ Input Embeddings
- ✓ Position Encodings
- ✓ Query, Key, & Value
- ✓ Attention
- ✓ Self Attention
- ✓ Multi-Head Attention
- ✓ Feed Forward
- ✓ Add & Norm
- ✓ Encoders
- ✓ Masked Attention
- ✓ Encoder Decoder Attention
- ✓ Linear
- ✓ Softmax
- ✓ Decoders
- ✓ Encoder-Decoder Models



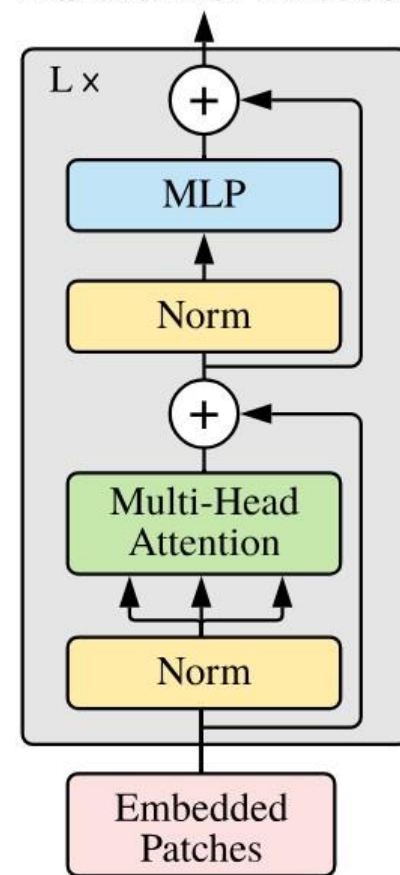
■ 视觉 ViT



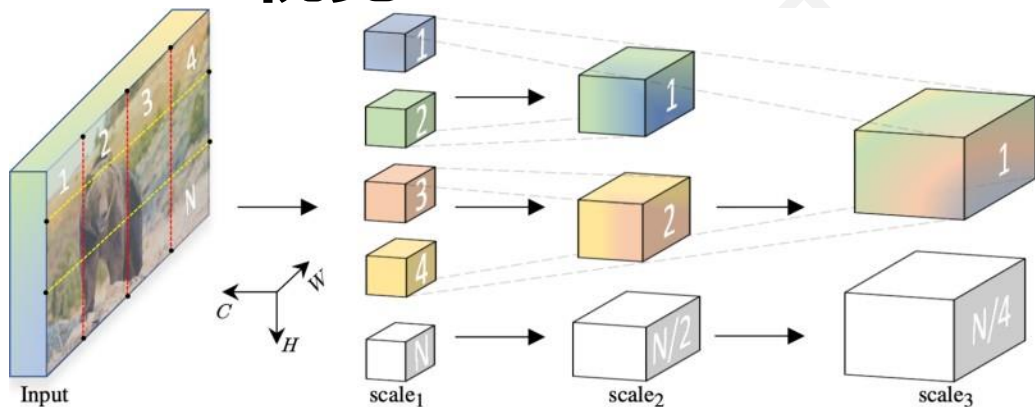
■ 视觉 transformer: ViT



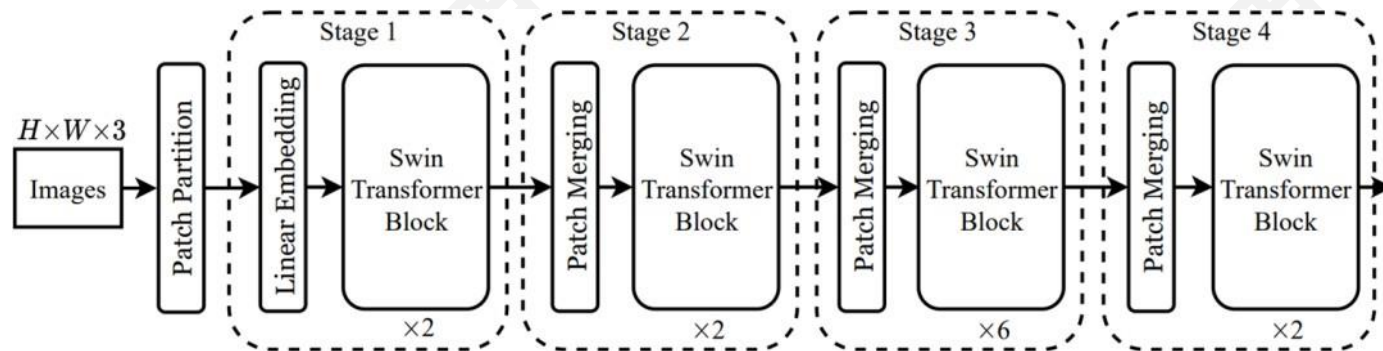
Transformer Encoder



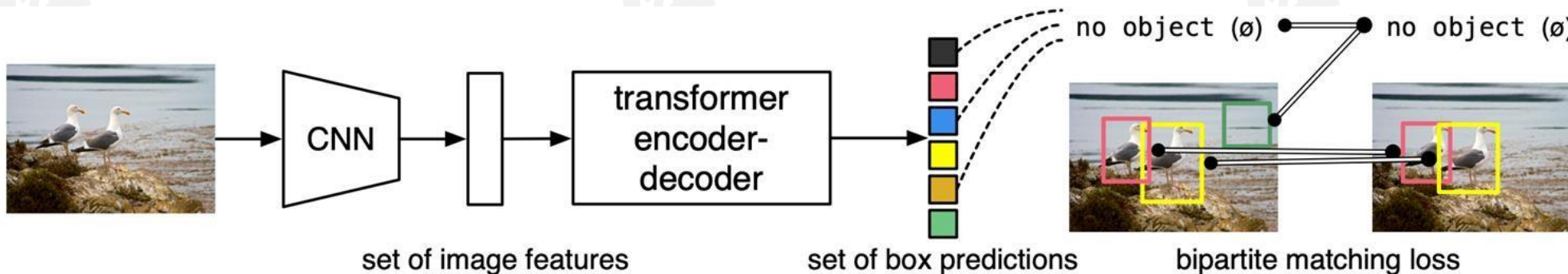
视觉 transformer: ViT



Fan et al, "Multiscale Vision Transformers", ICCV 2021

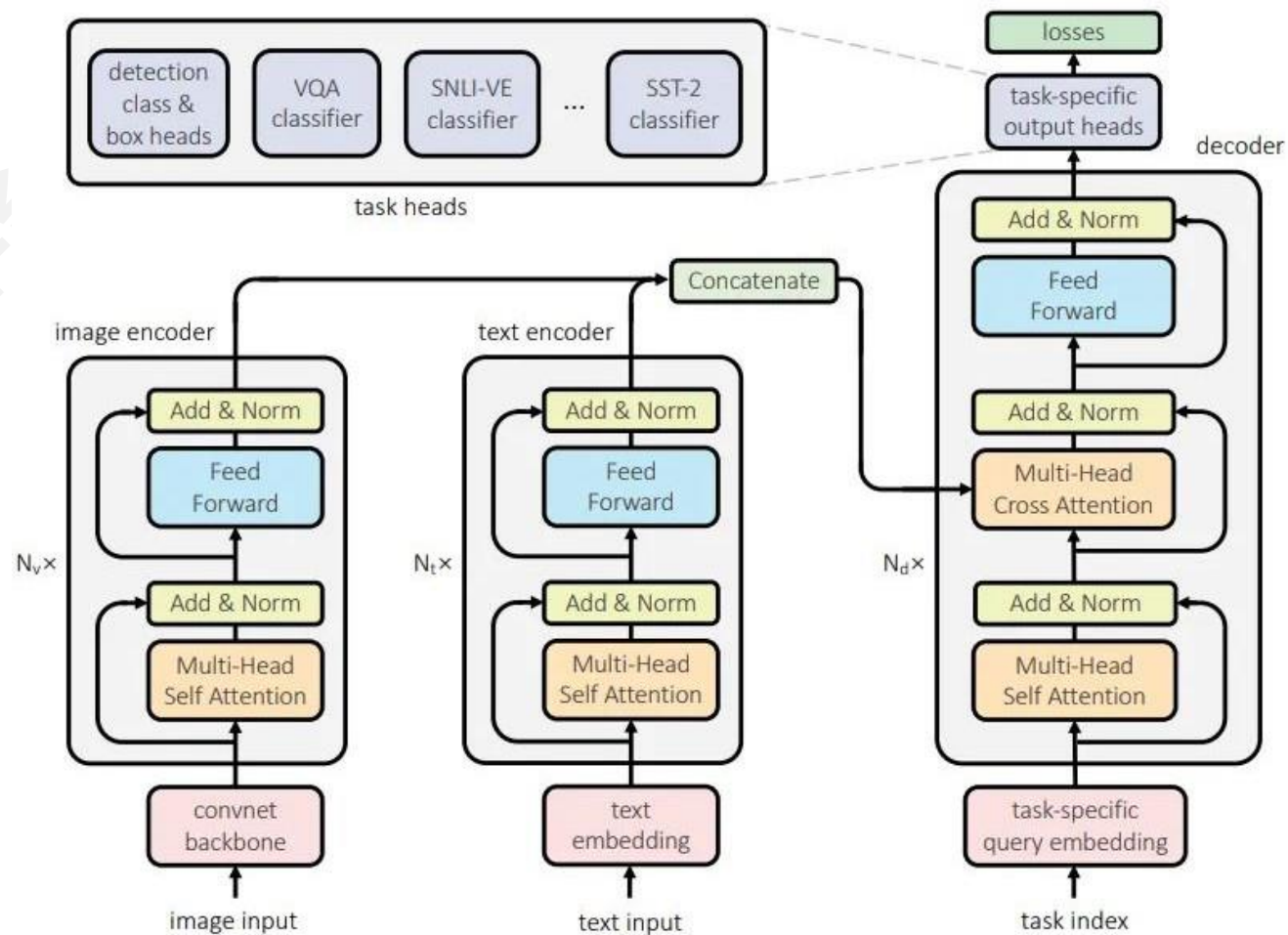


Liu et al, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows", CVPR 2021



Carion et al, "End-to-End Object Detection with Transformers", ECCV 2020

■ 多模态 transformer



■ Parameter-Efficient Finetuning

